Traitement de données en temps réel

Chapitre 0 — Notions de base

Le vocabulaire et les idées simples dont on a besoin avant de parler de flux : donnée, événement, serveur, réseau, message, temps, système distribué, Docker.

La méthode, en trois temps

C'EST QUOI ?	OÙ & QUAND ?	COMMENT ?
L'énoncé de la règle, son explication, le problème qu'elle traite et d'où elle vient.	Les domaines d'application et les signaux qui, dans un énoncé, désignent cette règle.	La méthode pas à pas, puis des exercices entièrement corrigés.

Ce document contient : 14 notions, chacune traitée en 11 sections — 8 figures — 3 problèmes du réel résolus — 30 exercices de fin de cours (10 faciles, 10 moyens, 10 difficiles) et leur corrigé détaillé.

Master Data Science & Big Data · S3 · Parcours Big Data
Faculté des Sciences Ben M'Sick · Université Hassan II de Casablanca
D'après le cours de Pr. Hafsa OUCHRA · Année universitaire 2026–2027

Chapitre 00

Notions de base du temps réel

Sommaire

Ouverture

1 · Objectifs du cours 4

2 · Un problème du réel : la seconde de Salma 5

3 · Introduction au cours 5

Les quatorze notions

N1 · Le traitement en temps réel 6

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N2 · Le traitement par lots et le traitement en flux 14

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N3 · La donnée 22

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N4 · Où ranger les données : fichier, base, entrepôt, lac 30

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N5 · La requête bornée et la requête continue 37

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N6 · Le client et le serveur 43

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N7 · Le réseau 50

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N8 · Le message JSON, ou l'événement 58

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N9 · La paire clé–valeur 66

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N10 · L'horodatage et les unités de temps 73

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N11 · Heure d'événement et heure de traitement 80

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N12 · Le système distribué et le cluster 88

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N13 · Le conteneur Docker 96

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

N14 · Le générateur Python 103

4 Règle · 5 Explication · 6 Problème traité · 7 Utilité · 8 Origine · 9 Où · 10 Quand · 11 Comment · 12 Applications corrigées · 13 Entraînement · 14 Difficulté croissante

Fin du cours

15 · Problèmes du réel et leur solution détaillée 110

16 · Banque d'exercices : 10 faciles, 10 moyens, 10 difficiles 115

17 · Corrigé de tous les exercices de la section 16 119

1. Objectifs du cours

Objectifs · à la fin de ce chapitre

À la fin de ce chapitre 0, vous devez être capable de :

- expliquer avec vos mots ce qu'est un **traitement en temps réel** et le distinguer d'un traitement par lots, sur n'importe quelle situation qu'on vous présente ;
- nommer correctement une **donnée**, un **enregistrement**, un **champ**, et dire où ces données sont rangées (fichier, base, entrepôt, lac) ;
- lire et écrire un **message JSON**, y repérer la **clé**, la **valeur** et l'**horodatage** ;
- distinguer un **client** d'un **serveur**, et décrire ce qui arrive à un message quand il traverse un **réseau** ;
- convertir des **unités de temps** (s, ms, µs) et calculer une **latence** à partir de deux horodatages ;
- expliquer la différence entre **heure d'événement** et **heure de traitement**, et dire pourquoi elle pose problème ;
- justifier l'emploi de **plusieurs machines** (cluster) et citer les trois problèmes que cela crée ;
- expliquer ce qu'apporte un **conteneur Docker** dans un TP ;
- écrire un petit **générateur Python** qui imite un flux sans fin.

2. Un problème du réel

Domaine · Monétique · Casablanca

Salma paie un café 18 dirhams avec sa carte bancaire, au Café Atlas, à 8 h 30 min 12 s. Trois secondes plus tard, le terminal affiche *Paiement accepté* et elle part travailler.

Pendant ces trois secondes, la banque a dû répondre à une question difficile : **ce paiement est-il une fraude ?** Pour le savoir, il faut comparer ce paiement aux habitudes de Salma, vérifier qu'elle n'a pas déjà payé ailleurs à la même minute, regarder si le commerce est suspect.

Le problème est le suivant : **la banque ne peut pas attendre**. Si elle fait ce calcul le lendemain matin, comme on calcule un relevé de compte, la réponse arrive trop tard : soit elle a laissé passer une fraude, soit elle a bloqué une cliente honnête qui est déjà partie. La valeur de la réponse **s'effondre avec le temps**.

Et ce problème n'est pas seulement bancaire. Un détecteur de fumée qui prévient demain est inutile. Un GPS qui recalcule l'itinéraire dix minutes après la sortie ratée est inutile. Une pompe industrielle dont on détecte la dérive de vibration une semaine après la casse est inutile.

3. Introduction au cours

La situation de Salma nous apprend quelque chose de simple : **il existe des décisions qui doivent être prises pendant que le phénomène se produit, pas le lendemain matin.** Tout le module « Traitement de données en temps réel » est construit autour de cette phrase.

Mais avant de parler de Kafka, de Flink ou de fenêtres glissantes, il faut d'abord poser le vocabulaire. Quand on dit « un message voyage du terminal vers la banque en quelques millisecondes », on a déjà employé six notions : *message*, *voyage* (réseau), *terminal* (client), *banque* (serveur), *milliseconde* (unité de temps), et implicitement l'*horodatage* qui permet de mesurer ce voyage. Si l'une de ces notions reste floue, tout le reste du module le restera aussi.

Ce chapitre 0 pose donc, une par une, les **quatorze notions** dont nous aurons besoin en permanence. Chacune est traitée complètement : la règle elle-même, son explication, le problème qu'elle résout, son origine, où, quand et comment l'utiliser, puis des exercices corrigés. Huit figures animées accompagnent le texte : elles ne sont pas décoratives, elles montrent le mouvement que les mots ont du mal à décrire.

Nous ne verrons ici **aucune technologie**. Kafka, Flink, Spark et River viendront aux chapitres 1 à 6. Ici, nous apprenons seulement à parler correctement de ce qu'ils font.

N1 — Le traitement en temps réel

▶ 4. La règle

Règle · Traitement en temps réel

Le traitement en temps réel consiste à traiter chaque donnée dès son arrivée, suffisamment rapidement pour pouvoir prendre une décision et agir au bon moment.

Formellement, si un fait se produit à l'instant t_e et que le résultat du traitement est disponible à l'instant t_r, on appelle **latence** la durée $L = t_r - t_e$. Le traitement est dit en temps réel lorsque $L \leq D$, où D est le **délai utile** imposé par le métier : le temps au-delà duquel la réponse n'a plus de valeur.

Conditions d'emploi. La règle n'a de sens que si trois choses sont vraies : (1) les données arrivent de façon continue et non en un seul bloc ; (2) il existe une action à déclencher à partir du résultat ; (3) le délai utile D est fini et connu, au moins approximativement.

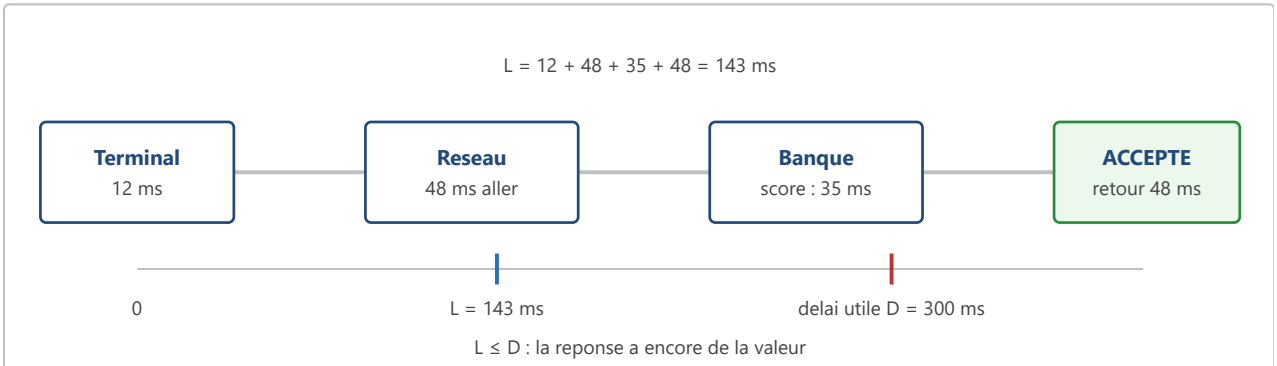


FIGURE 1 · LA SECONDE DE SALMA

L'événement est créé par le terminal, traverse le réseau, reçoit un score à la banque, et la décision revient. La latence totale L = 143 ms reste sous le délai utile D = 300 ms : la réponse a encore de la valeur. Si le réseau ralentit à 160 ms par trajet, L monte à 367 ms et le paiement est refusé pour dépassement de délai.

▶ 5. Explication de la règle

Cette règle dit une chose que beaucoup d'étudiants comprennent de travers au départ : **le temps réel n'est pas une question de vitesse absolue, c'est une question de délai utile**. Il n'existe pas de seuil universel à 10 ms ou à 1 seconde. Pour une autorisation de carte bancaire, D vaut quelques centaines de millisecondes. Pour un tableau de bord de supervision réseau, D vaut quelques secondes. Pour un système de freinage automobile, D vaut quelques millisecondes. Les trois sont du temps réel.

L'idée profonde derrière la règle est celle de la **décroissance de la valeur**. Une information vaut cher à l'instant où elle naît, puis sa valeur décroît : savoir qu'un incendie a commencé il y a dix secondes permet d'éteindre ; savoir qu'un incendie a commencé il y a dix heures permet seulement de constater. Le temps réel consiste à capturer la valeur avant qu'elle ne s'évapore.

C'est pour cela que la règle est formulée avec *deux* quantités et non une seule. La latence `L` est une propriété de votre système : vous pouvez la mesurer, l'améliorer, l'optimiser. Le délai utile `D` est une propriété du métier : c'est le client, le régulateur ou la physique qui l'imposent, et vous ne pouvez pas le négocier. Concevoir un système temps réel, c'est d'abord aller chercher `D` auprès du métier, puis construire une chaîne dont `L` reste en dessous.

Cas particuliers et pièges

- **Temps réel dur et temps réel souple.** En temps réel *dur* (airbag, pilotage d'avion), dépasser `D` une seule fois est une faute grave : le système a échoué. En temps réel *souple*, qui est le cas de presque tout ce module, dépasser `D` de temps en temps dégrade seulement la qualité du service. Ce module traite du temps réel souple.
- **Ne confondez pas latence et débit.** Un système peut traiter un million de messages par seconde (débit énorme) en mettant trente secondes à traiter chacun (latence énorme). Les deux grandeurs sont indépendantes.
- **La moyenne ment.** On ne juge jamais un système temps réel sur la latence moyenne, mais sur les *percentiles hauts* (p95, p99). Une moyenne de 20 ms avec un p99 à 4 s signifie qu'un paiement sur cent échoue.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui de la **décision tardive**. Avant cette règle, l'informatique de gestion traitait les données en les accumulant puis en les calculant à heure fixe. Ce modèle fonctionne parfaitement pour une paie ou un bilan comptable, où le résultat est attendu à une date précise et où rien ne presse entre-temps.

Il échoue dès que le résultat sert à *agir sur le phénomène lui-même*. On ne peut pas bloquer une fraude après l'autorisation, ni fermer une vanne après la rupture, ni recommander un film après que l'utilisateur a quitté l'application. La règle du temps réel formalise exactement cette situation et donne le critère qui permet de la reconnaître : $L \leq D$.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous donne d'abord un **critère de décision architecturale**. Devant n'importe quel cahier des charges, on peut maintenant poser une seule question — « quel est le délai au-delà duquel cette réponse ne sert plus à rien ? » — et la réponse décide seule entre un traitement par lots, simple et peu cher, et une chaîne de streaming, plus complexe et plus coûteuse.

Elle nous donne ensuite un **budget à répartir**. Puisque `L` est la somme des latences de tous les maillons (capture, réseau, file de messages, calcul, réponse), connaître `D` permet d'allouer à chaque maillon sa part. C'est exactement ce que nous ferons au chapitre 5 lorsque nous parlerons de performance : on ne dira plus « c'est lent », on dira « le maillon Kafka consomme 60 % du budget ».

Enfin elle nous permet de **refuser le temps réel quand il est inutile**. Beaucoup de projets coûtent cher parce qu'on a mis du streaming là où un traitement nocturne suffisait. Savoir formuler `D` est aussi un outil d'économie.

▶ 8. Comment on y est arrivé

Origine de la règle

Historiquement, les premiers systèmes informatiques travaillaient par *cartes perforées* : on déposait un paquet de travail, la machine le traitait la nuit, on récupérait les résultats le matin. Le mot *batch* vient littéralement de ce paquet. Le modèle était imposé par le matériel : la machine était rare et chère, il fallait la saturer.

Deux évolutions ont renversé la contrainte. D'une part les **sources de données sont devenues continues** : capteurs, terminaux de paiement, clics, journaux applicatifs produisent sans interruption, et il n'y a plus de « paquet » naturel. D'autre part le **coût du calcul s'est effondré** face au coût d'une mauvaise décision : une fraude non bloquée coûte plus cher que les serveurs qui l'auraient détectée.

La règle est donc une **généralisation d'une observation économique**, et non un théorème. On la justifie ainsi : notons `V(L)` la valeur d'une réponse obtenue avec une latence `L`. L'observation empirique, dans tous les domaines cités, est que `V` est décroissante et qu'elle possède un point d'effondrement. On appelle `D` ce point. Traiter en temps réel, c'est rester à gauche de `D` ; cela n'a de sens que parce que `V` s'effondre, et c'est pourquoi la règle exige qu'une action soit attachée au résultat.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Monétique et lutte contre la fraude.** Chaque autorisation de paiement est un événement qu'il faut scorer avant de répondre au terminal. Le délai utile est imposé par les réseaux de cartes et par la patience du client : quelques centaines de millisecondes. C'est le domaine où D est le plus clairement contractualisé.
- **Cybersécurité et supervision.** On surveille en continu les connexions, les tentatives d'authentification, les volumes sortants. Ici le délai utile se compte en secondes ou en minutes : l'objectif est de contenir une attaque avant qu'elle ne se propage, pas de répondre à l'utilisateur.
- **Industrie et maintenance prédictive.** Des capteurs de vibration, de température ou de pression produisent des mesures à cadence fixe. Le délai utile est celui qui sépare la dérive détectable de la casse effective : souvent quelques minutes, parfois quelques heures.
- **Recommandation et expérience utilisateur.** Tenir compte de ce que l'utilisateur vient de regarder n'a de valeur que pendant sa session. Le délai utile est la durée de la session, soit quelques minutes.
- **Mobilité et logistique.** Recalcul d'itinéraire, affectation d'un chauffeur, estimation d'arrivée. Le délai utile est le temps dont dispose l'usager pour changer de comportement, typiquement quelques secondes.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Quand l'énoncé contient un verbe d'action immédiate.** « Bloquer », « alerter », « arrêter », « proposer pendant », « refuser » indiquent qu'une décision est attachée au résultat. Un énoncé qui dit seulement « analyser » ou « produire un rapport » ne demande pas de temps réel.
- **Quand une durée apparaît dans le cahier des charges.** « En moins de 200 ms », « dans la minute », « avant la fin de la session » sont des écritures directes de D. Dès qu'une telle durée est écrite, la règle s'applique et il faut vérifier $L \leq D$.
- **Quand la source ne s'arrête jamais.** Capteurs, journaux, clics, transactions : s'il n'existe aucun instant où l'on peut dire « les données sont complètes », le traitement par lots devient artificiel et la règle s'impose.
- **Quand on hésite, posez la question du lendemain.** « Si je donne cette réponse demain matin, est-ce qu'elle sert encore à quelque chose ? » Si la réponse est oui, ce n'est pas du temps réel, et il ne faut pas en faire.

▶ 11. Comment on va l'utiliser

Méthode · qualifier une situation

1. **Identifier le fait et son instant t_e.** On nomme précisément l'événement déclencheur (« la carte est présentée au terminal ») et le moment où il se produit. Sans fait daté, il n'y a pas de latence à mesurer.
2. **Identifier l'action et son instant limite.** On demande : *que fait-on du résultat ?* S'il n'y a aucune action, on s'arrête ici : c'est de l'analyse, pas du temps réel.
3. **Écrire `D` explicitement, avec son unité.** On note par exemple `D = 300 ms`. Si le métier ne sait pas répondre, on propose une valeur et on la fait valider ; un `D` implicite est la première cause d'échec d'un projet.
4. **Décomposer `L` en maillons.** On liste la chaîne : capture, transport aller, file d'attente, calcul, transport retour. Chaque maillon reçoit une estimation.
5. **Comparer et conclure.** On somme les maillons, on compare à `D`. Si `L ≤ D` avec une marge, la conception tient ; sinon on désigne le maillon dominant et c'est lui qu'on optimise, pas les autres.

À retenir

Le temps réel se justifie quand la valeur de la réponse diminue vite avec le temps : on compare toujours une latence mesurée `L` à un délai utile `D` imposé par le métier.

▶ 12. Exercices d'application corrigés

Application 1.1

Un détecteur de fumée d'un entrepôt détecte un départ de feu à `08:30:12.000`. L'alarme sonore se déclenche à `08:30:12.900`. Le service sécurité indique qu'au-delà de 3 secondes, le feu atteint les palettes voisines et l'évacuation devient dangereuse. S'agit-il d'un traitement en temps réel ?

Corrigé 1.1

Étape 1 — identifier le fait et son instant. Le fait est « un départ de feu est détecté », à t_e = 08:30:12.000.

Étape 2 — identifier l'action. L'action est le déclenchement de l'alarme, qui provoque l'évacuation. Il y a bien une action attachée au résultat, donc la règle s'applique.

Étape 3 — écrire D. Le métier donne D = 3 s = 3000 ms.

Étape 4 — décomposer L. Ici la chaîne n'est pas détaillée, on dispose directement de l'instant du résultat : t_r = 08:30:12.900.

Étape 5 — comparer. L = 12.900 − 12.000 = 0,900 s = 900 ms. Or 900 ms ≤ 3000 ms, donc L ≤ D avec une marge confortable de 2100 ms.

Conclusion. Oui, c'est un traitement en temps réel, et même un cas de temps réel *dur* : dépasser D une seule fois met des vies en jeu.

Application 1.2

Une université calcule les moyennes du semestre. Les notes sont saisies par les enseignants pendant trois semaines ; le bulletin est publié le 15 février. Le service scolarité demande « un système temps réel, parce que c'est moderne ». Que répondez-vous, en appliquant la méthode ?

Corrigé 1.2

Étape 1 — le fait et son instant. Le fait est « un enseignant saisit une note ». Ces faits sont bien continus et datés, répartis sur trois semaines.

Étape 2 — l'action. C'est l'étape décisive. Que fait-on d'une moyenne calculée à l'instant où la dernière note est saisie ? Rien. Aucune décision n'est prise avant le 15 février, date administrative fixe. La condition (2) de la règle n'est pas satisfaite.

Étape 3 — écrire `D`. Le délai utile est la date de publication : `D` vaut plusieurs jours, voire plusieurs semaines, compté depuis la dernière saisie.

Étape 4 et 5 — décomposer et comparer. Même un traitement nocturne très lent donne `L` de l'ordre de quelques heures, très inférieur à `D`. Le critère `L ≤ D` est satisfait trivialement par un traitement par lots.

Conclusion. On répond que le temps réel est ici **inutile et coûteux** : un traitement par lots nocturne suffit. On applique le test du lendemain : « une moyenne donnée demain matin sert-elle encore ? » — oui, donc ce n'est pas du temps réel. On peut en revanche proposer un affichage « notes manquantes » en continu pour les enseignants : là, une action existe (relancer un collègue), et le temps réel redevient légitime.

▶ 13. Exercices d'entraînement

Entraînement

1. Un péage autoroutier lit une plaque minéralogique et doit lever la barrière avant que la voiture, qui roule à 30 km/h, n'arrive à 4 mètres de la barrière. Calculez `D` en millisecondes, puis dites si une chaîne de latence 350 ms convient.
2. Pour chacune de ces situations, écrivez le fait, l'action et une valeur plausible de `D` : (a) recommandation de vidéo ; (b) facturation mensuelle d'un opérateur téléphonique ; (c) alerte de dépassement de quota de données mobiles.
3. Un système affiche une latence moyenne de 40 ms, un p95 de 120 ms et un p99 de 2200 ms, pour `D = 300 ms`. Le système respecte-t-il la règle ? Sur combien de transactions par million échoue-t-il ?

▶ 14. Exercices de difficulté croissante

Progression

1. facile Un capteur mesure à `10:00:00.000`, le résultat est affiché à `10:00:00.250`. Donnez `L` en millisecondes.
2. moyen La chaîne est : capture 5 ms, réseau aller 45 ms, file d'attente 60 ms, calcul 30 ms, réseau retour 45 ms. Le métier impose `D = 200 ms`. La conception tient-elle ? Quel maillon optimiseriez-vous en premier et pourquoi ?
3. difficile Même chaîne, mais la file d'attente passe de 60 ms à 900 ms dès que le débit d'entrée dépasse 8000 messages/s, ce qui arrive 2 % du temps. Discutez : la règle `L ≤ D` est-elle respectée ? Reformulez le critère d'acceptation d'une façon qui tienne compte de ce comportement.

N2 — Le traitement par lots et le traitement en flux

▶ 4. La règle

Règle · Batch et streaming

Il existe deux façons de traiter l'information. Le traitement par lots (*batch*) accumule les données pendant une période, puis les traite toutes ensemble à la fin. Le traitement en flux (*streaming*) traite chaque donnée dès qu'elle arrive, sans jamais attendre les suivantes.

Soit un ensemble d'événements $e_1, e_2, \ldots$ arrivant aux instants $t_1 < t_2 < \ldots$. En batch de période P, le résultat portant sur l'intervalle $[kP, (k+1)P[$ est produit à l'instant $(k+1)P$: la latence d'un événement arrivé en début de période vaut presque P. En streaming, le résultat qui tient compte de e_i est produit à $t_i + \varepsilon$, où ε ne dépend pas de P.

Conditions d'emploi. Le batch suppose qu'il existe un instant où l'on peut dire « le lot est complet ». Le streaming suppose que le traitement de e_i ne nécessite pas de connaître e_{i+1}, ou qu'on accepte un résultat révisable.

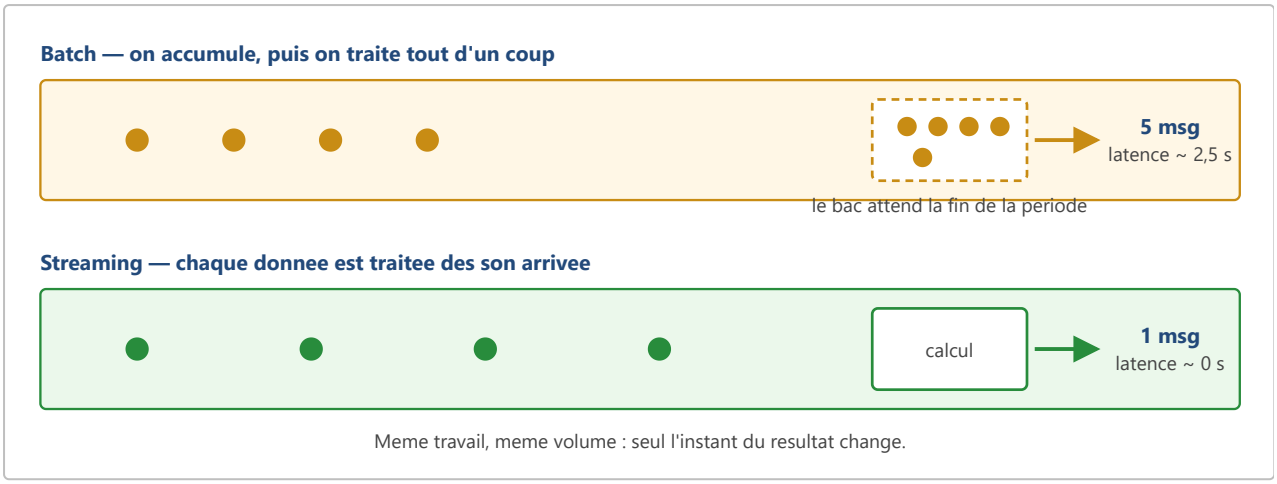


FIGURE 2 · DEUX FAÇONS DE TRAITER

Les mêmes événements arrivent dans les deux couloirs. En haut ils s'accumulent dans le bac et ne produisent un résultat qu'à la fin de la période ; en bas chacun traverse le calcul immédiatement. Le travail total est identique : seul l'instant du résultat change, et avec lui la latence.

▶ 5. Explication de la règle

La règle oppose deux façons d'organiser le travail, et l'image de la vaisselle la résume mieux qu'une formule : on peut empiler toutes les assiettes et laver une fois le repas terminé, ou laver chaque assiette juste après l'avoir utilisée. Le travail total est le même ; ce qui change, c'est *quand* le résultat est disponible et *combien* de choses on doit garder en attente.

L'idée profonde est un **échange entre latence et simplicité**. Le batch est simple parce qu'il travaille sur un ensemble *fini et complet* : on peut trier, compter, recommencer le calcul si on s'est trompé, tout est là. Le streaming est plus difficile parce qu'il travaille sur un ensemble *infini et incomplet* : on ne verra jamais la dernière donnée, donc on ne peut jamais dire « le calcul est terminé ». En échange, le streaming donne un résultat tout de suite.

Une conséquence importante : en batch, la latence est **imposée par la période, pas par la vitesse de la machine**. Un lot quotidien reste quotidien même si le calcul ne dure que trois secondes. Accélérer la machine ne change rien ; seul réduire `P` change quelque chose, et quand `P` devient très petit, on a en réalité construit un streaming maladroit, qu'on appelle *micro-batch*.

Cas particuliers et pièges

- **Le micro-batch.** Spark Structured Streaming traite par tout petits paquets (quelques centaines de millisecondes). C'est un cas intermédiaire : le modèle de programmation est celui du batch, le comportement observé est proche du streaming. L'image donnée en cours est « un bus qui part toutes les deux minutes ».
- **Batch ne veut pas dire lent à calculer.** Un lot peut traiter des milliards de lignes très vite. Ce qui est lent, c'est l'*attente* avant le départ du lot.
- **Un même système contient souvent les deux.** Le scoring de fraude est en streaming ; le réentraînement du modèle qui calcule ce score est en batch, la nuit. Les opposer ne veut pas dire choisir une fois pour toutes.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui du **choix de l'instant de traitement**. Devant une source de données, un concepteur doit décider : j'attends, ou je traite tout de suite ? Sans cette règle, la décision se prend par habitude ou par mode, et l'on obtient soit des rapports qui arrivent trop tard, soit des architectures de streaming inutilement complexes pour produire un bilan mensuel.

La règle donne un vocabulaire pour nommer les deux options et, surtout, pour nommer leur coût respectif : la latence d'un côté, la complexité de l'autre.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous permet de **lire une architecture existante**. Devant n'importe quel schéma, on sait désormais repérer les flèches qui portent un lot (un fichier déposé chaque nuit) et celles qui portent un flux (une file de messages). Cette lecture est la base des chapitres 2 et 3, où nous verrons des architectures qui combinent les deux (Lambda, Kappa).

Elle nous permet aussi de **prévoir la latence sans mesurer**. Dès qu'on voit un traitement par lots de période `P`, on sait que la latence moyenne vaut environ `P/2` et la latence maximale environ `P`. C'est un calcul qu'on peut faire sur un coin de table, avant d'écrire une ligne de code.

Enfin elle nous donne un **levier d'optimisation honnête** : si la latence d'un batch ne convient pas, la seule action utile est de réduire `P` ou de passer en flux ; acheter une machine plus puissante ne changera rien.

▶ 8. Comment on y est arrivé

Origine de la règle

Le batch vient du matériel : quand une machine coûtait le prix d'un immeuble, on la faisait tourner sans interruption sur de gros paquets de travail, et l'attente des utilisateurs était secondaire. Le vocabulaire (*job*, *batch*, *ordonnanceur*) en est resté.

Le streaming est apparu quand deux choses se sont inversées. D'abord, les données sont devenues **continues par nature** : un capteur ne produit pas un fichier, il produit un signal ; découper ce signal en lots est une convention qu'on ajoute, pas une propriété de la source. Ensuite, la mémoire et le réseau sont devenus assez bon marché pour qu'on puisse garder un état en cours de calcul au lieu de tout relire.

On peut dériver la règle en raisonnant sur la latence. Pour un batch de période `P`, un événement arrivant uniformément dans la période attend en moyenne `P/2` avant le départ du lot, puis la durée du calcul `c` : $L_{batch} \approx P/2 + c$. Pour un flux, l'attente de regroupement disparaît : $L_{flux} \approx c'$ où `c'` est le coût de traitement d'un seul événement. Comme `P` est généralement très grand devant `c` et `c'`, l'écart vient presque entièrement du terme `P/2`. C'est cette inégalité qui justifie la distinction.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Reporting financier et comptable.** Bilans, relevés de compte, déclarations fiscales : le lot est imposé par le calendrier légal, et la notion de « période close » a une valeur juridique. Le batch n'est pas un choix technique ici, c'est une obligation.
- **Entraînement de modèles d'apprentissage.** On rassemble un historique complet, on entraîne, on évalue, on déploie. Le batch permet de recommencer à l'identique, ce qui est indispensable pour comparer deux modèles.
- **Détection d'anomalie et alerte.** Fraude, intrusion, panne : la décision porte sur un événement isolé et doit être immédiate. C'est le domaine naturel du streaming.
- **Tableaux de bord opérationnels.** Nombre de commandes en cours, température d'un four, trafic sur un site : l'utilisateur regarde l'écran *maintenant*, donc la valeur affichée doit être fraîche. Streaming, ou micro-batch de quelques secondes.
- **Migration et reprise d'historique.** Cas mixte fréquent : on rejoue en batch trois ans d'archives pour initialiser un état, puis on bascule en streaming pour la suite. C'est exactement l'architecture Kappa du chapitre 2.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Choisissez le batch quand l'énoncé nomme une période.** « Chaque nuit », « en fin de mois », « par semestre » sont des signaux directs : la période existe déjà dans le métier, inutile de la combattre.
- **Choisissez le flux quand l'énoncé dit « dès que ».** « Dès qu'une transaction dépasse un seuil », « dès qu'un capteur sort de sa plage » : la condition porte sur un événement unique, pas sur un ensemble.
- **Méfiez-vous quand le calcul a besoin de tout l'ensemble.** Une médiane exacte, un classement global, une déduplication sur l'historique complet : ces calculs sont naturellement batch. En flux, il faudra les approcher (sketches, chapitre 4) ou accepter un résultat révisable.
- **Quand les deux sont demandés.** Si l'énoncé réclame à la fois une alerte immédiate et un rapport exact en fin de journée, ne choisissez pas : c'est une architecture mixte, et c'est un cas très fréquent en entreprise.

▶ 11. Comment on va l'utiliser

Méthode · trancher entre lot et flux

1. **Demander s'il existe un instant de complétude.** « À quel moment peut-on dire que les données sont toutes là ? » S'il n'y en a aucun, le batch sera artificiel.
2. **Reprendre `D` de la notion 1.** Le délai utile a déjà été écrit ; on le réutilise tel quel, on ne le redécouvre pas.
3. **Estimer la latence du batch : `P/2 + c`.** On choisit la période `P` la plus naturelle pour le métier et on calcule.
4. **Comparer à `D`.** Si `P/2 + c ≤ D`, on garde le batch : c'est plus simple, plus facile à rejouer, moins cher. Sinon, on essaie de diminuer `P`.
5. **Basculer en flux si `P` doit descendre sous quelques secondes.** À ce stade, le batch coûte plus cher à opérer qu'un flux, et on passe en streaming (ou en micro-batch).

À retenir

La latence d'un traitement par lots vaut environ la moitié de sa période : elle ne dépend pas de la puissance de la machine, mais du rythme auquel on a décidé de regrouper.

▶ 12. Exercices d'application corrigés

Application 2.1

Un opérateur téléphonique envoie une alerte « quota de données dépassé ». Le traitement actuel est un lot toutes les 6 heures, dont le calcul dure 4 minutes. Le service client exige que l'abonné soit prévenu **dans les 15 minutes** suivant le dépassement. Le système actuel convient-il ? Que proposez-vous ?

Corrigé 2.1

Étape 1 — instant de complétude. La consommation de données est continue : il n'existe aucun instant naturel où l'on pourrait dire « c'est complet ». La période de 6 heures est une convention ajoutée par l'exploitant.

Étape 2 — reprendre `D`. Le métier impose `D = 15 min = 900 s`.

Étape 3 — latence du batch. `P = 6 h = 21 600 s` et `c = 4 min = 240 s`. Donc `L ≈ P/2 + c = 10 800 + 240 = 11 040 s`, soit environ 3 h 04. Dans le pire cas, `L ≈ P + c = 21 840 s`, soit plus de 6 heures.

Étape 4 — comparer. `11 040 s > 900 s` : le système actuel est hors budget d'un facteur 12. Il ne convient pas.

Étape 5 — réduire `P` ou basculer. Pour respecter `P/2 + 240 ≤ 900`, il faudrait `P ≤ 1320 s`, soit 22 minutes. Un lot toutes les 20 minutes satisferait donc le besoin : c'est la solution la moins intrusive, à privilégier si l'infrastructure existe déjà. Si l'opérateur souhaite ensuite descendre à la minute, la règle indique qu'il faudra passer en flux, car `c = 240 s` deviendrait lui-même supérieur à la période : le lot ne finirait pas avant le départ du suivant.

Application 2.2

Un site de vente veut afficher sur son tableau de bord : (a) le chiffre d'affaires du mois précédent, (b) le nombre de visiteurs connectés en ce moment. Pour chaque indicateur, dites si c'est du batch ou du flux, et justifiez avec la méthode.

Corrigé 2.2

(a) Chiffre d'affaires du mois précédent. Étape 1 : il existe un instant de complétude évident, la clôture du mois, qui a de plus une valeur comptable. Étape 2 : `D` vaut plusieurs jours, personne ne prend de décision à la seconde sur ce chiffre. Étape 3 : avec `P = 1 mois`, la latence est énorme mais sans importance puisqu'on parle du mois *précédent*, déjà clos. Étape 4 : `L ≤ D` largement. **Conclusion : batch**, et même un batch mensuel.

(b) Visiteurs connectés en ce moment. Étape 1 : aucun instant de complétude ; les connexions et déconnexions arrivent sans cesse. Étape 2 : l'expression « en ce moment » *est* le délai utile ; on peut l'écrire `D ≈ 5 s`, au-delà le chiffre affiché ne décrit plus le présent. Étape 3 : un batch respectant `P/2 ≤ 5 s` exigerait `P ≤ 10 s`. Étape 5 : une période de 10 secondes place le système dans la zone où le batch coûte plus cher qu'un flux. **Conclusion : flux** (ou micro-batch de quelques secondes).

Remarque de conception. Les deux indicateurs vivent sur le même tableau de bord mais pas dans la même chaîne ; c'est normal, et c'est l'exemple le plus simple d'architecture mixte.

▶ 13. Exercices d'entraînement

Entraînement

1. Un lot tourne toutes les 30 minutes et son calcul dure 2 minutes. Donnez la latence moyenne et la latence maximale d'un événement.
2. À partir de quelle valeur de `P` un traitement par lots dont le calcul dure 90 secondes devient-il impossible à tenir ? Expliquez ce qui se passe physiquement en dessous de cette valeur.
3. Classez en batch ou en flux, avec une phrase de justification chacun : (a) bulletin de notes du semestre ; (b) alerte d'un détecteur de fumée ; (c) calcul du salaire mensuel ; (d) GPS qui recalcule l'itinéraire après une erreur de route.

▶ 14. Exercices de difficulté croissante

Progression

1. facile Un lot quotidien part à 2 h du matin. Un événement se produit à 2 h 05. Quand sera-t-il traité ? Quelle est sa latence ?
2. moyen On hésite entre un micro-batch de 2 s et un flux pur de latence 50 ms, pour `D = 1 s`. Lequel respecte la règle ? Le micro-batch peut-il être sauvé en changeant un seul paramètre ?
3. difficile Un métier demande le nombre exact de visiteurs *uniques* du jour, rafraîchi toutes les 10 secondes. Montrez que ce besoin mélange une contrainte batch et une contrainte flux, et proposez deux conceptions différentes en indiquant ce que chacune sacrifie.

N3 — La donnée

▶ 4. La règle

Règle · Donnée

Une donnée est une information enregistrée : un nombre, un texte, une date, une position. Seule, elle dit peu de chose ; avec son contexte, elle devient utile.

En informatique, on regroupe les données en **enregistrements** (une ligne d'un tableau), eux-mêmes composés de **champs** (les colonnes). Un enregistrement est donc un ensemble de couples `(nom de champ, valeur)` décrivant un même objet.

Conditions d'emploi. Pour qu'une donnée soit exploitable, il faut connaître trois choses : *de quoi* elle parle (le sujet), *quelle grandeur* elle mesure avec son unité, et *à quel moment* elle se rapporte.

▶ 5. Explication de la règle

La règle dit qu'une donnée n'est pas un nombre : c'est un nombre *situé*. L'exemple du cours est parfait : « 38,5 » ne veut rien dire. Ce peut être une température, un prix, un angle, un pourcentage. Écrivez « température de Youssef = 38,5 °C, le 15 octobre à 8 h » et tout change : on comprend que Youssef a de la fièvre, et un médecin peut agir.

Ce qui a été ajouté entre les deux écritures, ce n'est pas de la décoration : c'est le **sujet** (Youssef), la **grandeur et son unité** (température en degrés Celsius) et l'**instant** (15 octobre, 8 h). Ces trois

éléments forment le contexte, et ils reviendront exactement sous cette forme dans le message JSON de la notion 8 : une clé qui identifie le sujet, des champs qui portent les grandeurs, un horodatage qui porte l'instant.

La seconde idée de la règle est celle de la **structure en lignes et colonnes**. Une donnée isolée est rare ; en pratique on manipule des enregistrements qui portent plusieurs champs d'un coup. C'est ce qui permet de comparer, de regrouper, de filtrer : on ne peut dire « les ventes de Casablanca » que si chaque enregistrement de vente porte un champ *ville*.

Cas particuliers et pièges

- **Donnée, information, connaissance.** On réserve souvent *donnée* au fait brut enregistré, *information* à la donnée interprétée dans son contexte, et *connaissance* à ce qu'on en déduit. Dans ce module, « donnée » désigne l'enregistrement tel qu'il circule.
- **L'unité manquante est une faute classique.** Un champ `duree: 500` sans unité a provoqué des pannes réelles : 500 ms ou 500 s ? Nommez le champ `duree_ms`.
- **Une valeur absente n'est pas zéro.** Un capteur qui n'a pas répondu produit une absence ; écrire 0 à la place transforme une panne en mesure, et fausse toutes les moyennes.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui de l'**ambiguïté des valeurs brutes**. Une chaîne de traitement reçoit des millions de valeurs venues de sources différentes ; si chacune arrive sans son contexte, aucun rapprochement n'est possible et aucune vérification non plus.

La règle résout ce problème en imposant que l'unité de circulation ne soit pas la valeur mais l'*enregistrement contextualisé*. C'est ce qui permettra, plus tard, de joindre deux flux, de regrouper par clé ou d'ordonner par temps.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous donne la **grille de lecture d'un message**. Quand nous ouvrirons un message JSON à la notion 8, nous saurons quoi y chercher : le sujet, les grandeurs, l'instant. Un message où l'un des trois manque sera immédiatement signalé comme incomplet.

Elle nous permet aussi de **concevoir un schéma**. Choisir les champs d'un événement, c'est décider quel contexte voyage avec la valeur. Un champ oublié à la conception coûte très cher : il faudra le rajouter plus tard sans casser les consommateurs existants.

Enfin elle fonde la **validation en entrée** : un pipeline sérieux rejette ou met de côté les enregistrements dont un champ obligatoire manque, plutôt que de les laisser polluer les calculs.

▶ 8. Comment on y est arrivé

Origine de la règle

La forme « lignes et colonnes » vient directement des registres comptables sur papier, puis des cartes perforées où chaque carte était un enregistrement et chaque zone de colonnes un champ. Le modèle relationnel, formalisé dans les années 1970, en a fait une théorie : une table est une relation, une ligne un n-uplet.

La justification du besoin de contexte est plus simple et purement logique. Une valeur seule, disons `v`, ne permet aucune opération utile : on ne peut ni la comparer à une autre valeur du même sujet (on ignore le sujet), ni vérifier sa cohérence (on ignore l'unité), ni la placer dans une suite (on ignore l'instant). Dès qu'on adjoint les trois éléments `(sujet, grandeur+unité, instant)`, les trois opérations redeviennent possibles. La règle n'est donc pas une convention arbitraire : c'est le minimum d'information qui rend la donnée exploitable.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Conception des schémas d'événements.** Avant d'écrire un producteur Kafka, on liste les champs de l'événement. La règle fournit la liste minimale : identifiant du sujet, grandeurs avec unités, horodatage.
- **Qualité des données.** Les contrôles d'entrée d'un pipeline (champ obligatoire, type attendu, plage de valeurs) découlent tous de cette règle ; on ne peut contrôler que ce qui a été nommé.
- **Capteurs et IoT.** Un capteur industriel envoie une valeur toutes les secondes ; sans identifiant de capteur ni unité, un parc de mille capteurs devient ininterprétable.
- **Journaux applicatifs.** Une ligne de log est un enregistrement : niveau, service, message, horodatage. Les outils d'analyse de logs ne fonctionnent que parce que cette structure existe.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Quand un énoncé vous donne une valeur nue.** « Le capteur indique 72 » : le réflexe est de demander aussitôt quel capteur, quelle grandeur, quelle unité, à quelle heure. C'est le signal qu'il manque du contexte.
- **Quand on doit rapprocher deux sources.** Dès qu'un exercice demande de joindre, comparer ou agréger, il faut vérifier que les deux côtés portent bien un sujet commun : sans cela la jointure est impossible.
- **Quand un résultat paraît absurde.** Une moyenne de température à 3 °C dans un four signale presque toujours des absences comptées comme zéros, ou deux unités mélangées. La règle indique où regarder.

▶ 11. Comment on va l'utiliser

Méthode · qualifier une donnée

1. **Nommer le sujet.** De qui ou de quoi parle-t-on ? On lui donne un identifiant stable : numéro de carte, identifiant de capteur, identifiant de session.
2. **Nommer la grandeur et son unité.** On écrit l'unité dans le nom du champ chaque fois que c'est possible : `temperature_c`, `montant_mad`, `duree_ms`.
3. **Dater.** On attache l'instant auquel la valeur se rapporte, pas celui où on l'a reçue ; la distinction fera l'objet de la notion 11.
4. **Regrouper en enregistrement.** On rassemble ces éléments en une ligne, en vérifiant que chaque champ est atomique : une seule valeur par champ, pas de liste cachée dans un texte.
5. **Vérifier la complétude.** On relit l'enregistrement en se demandant : un collègue qui ne connaît pas ce système pourrait-il l'interpréter seul ? Si non, il manque du contexte.

À retenir

Une donnée utile porte toujours trois choses avec elle : de qui elle parle, quelle grandeur elle mesure et dans quelle unité, et à quel instant elle se rapporte.

▶ 12. Exercices d'application corrigés

Application 3.1

On reçoit la valeur brute : `1013`. Rendez-la exploitable sous forme d'un enregistrement, sachant qu'elle provient du capteur `BARO-07` installé dans l'atelier B, qu'elle mesure une pression atmosphérique en hectopascals, et qu'elle a été relevée le 15 octobre 2026 à 8 h 30 min 12 s UTC.

Corrigé 3.1

Étape 1 — nommer le sujet. Le sujet est le capteur : identifiant stable `BARO-07`. L'atelier est une information utile mais secondaire ; on la garde comme champ descriptif.

Étape 2 — nommer la grandeur et l'unité. La grandeur est une pression, l'unité l'hectopascal. On nomme donc le champ `pression_hpa` et non `valeur`.

Étape 3 — dater. L'instant de la mesure est `2026-10-15T08:30:12Z` ; le `Z` indique l'heure universelle, ce qui évite toute ambiguïté de fuseau.

Étapes 4 et 5 — regrouper et vérifier. On obtient un enregistrement à quatre champs atomiques :

```
{ "capteur": "BARO-07", "atelier": "B",
  "pression_hpa": 1013,
  "mesure_le": "2026-10-15T08:30:12Z" }
```

Relu par un collègue extérieur, cet enregistrement s'interprète sans explication : la condition de complétude est satisfaite.

Application 3.2

Un collègue vous propose ce schéma d'événement pour un flux de livraisons : `{ "info": "Rabat;Casa;42;ok", "t": 1792182612 }`. Relevez tous les défauts au regard de la règle et proposez une correction.

Corrigé 3.2

Défaut 1 — le sujet n'est pas identifié. Aucun champ ne dit de quelle livraison on parle. Impossible de suivre une livraison dans le temps ni de regrouper par clé : toutes les opérations de la notion 9 deviendraient impossibles.

Défaut 2 — les champs ne sont pas atomiques. `"Rabat;Casa;42;ok"` contient quatre informations dans un seul texte. Chaque consommateur devra découper lui-même, avec un risque d'erreur dès qu'une ville contiendra un point-virgule.

Défaut 3 — les unités manquent. Que vaut `42` ? Des kilogrammes, des kilomètres, des minutes ? Le nom du champ doit le dire.

Défaut 4 — l'horodatage est ambigu. `t` est un nombre : secondes ou millisecondes depuis 1970 ? Et s'agit-il de l'instant de l'événement ou de l'envoi ? La notion 11 montrera que confondre les deux coûte cher.

Correction proposée :

```
{ "livraison_id": "LIV-2026-00842",
  "origine": "Rabat", "destination": "Casablanca",
  "poids_kg": 42, "statut": "livree",
  "evenement_le": "2026-10-15T08:30:12Z" }
```

Chaque champ est atomique, porté par un nom explicite, et le sujet comme l'instant sont présents.

▶ 13. Exercices d'entraînement

Entraînement

1. Transformez en enregistrement complet : « le compteur électrique du logement A12 a relevé 3,4 ce matin à 7 h ». Précisez l'unité que vous choisissez et justifiez.
2. Un fichier de ventes a les colonnes `ville`, `montant`, `date`. Quel élément du contexte manque-t-il pour pouvoir suivre un client dans le temps ?
3. Donnez trois exemples de noms de champs mal choisis que vous avez déjà rencontrés, et réécrivez-les correctement.

▶ 14. Exercices de difficulté croissante

Progression

1. facile Dans l'enregistrement `{"carte":"4970-XXXX-1234","montant":18.0,"devise":"MAD"}`, nommez les champs et dites lequel porte le sujet.
2. moyen Un capteur tombe en panne et son pilote écrit `0` au lieu de ne rien écrire. Sur 1000 mesures réelles de moyenne 25 °C, 200 sont remplacées par des zéros. Calculez la moyenne observée et commentez.
3. difficile Vous devez concevoir le schéma d'événement d'un service de trottinettes en libre-service. Listez les champs, justifiez chacun par la règle, et expliquez comment vous gérerez l'ajout futur d'un champ « niveau de batterie » sans casser les consommateurs déjà en place.

N4 — Où ranger les données

▶ 4. La règle

Règle · Les quatre rangements

Les données au repos se rangent dans quatre types de contenants : le fichier, la base de données, l'entrepôt de données et le lac de données. Leur point commun est que les données y attendent qu'on vienne les interroger.

Rangement	Image simple	Exemple	Usage
Fichier	Une feuille dans un classeur	`ventes_octobre.csv`	Échanger, archiver
Base de données	Une bibliothèque rangée, avec un bibliothécaire	Table `clients` dans PostgreSQL	Chercher et modifier vite
Entrepôt de données	Un grand entrepôt pour les analyses	Toutes les ventes depuis 10 ans	Tableaux de bord, rapports
Lac de données	Un grand lac où l'on dépose tout, brut	Fichiers Parquet sur un stockage cloud	Big Data, apprentissage automatique

Conditions d'emploi. La règle décrit des données *au repos* : bornées, déjà écrites, interrogeables à la demande. Elle ne s'applique pas à un flux, qui est une donnée *en mouvement*.

▶ 5. Explication de la règle

Ces quatre contenants ne diffèrent pas par ce qu'ils contiennent — ce sont toujours des enregistrements — mais par **ce qu'ils promettent**. Un fichier ne promet rien d'autre que de garder des octets dans l'ordre : pour trouver une ligne, il faut tout relire. Une base promet de retrouver vite une ligne précise et de la modifier sans casser les autres ; c'est le rôle du « bibliothécaire », qui tient les index et arbitre les accès simultanés.

L'entrepôt et le lac se distinguent par le moment où l'on impose une structure. L'**entrepôt** range des données déjà nettoyées, modélisées pour l'analyse : la structure est décidée avant l'écriture, ce qui rend les requêtes rapides mais les changements lourds. Le **lac** accepte tout, brut, sans modèle imposé :

la structure est décidée au moment de la lecture, ce qui est souple mais reporte tout le travail de nettoyage sur l'analyste.

Le point le plus important de la règle est la phrase de conclusion : **dans les quatre cas, les données sont au repos et attendent**. C'est cette passivité qui va devenir impossible en temps réel. Toute la suite du module consiste à remplacer « les données attendent qu'on les interroge » par « les données arrivent et déclenchent le calcul ».

Cas particuliers et pièges

- **Le lakehouse.** Une catégorie récente qui pose une couche transactionnelle sur un lac, pour obtenir la souplesse du lac avec les garanties de l'entrepôt. Nous la retrouverons au chapitre 2.
- **Le « marécage » de données.** Un lac sans catalogue ni documentation devient inexploitable : on y a tout déposé mais plus personne ne sait ce qu'il contient. La souplesse a un coût de gouvernance.
- **Une file de messages n'est pas un rangement.** Kafka conserve pourtant les messages sur disque. La différence est l'intention : on ne l'interroge pas à la demande, on s'y abonne pour être poussé les nouveautés.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui du **choix du lieu de stockage selon l'usage**. Une même donnée de vente peut servir à émettre une facture (accès unitaire, immédiat), à produire un rapport annuel (lecture massive, agrégée) ou à entraîner un modèle (lecture brute, volumineuse). Stocker les trois usages au même endroit conduit soit à des requêtes lentes, soit à un coût de stockage déraisonnable.

La règle nomme les quatre options et les promesses associées, ce qui permet d'affecter chaque usage au bon contenant au lieu de tout empiler dans une seule base.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous sert d'abord à **situer le streaming par contraste**. En comprenant que les quatre rangements partagent la même passivité, on comprend ce que le flux apporte de nouveau : l'initiative passe de celui qui lit à celui qui écrit.

Elle nous sert ensuite à **lire les architectures des chapitres suivants**. Lambda, Kappa et Lakehouse sont des façons d'articuler un flux avec un ou plusieurs de ces rangements ; sans le vocabulaire, les schémas resteraient opaques.

Enfin elle nous sert dans les TP : chaque fois qu'un résultat de traitement doit être conservé, il faudra choisir où l'écrire, et la table ci-dessus donne le critère.

▶ 8. Comment on y est arrivé

Origine de la règle

Chacun de ces contenants est né d'une limite du précédent. Le fichier existe depuis les débuts de l'informatique ; sa limite est qu'une recherche coûte une lecture complète et que deux programmes qui écrivent en même temps se corrompent. La base de données est apparue pour lever ces deux limites, avec les index et les transactions.

La base, à son tour, s'est révélée inadaptée aux analyses massives : optimisée pour des accès unitaires rapides, elle s'effondre quand on lit dix ans d'historique. D'où l'entrepôt, organisé en colonnes et pensé pour l'agrégation. Puis le volume et la variété du Big Data ont rendu coûteux le nettoyage préalable qu'exige l'entrepôt : le lac a inversé l'ordre en stockant d'abord et en structurant à la lecture.

La règle est donc le résultat d'une **succession de compromis entre coût d'écriture et coût de lecture**. On peut la résumer ainsi : plus on travaille au moment d'écrire (nettoyer, indexer, modéliser), moins on travaille au moment de lire, et inversement. Les quatre contenants sont quatre points sur cet axe.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Échange entre organisations.** Un fichier CSV reste le format d'échange par défaut entre deux entreprises, parce qu'il ne suppose aucun logiciel commun. Son défaut : aucune garantie de cohérence.
- **Applications transactionnelles.** Une base relationnelle porte les données vivantes d'une application : clients, commandes, stocks. C'est là que la notion d'accès concurrent prend tout son sens.
- **Décisionnel et pilotage.** L'entrepôt alimente les tableaux de bord de direction ; ses requêtes lisent des millions de lignes mais n'en modifient aucune.
- **Science des données et apprentissage.** Le lac conserve les données brutes dont les modèles ont besoin, y compris celles dont on ne connaît pas encore l'usage ; c'est son principal argument.
- **Archivage réglementaire.** Fichiers immuables conservés plusieurs années ; ici l'absence de promesse du fichier devient une qualité : rien ne peut le modifier silencieusement.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Quand l'énoncé dit « retrouver un enregistrement précis ».** Recherche par identifiant, mise à jour unitaire, accès simultanés : c'est une base de données, pas un fichier.
- **Quand l'énoncé dit « par année, par région, par catégorie ».** Ces regroupements sur de longues périodes désignent un entrepôt : on lit beaucoup, on écrit peu.
- **Quand on ne sait pas encore ce qu'on cherchera.** « Gardons tout, on verra plus tard » désigne un lac. Le signal est l'absence de question précise au moment du stockage.
- **Quand il faut sortir les données du système.** Transmission à un partenaire, dépôt réglementaire, sauvegarde : le fichier reste le bon choix, pour sa portabilité.

▶ 11. Comment on va l'utiliser

Méthode · choisir un rangement

1. **Décrire l'accès dominant.** Lecture unitaire ou lecture massive ? Écriture fréquente ou écriture unique ? Cette seule question élimine déjà deux options sur quatre.
2. **Décider si la structure est connue d'avance.** Si oui, entrepôt ou base ; si non, lac.
3. **Évaluer le besoin de cohérence.** Plusieurs écrivains simultanés imposent une base ; un seul producteur permet le fichier.
4. **Évaluer le volume et la durée de conservation.** Au-delà de quelques téraoctets conservés des années, le coût pousse vers le lac.
5. **Vérifier qu'on parle bien de données au repos.** Si l'énoncé dit « dès que » ou « en continu », aucun des quatre ne répond seul : il faut un flux, et l'un de ces contenants servira seulement à garder le résultat.

À retenir

Fichier, base, entrepôt et lac partagent une même propriété : les données y sont au repos et attendent qu'on vienne les interroger. Le temps réel consiste à inverser cette initiative.

▶ 12. Exercices d'application corrigés

Application 4.1

Une chaîne de supermarchés veut : (a) connaître le stock exact d'un produit dans un magasin au moment où un client le demande ; (b) produire chaque lundi le chiffre d'affaires par région des cinq dernières années ; (c) conserver les vidéos des caméras pour un futur projet de comptage de clients. Choisissez un rangement pour chaque besoin.

Corrigé 4.1

(a) Stock exact à la demande. Étape 1 : l'accès est unitaire (un produit, un magasin) et l'écriture est fréquente (chaque vente décrémente). Étape 3 : plusieurs caisses écrivent en même temps, donc la cohérence est indispensable. **Base de données.** Un fichier serait corrompu par les écritures concurrentes ; un entrepôt répondrait trop lentement à un accès unitaire.

(b) Chiffre d'affaires par région sur cinq ans. Étape 1 : lecture massive, agrégation, aucune écriture. Étape 2 : la structure est parfaitement connue (date, région, montant). Étape 4 : le volume est important mais modélisable. **Entrepôt de données.** Le rythme hebdomadaire confirme qu'on est en batch, conformément à la notion 2.

(c) Vidéos pour un projet futur. Étape 2 : on ne sait pas encore quelle structure on en tirera ; c'est le signal du lac. Étape 4 : volume très important, conservation longue, coût au gigaoctet déterminant. **Lac de données.** On ajoutera un catalogue dès le début pour éviter le marécage.

Application 4.2

Un étudiant écrit : « Kafka est une base de données, puisqu'il garde les messages sur disque et qu'on peut les relire. » Discutez cette affirmation en vous appuyant sur la règle.

Corrigé 4.2

Ce qui est exact. Kafka écrit bien les messages sur disque et permet de les relire plus tard, y compris depuis le début. Sur ce point précis, l'étudiant a raison : la persistance existe.

Ce qui ne colle pas avec la règle. La règle ne définit pas les contenants par leur support physique mais par leur *promesse d'accès*. Une base promet de retrouver rapidement un enregistrement désigné par son identifiant, grâce à des index, et de le modifier. Kafka ne promet ni l'un ni l'autre : il n'y a pas d'index sur le contenu, et un message écrit n'est pas modifiable.

Le point décisif : l'initiative. Dans les quatre rangements, les données sont au repos et c'est le lecteur qui vient les chercher quand il le décide. Avec Kafka, le consommateur s'abonne et reçoit les nouveaux messages à mesure qu'ils arrivent : l'initiative appartient au producteur. C'est exactement la propriété que la règle identifie comme absente du flux.

Conclusion. Kafka est un *journal de messages distribué*, pas un rangement au sens de la règle. La confusion est fréquente et sans gravité tant qu'on garde en tête le critère de l'initiative.

▶ 13. Exercices d'entraînement

Entraînement

1. Pour chacun, nommez le rangement : (a) un export mensuel envoyé au commissaire aux comptes ; (b) la table des utilisateurs d'une application mobile ; (c) dix ans de relevés météo bruts gardés pour un futur modèle.
2. Expliquez en trois phrases pourquoi un entrepôt répond mal à la question « quel est le solde du compte 4521 maintenant ? ».
3. Qu'est-ce qu'un marécage de données, et quelle pratique permet de l'éviter ?

▶ 14. Exercices de difficulté croissante

Progression

1. facile Citez les quatre rangements et donnez pour chacun l'image simple vue en cours.
2. moyen Une équipe stocke les journaux applicatifs dans la base de production pour pouvoir « tout avoir au même endroit ». Donnez deux conséquences prévisibles et proposez une répartition conforme à la règle.
3. difficile Une banque doit conserver chaque transaction : pour l'application mobile (solde instantané), pour le reporting réglementaire (5 ans, requêtes agrégées), et pour la détection de fraude en temps réel. Décrivez la circulation de la même transaction à travers les différents contenants, et dites lequel reçoit l'écriture en premier.

N5 — La requête bornée et la requête continue

▶ 4. La règle

Règle · Deux sortes de requêtes

Une requête SQL classique est posée une fois : elle lit les données présentes, produit un résultat, puis s'arrête. Dans un flux, on écrit des requêtes qui ne s'arrêtent jamais : elles restent en place et mettent leur résultat à jour à chaque nouvelle donnée.

```sql
-- Requete bornee : « donne-moi le total des ventes par ville »
SELECT ville, SUM(montant) AS total
FROM   ventes
GROUP BY ville;
```

La requête bornée s'applique à un ensemble fini : elle possède un début, une fin, et un résultat définitif. La requête continue s'applique à une suite infinie : elle n'a pas de fin, et son résultat est une suite de résultats successifs.

Conditions d'emploi. La requête bornée suppose que la table ne change pas pendant son exécution. La requête continue suppose qu'on accepte un résultat provisoire, car il n'existe aucun instant où l'on pourrait dire « c'est fini ».

▶ 5. Explication de la règle

Reprenons la requête de l'exemple. `SELECT` dit ce que l'on veut voir, `FROM` désigne la table où se trouvent les données, `GROUP BY` demande de regrouper par ville avant de faire la somme. Rien de surprenant : c'est une question posée à des données au repos, au sens de la notion 4.

L'idée profonde de la règle est le **renversement de ce qui est fixe et de ce qui bouge**. Dans le monde borné, les données sont fixes et la requête passe : on l'exécute, elle traverse les lignes, elle rend un résultat et disparaît. Dans le monde du flux, c'est l'inverse : la requête est fixe, installée une fois pour toutes, et ce sont les données qui passent à travers elle. On dit parfois que le flux « traverse » la requête.

Cette inversion a une conséquence qu'il faut accepter tout de suite : **un résultat continu est toujours provisoire**. Quand une requête continue annonce « total de Casablanca : 48 320 MAD », cela signifie « compte tenu de ce que j'ai vu jusqu'à maintenant ». Une vente supplémentaire modifiera ce total une seconde plus tard. Ce n'est pas un défaut : c'est la seule réponse possible à une question posée sur un ensemble infini.

Cas particuliers et pièges

- **La fenêtre réintroduit une fin artificielle.** Pour obtenir un résultat définitif à partir d'un flux, on découpe le temps en fenêtres (« total des 5 dernières minutes »). C'est tout l'objet du chapitre 4.
- **Certaines opérations sont impossibles telles quelles.** `ORDER BY` global, médiane exacte, `COUNT(DISTINCT)` exact sur l'historique complet : toutes exigent de connaître l'ensemble entier. En flux, on les approche ou on les borne par fenêtre.
- **Le SQL continu ressemble au SQL borné, mais ne se lit pas pareil.** Flink SQL et Spark SQL utilisent la même syntaxe ; ne vous laissez pas tromper par la ressemblance, la sémantique du résultat est différente.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui de l'**interrogation d'un ensemble qui n'est jamais complet**. Tout le langage SQL a été conçu pour des tables finies ; ses opérateurs supposent qu'on peut parcourir toutes les lignes avant de répondre. Appliqué à un flux, le même `SELECT` ne se terminerait jamais.

La règle traite ce problème en changeant la définition du résultat : au lieu d'une valeur unique produite à la fin, le résultat devient une *suite de valeurs* produite au fil de l'eau. C'est ce changement de définition qui rend le SQL utilisable sur un flux.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous permet de **réutiliser tout ce que nous savons déjà en SQL**. Filtrer, regrouper, joindre, agréger : les opérations restent les mêmes, seule la lecture du résultat change. C'est un gain énorme, car cela évite d'apprendre un langage entièrement nouveau.

Elle nous prépare aussi à **comprendre pourquoi les fenêtres existent**. Un étudiant qui découvre les fenêtres au chapitre 4 sans avoir compris cette règle les prend pour une complication arbitraire ; en réalité elles répondent exactement au caractère provisoire du résultat.

Enfin elle nous donne un **test de faisabilité** : devant une requête à porter en flux, on se demande si elle a besoin de l'ensemble complet. Si oui, il faudra la réécrire.

▶ 8. Comment on y est arrivé

Origine de la règle

SQL est né avec le modèle relationnel, dans un monde où une table était un ensemble fini de n-uplets. Toute la théorie repose sur cette finitude : l'algèbre relationnelle définit ses opérateurs comme des fonctions d'ensembles finis vers des ensembles finis.

Quand les flux sont apparus, deux voies étaient possibles : inventer un langage spécifique, ou étendre SQL. La seconde l'a emporté pour une raison pratique : des millions de personnes savent déjà écrire du SQL. Les travaux de recherche des années 2000 sur les *Data Stream Management Systems* ont montré comment donner un sens aux opérateurs relationnels sur une suite infinie.

Le raisonnement qui fonde la règle est le suivant. Soit un flux `S` et une requête `Q`. On ne peut pas calculer `Q(S)`, car `S` est infini. Mais pour tout instant `t`, l'ensemble $S_{\leq t}$ des événements déjà arrivés est fini, donc $Q(S_{\leq t})$ est calculable par le SQL ordinaire. On définit alors le résultat continu comme la *famille* $(Q(S_{\leq t}))_t$. La requête continue n'est donc rien d'autre que la requête bornée, réévaluée à chaque instant — et toute la technique consiste ensuite à obtenir cette réévaluation sans tout recalculer.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Tableaux de bord temps réel.** Les compteurs affichés (commandes du jour, erreurs par minute) sont des requêtes continues ; ils se mettent à jour seuls parce que la requête reste installée.
- **Détection par seuil.** « Alerter si plus de 5 échecs d'authentification en 1 minute pour un même compte » s'écrit en SQL continu avec un `GROUP BY` et une fenêtre.
- **Enrichissement de flux.** Joindre un flux de transactions à une table de référence des commerces : c'est une jointure, mais entre un objet infini et un objet fini. Nous y reviendrons au chapitre 4.
- **Vues matérialisées.** Une vue matérialisée rafraîchie en continu est conceptuellement une requête continue ; la frontière entre base et flux s'estompe ici.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Quand l'énoncé contient « en continu » ou « à tout moment ».** Ces formules disent que la question doit rester posée ; c'est la signature d'une requête continue.
- **Quand le résultat est affiché plutôt qu'archivé.** Un chiffre qu'on regarde sur un écran doit être frais ; un chiffre qu'on dépose dans un rapport peut être borné.
- **Quand la requête demande un ordre global ou une valeur exacte sur tout l'historique.** C'est le signal inverse : il faudra soit rester borné, soit accepter une approximation.
- **Quand on vous demande de porter une requête existante en flux.** Reprenez-la opérateur par opérateur : `WHERE` et `SELECT` passent sans difficulté, `GROUP BY` demande une fenêtre, `ORDER BY` global est à proscrire.

▶ 11. Comment on va l'utiliser

Méthode · porter une requête en flux

1. **Écrire d'abord la version bornée.** On formule la question comme si les données étaient dans une table. C'est toujours plus facile, et cela sert de référence pour vérifier le résultat.
2. **Repérer les opérateurs qui exigent l'ensemble complet.** On souligne `ORDER BY`, `DISTINCT`, les agrégats sans regroupement temporel, les jointures entre deux flux.
3. **Introduire une fenêtre pour chacun.** « Total par ville » devient « total par ville sur les 5 dernières minutes » : la question redevient finie à chaque instant.
4. **Décider du mode de sortie.** Veut-on un résultat mis à jour en permanence, ou une valeur émise une fois la fenêtre fermée ? Les deux sont légitimes mais ne se lisent pas de la même façon.
5. **Vérifier sur un cas rejoué.** On fait passer un petit jeu d'événements connus et on compare la suite des résultats continus avec le résultat borné de l'étape 1.

À retenir

En SQL borné, les données sont fixes et la requête passe ; en SQL continu, la requête est fixe et ce sont les données qui passent. Le résultat cesse alors d'être une valeur pour devenir une suite de valeurs.

▶ 12. Exercices d'application corrigés

Application 5.1

On dispose d'un flux `paiements(carte, montant, ville, heure)`. Écrivez la requête bornée qui donne le total par ville, puis transformez-la en requête continue utilisable sur le flux. Expliquez chaque modification.

Corrigé 5.1

Étape 1 — version bornée.

```
SELECT ville, SUM(montant) AS total
FROM   paiements
GROUP BY ville;
```

Étape 2 — repérer l'opérateur problématique. C'est `SUM` combiné à `GROUP BY ville`. Sur un flux infini, ce total grandirait indéfiniment et ne serait jamais définitif : la somme de toutes les ventes depuis le début des temps n'a pas d'usage opérationnel.

Étape 3 — introduire une fenêtre. On borne la question dans le temps :

```
SELECT ville, SUM(montant) AS total
FROM   paiements
GROUP BY ville, TUMBLE(heure, INTERVAL '5' MINUTE);
```

Le regroupement se fait désormais par ville *et* par tranche de 5 minutes. À chaque instant, l'ensemble concerné est fini.

Étape 4 — mode de sortie. Avec une fenêtre `TUMBLE`, on émet une ligne définitive à la fermeture de chaque tranche : c'est le mode le plus simple à lire pour un tableau de bord qui affiche « ventes des 5 dernières minutes ».

Étape 5 — vérification. Sur un jeu de dix paiements dont on connaît les heures, la somme des totaux de toutes les fenêtres doit égaler le total borné de l'étape 1. Si ce n'est pas le cas, un événement est tombé hors fenêtre : c'est le problème du retard, traité à la notion 11.

Application 5.2

Un responsable demande : « Donnez-moi en continu le classement des 10 villes par chiffre d'affaires, depuis le début de l'année. » Analysez la faisabilité de cette demande.

Corrigé 5.2

Étape 1 — version bornée. Elle s'écrit sans difficulté : `SELECT ville, SUM(montant) FROM paiements WHERE annee = 2026 GROUP BY ville ORDER BY 2 DESC LIMIT 10;`

Étape 2 — opérateurs problématiques. Deux difficultés. D'abord `ORDER BY` global : classer exige de connaître toutes les villes, donc tout l'ensemble. Ensuite « depuis le début de l'année » : la fenêtre est très longue, l'état à conserver grandit pendant douze mois.

Étape 3 — ce qui sauve la demande. Le nombre de villes est *borné* et petit : quelques dizaines. L'état à maintenir est donc un simple tableau « ville → total courant », qui tient en mémoire. Le classement se recalcule sur ce petit tableau à chaque mise à jour : c'est un `ORDER BY` sur un ensemble de 50 lignes, pas sur le flux.

Étape 4 — mode de sortie. Résultat mis à jour en permanence, et non émis à la fermeture d'une fenêtre : le responsable veut voir le classement évoluer.

Conclusion. La demande est réalisable, mais pas pour la raison qu'on croit : elle l'est parce que la *cardinalité du regroupement* est faible. La même demande sur un classement de clients (plusieurs millions) exigerait un résultat approché ou un traitement par lots nocturne. On le signalera au demandeur : la faisabilité dépend du nombre de groupes, pas du volume du flux.

▶ 13. Exercices d'entraînement

Entraînement

1. Écrivez la requête bornée qui compte les paiements refusés par commerce, puis indiquez ce qu'il faut ajouter pour la rendre continue.
2. Parmi ces requêtes, lesquelles passent telles quelles sur un flux ? (a) `WHERE montant > 1000` ; (b) `ORDER BY heure` ; (c) `SELECT carte, montant * 10` ; (d) `COUNT(DISTINCT carte)`.
3. Expliquez en trois phrases pourquoi un résultat de requête continue est toujours provisoire.

▶ 14. Exercices de difficulté croissante

Progression

1. facile Dans la requête du cours, dites ce que font `SELECT`, `FROM` et `GROUP BY`.
2. moyen Transformez « nombre de connexions par utilisateur » en requête continue avec une fenêtre d'une minute, et dites ce que devient le résultat pour un utilisateur qui ne se connecte pas pendant dix minutes.
3. difficile On veut « la médiane des montants de la dernière heure, mise à jour chaque seconde ». Montrez pourquoi le calcul exact est coûteux, et décrivez deux stratégies possibles en indiquant ce que chacune coûte en précision ou en mémoire.

N6 — Le client et le serveur

▶ 4. La règle

Règle · Client et serveur

Un serveur est un programme, souvent hébergé sur une machine dédiée, qui rend un service et attend les demandes. Un client est un programme qui envoie une demande au serveur et attend la réponse.

La relation est asymétrique et définie par l'initiative : c'est toujours le client qui parle le premier. Un même programme peut être client dans une relation et serveur dans une autre ; les mots désignent des *rôles*, pas des machines.

Conditions d'emploi. Il faut que le serveur soit joignable (une adresse et un port, notion 7), qu'il soit démarré avant la demande, et que les deux programmes se soient mis d'accord sur le format des messages (notion 8).

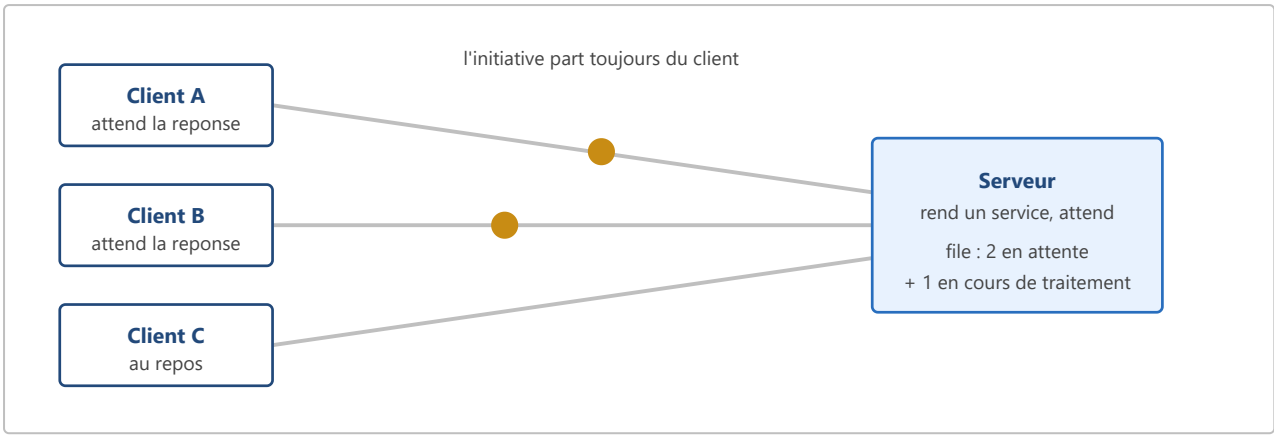


FIGURE 3 · L'INITIATIVE APPARTIENT AU CLIENT
Le serveur ne fait rien tant qu'on ne lui demande rien. Quand plusieurs demandes arrivent ensemble, elles s'empilent dans sa file et sont traitées l'une après l'autre : c'est là que naît le maillon « file d'attente » du budget de latence de la notion 1.

▶ 5. Explication de la règle

L'image du restaurant donnée en cours dit l'essentiel : le client s'assoit et passe commande, le serveur prend la commande, la transmet en cuisine, puis rapporte le plat ; et plusieurs clients peuvent être servis en même temps par le même restaurant. Ce qui compte dans cette image, ce n'est pas la politesse de l'échange : c'est que **le restaurant ne vient jamais vous apporter un plat que vous n'avez pas commandé**.

L'idée profonde est donc celle de l'**initiative**. Le serveur est passif : il ouvre un port, il attend, il répond. Cette passivité a une conséquence importante pour le temps réel : si personne ne demande rien, rien ne se passe, même si le serveur possède une information urgente. C'est précisément la limite que les files de messages lèveront en permettant au producteur de pousser l'information vers des consommateurs abonnés.

La seconde idée est celle de la **mutualisation**. Un serveur sert plusieurs clients simultanés ; il possède donc une file d'attente interne, et cette file est une source de latence que l'on sous-estime souvent. Dans le découpage de L vu à la notion 1, le maillon « file d'attente » vient exactement de là : ce n'est pas le calcul qui est lent, c'est l'attente de son tour.

Cas particuliers et pièges

- **Un programme peut être les deux à la fois.** Un serveur web est serveur pour le navigateur et client pour la base de données. Ne cherchez pas à classer les machines : classez les relations.
- **Serveur ne veut pas dire « grosse machine ».** Un serveur est d'abord un programme. Votre ordinateur portable exécute un serveur Kafka pendant les TP.
- **Pair à pair.** Dans certaines architectures, chaque nœud est tour à tour client et serveur, sans hérarchie. Le modèle client-serveur reste néanmoins la base de tout ce que nous verrons.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui du **partage d'une ressource ou d'un service entre plusieurs programmes**. Si chaque application devait embarquer sa propre copie de la base de données, de la file de messages ou du moteur de calcul, les copies divergeraient immédiatement et aucune cohérence ne serait possible.

Le modèle client-serveur résout ce problème en centralisant le service en un point unique, joignable par tous, et en définissant clairement qui prend l'initiative de l'échange. C'est ce qui permet à dix programmes de lire et d'écrire les mêmes données sans se marcher dessus.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous donne le **vocabulaire des TP**. Dans tout ce module, Kafka sera un serveur ; les programmes Python qui lui envoient des messages (producteurs) ou qui lui en demandent (consommateurs) seront des clients. Savoir qui est client et qui est serveur permet de comprendre immédiatement quel processus doit démarrer en premier.

Elle nous permet aussi de **localiser une panne**. Quand un TP ne fonctionne pas, la première question est : le serveur est-il démarré et joignable ? La seconde : le client vise-t-il la bonne adresse et le bon port ? Ces deux questions résolvent la majorité des blocages.

Enfin elle nous prépare à comprendre **pourquoi le modèle de publication-abonnement existe** : il est né de l'insuffisance du client-serveur pur pour les notifications.

▶ 8. Comment on y est arrivé

Origine de la règle

Le modèle vient du temps où les machines puissantes étaient rares. On installait le calcul et les données sur un gros ordinateur central, et les utilisateurs y accédaient depuis des terminaux qui ne faisaient qu'afficher. Quand les ordinateurs personnels sont apparus, le terminal est devenu un programme capable de calculer lui-même : c'est la naissance du client.

La formalisation s'est faite avec les réseaux : pour que deux programmes distants dialoguent, il faut que l'un écoute à une adresse connue et que l'autre sache où frapper. L'asymétrie n'est donc pas un choix esthétique, elle est **imposée par la manière dont une connexion s'établit** : quelqu'un doit attendre à une adresse fixe, sinon personne ne peut initier le contact.

On peut le dire ainsi : pour qu'une communication s'établisse entre deux programmes, il faut qu'au moins l'un des deux soit joignable à une adresse connue à l'avance et disponible avant l'autre. Celui-là est, par définition, le serveur. L'autre, qui connaît cette adresse et choisit le moment de l'appel, est le client. Toute la règle découle de cette nécessité.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Le web.** Le navigateur est client, le serveur web répond. C'est l'exemple donné en cours et le plus familier : vous tapez une adresse, votre navigateur demande une page.
- **Les bases de données.** L'application est cliente, le moteur de base est serveur. Les notions de pool de connexions et de temps d'attente viennent directement de la file côté serveur.
- **Les files de messages.** Kafka est un serveur ; producteurs et consommateurs sont ses clients. C'est l'usage central de ce module.
- **Les API de service.** Un service de scoring de fraude exposé en HTTP est un serveur appelé par le système d'autorisation : la latence de cet appel entre directement dans le budget L.
- **La supervision.** Un agent installé sur une machine est client d'un serveur de métriques auquel il envoie ses mesures.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Quand un énoncé dit « interroge », « demande », « appelle ».** Le sujet du verbe est le client, le complément est le serveur. C'est le repérage le plus rapide.
- **Quand il faut décider de l'ordre de démarrage.** Le serveur démarre toujours en premier. Dans un `docker compose`, c'est ce qui dicte les dépendances entre conteneurs.
- **Quand une information doit partir du producteur.** Si l'énoncé dit « prévenir dès que », le client-serveur pur ne suffit pas : il faudrait que le client interroge sans cesse. C'est le signal qu'il faut une file de messages.
- **Quand on dimensionne.** Un seul serveur pour mille clients concentre toute la file d'attente ; c'est le point à surveiller en premier dans un calcul de latence.

▶ 11. Comment on va l'utiliser

Méthode · identifier les rôles

1. **Lister les programmes, pas les machines.** On écrit les processus en jeu : navigateur, API, base, Kafka, script Python.
2. **Tracer les flèches de demande.** Pour chaque paire, on se demande qui parle en premier. La flèche part du client.
3. **Noter l'adresse d'écoute de chaque serveur.** Par exemple `localhost:9092` pour Kafka. Sans cette adresse, le client ne peut pas exister.
4. **Déterminer l'ordre de démarrage.** On remonte les flèches : tout ce qui reçoit doit être prêt avant ce qui envoie.
5. **Identifier les files d'attente.** Chaque serveur sollicité par plusieurs clients porte une file ; on la note comme maillon dans le calcul de latence.

À retenir

Client et serveur sont des rôles, pas des machines : le serveur attend à une adresse connue, le client choisit le moment de la demande. L'initiative appartient toujours au client.

▶ 12. Exercices d'application corrigés

Application 6.1

Dans un TP, un script Python `producteur.py` envoie des mesures à Kafka ; un script `consommateur.py` les lit et écrit les alertes dans PostgreSQL ; une page web affiche ces alertes. Identifiez tous les rôles et donnez l'ordre de démarrage.

Corrigé 6.1

Étape 1 — lister les programmes. `producteur.py`, le serveur Kafka, `consommateur.py`, le serveur PostgreSQL, le serveur web, le navigateur.

Étape 2 — tracer les flèches. `producteur.py` → Kafka (producteur est client). `consommateur.py` → Kafka (il demande les messages : client). `consommateur.py` → PostgreSQL (client). Navigateur → serveur web (client). Serveur web → PostgreSQL (client).

Étape 3 — adresses d'écoute. Kafka sur `localhost:9092`, PostgreSQL sur `localhost:5432`, serveur web sur `localhost:8000`.

Étape 4 — ordre de démarrage. On remonte les flèches : d'abord Kafka et PostgreSQL (ils ne sont clients de personne), puis `consommateur.py` et le serveur web, puis `producteur.py`, et enfin le navigateur. Démarrer `producteur.py` avant Kafka provoquerait l'erreur classique « connexion refusée ».

Étape 5 — files d'attente. Kafka et PostgreSQL sont tous deux sollicités par plusieurs clients : ce sont les deux maillons à surveiller si la latence dépasse le budget.

Remarque. `consommateur.py` est client dans ses deux relations, mais il n'est serveur de personne ; le serveur web, lui, est serveur pour le navigateur et client pour PostgreSQL. C'est l'illustration directe du fait que les rôles se définissent par relation.

Application 6.2

Une application doit prévenir l'utilisateur dès qu'une alerte apparaît. Un développeur propose que le navigateur interroge le serveur toutes les secondes. Analysez cette solution avec la règle, puis proposez une alternative.

Corrigé 6.2

Analyse de la proposition. Elle respecte strictement le modèle : comme le serveur ne peut pas prendre l'initiative, le client compense en demandant sans arrêt. On appelle cela l'interrogation périodique.

Premier coût : le gaspillage. Si une alerte survient toutes les heures, le navigateur aura posé 3600 questions pour une seule réponse utile. Avec mille utilisateurs, cela fait 3,6 millions de requêtes par heure dont 999 999 inutiles, et autant d'entrées dans la file d'attente du serveur.

Second coût : la latence résiduelle. Une alerte née juste après une interrogation attend jusqu'à la suivante. On retrouve exactement la formule du batch de la notion 2 : avec une période `P = 1 s`, la latence moyenne ajoutée vaut `P/2 = 500 ms`. Réduire `P` améliore la latence et aggrave le gaspillage : les deux coûts s'opposent.

Alternative. Inverser l'initiative, en maintenant une connexion ouverte sur laquelle le serveur pousse l'alerte dès qu'elle existe (WebSocket, Server-Sent Events). La latence ajoutée tombe à quelques millisecondes et le nombre de messages devient égal au nombre d'alertes.

Conclusion. Cet exercice montre la limite précise du modèle client-serveur : il est excellent pour demander, mauvais pour être prévenu. C'est la raison d'être du modèle publication-abonnement que nous verrons au chapitre 3.

▶ 13. Exercices d'entraînement

Entraînement

1. Pour chaque paire, dites qui est client et qui est serveur : (a) application mobile et API ; (b) API et base ; (c) consommateur Python et Kafka ; (d) Grafana et base de métriques.
2. Un étudiant lance `producteur.py` avant Kafka. Quel message d'erreur attendez-vous et pourquoi ?
3. Expliquez pourquoi on dit que « serveur » désigne un rôle et non une machine, avec un exemple où un même programme tient les deux rôles.

▶ 14. Exercices de difficulté croissante

Progression

1. facile Complétez : « Dans notre cours, Kafka sera un … et les programmes Python qui lui envoient des messages seront des … ».
2. moyen Un serveur traite une demande en 20 ms et reçoit 80 demandes par seconde. Estimez le temps d'attente moyen en file si le serveur est mono-tâche, et dites à partir de quel débit il sature.
3. difficile Comparez interrogation périodique, interrogation longue et connexion poussée sur trois critères : latence ajoutée, nombre de messages, complexité de mise en œuvre. Concluez sur le choix pour un système d'alerte de fraude.

N7 — Le réseau

▶ 4. La règle

Règle · Le réseau

Un réseau relie des machines, comme des routes relient des villes ; Internet est un immense réseau de réseaux. Chaque machine y possède une adresse IP et des portes numérotées appelées ports. Le voyage d'un message prend du temps, et le réseau n'est pas parfait : un message peut être retardé, perdu, ou arriver deux fois.

On écrit une destination sous la forme `adresse:port`, par exemple `localhost:9092`. L'adresse désigne la machine, comme une adresse postale ; le port désigne le programme qui écoute sur cette machine, comme un numéro d'appartement.

Conditions d'emploi. Les trois imperfections (retard, perte, duplication) sont des propriétés du réseau lui-même : aucun programme ne peut les supprimer, il peut seulement les détecter et les compenser.

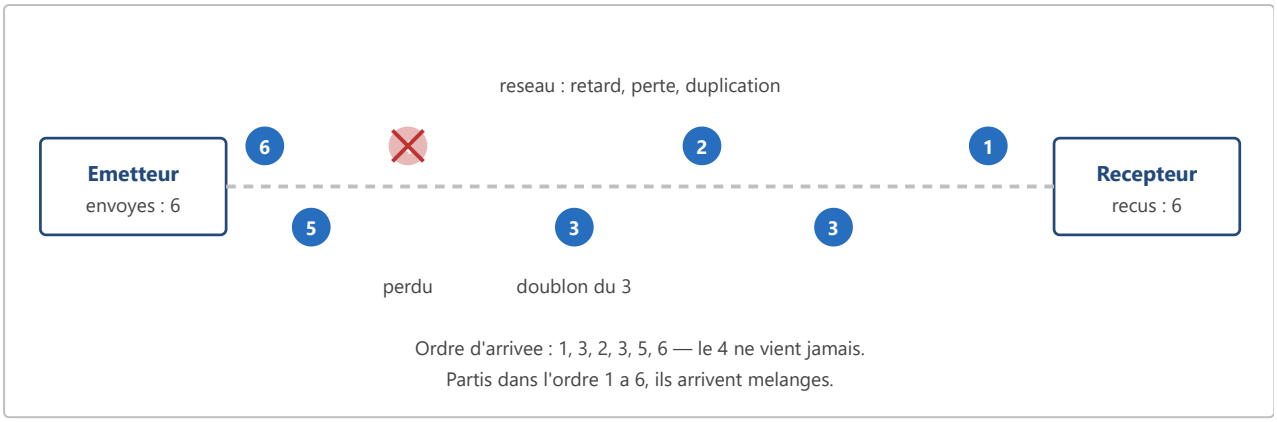


FIGURE 4 · LE RÉSEAU N'EST PAS PARFAIT

Les six messages partent numérotés, dans l'ordre. Ils arrivent mélangés (1, 3, 2, 3, 5, 6) : le 3 est reçu deux fois, le 4 n'arrive jamais. Retard, duplication et perte ne sont pas des pannes : c'est le fonctionnement normal d'un réseau partagé.

▶ 5. Explication de la règle

La règle dit trois choses de nature différente, et il faut les séparer. La première est une question d'**adressage** : pour joindre un programme, il ne suffit pas de connaître sa machine, il faut aussi savoir sur quelle porte il écoute. C'est pour cela que l'on écrit toujours deux informations : `localhost:9092` désigne le programme Kafka sur la machine locale, et un autre programme sur la même machine portera un autre numéro.

La deuxième est une question de **durée**. La lumière met environ 5 µs pour parcourir un kilomètre de fibre ; à cela s'ajoutent les équipements traversés. En pratique, quelques millisecondes dans une même salle machine, plus de 100 ms entre deux continents. Ce temps n'est pas négociable : aucune optimisation logicielle ne rendra un aller-retour Casablanca–Sydney plus rapide que la physique ne l'autorise. C'est pourquoi la notion 1 parlait de budget : le réseau en consomme une part fixe.

La troisième est la plus importante pour la suite du module : le réseau est **imparfait**. Un message peut arriver en retard parce qu'un équipement était saturé ; il peut être perdu parce qu'un tampon a débordé ; il peut arriver deux fois parce que l'émetteur, n'ayant pas reçu d'accusé, l'a renvoyé. Ces trois phénomènes ne sont pas des pannes exceptionnelles : ce sont le fonctionnement normal d'un réseau chargé.

Cas particuliers et pièges

- **« Perdu » et « très en retard » sont indiscernables.** L'émetteur qui n'a pas reçu d'accusé ne peut pas savoir si le message est perdu ou simplement lent. C'est cette ambiguïté qui engendre les doublons.
- **La duplication est une conséquence, pas une cause.** Elle naît du mécanisme de réémission ; on ne peut donc pas la supprimer sans renoncer à la fiabilité. On la compense par l'idempotence, au chapitre 5.
- **TCP ne résout pas tout.** TCP garantit l'ordre et l'absence de perte *à l'intérieur d'une connexion*. Dès qu'il y a plusieurs connexions, plusieurs chemins ou une reprise après coupure, le désordre et les doublons reviennent au niveau applicatif.
- **Attention à `localhost` dans Docker.** À l'intérieur d'un conteneur, `localhost` désigne le conteneur lui-même, pas votre machine. C'est l'erreur de TP la plus fréquente.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui de la **communication entre programmes qui ne partagent pas la même mémoire**. À l'intérieur d'un programme, passer une valeur d'une fonction à une autre est instantané et sûr. Dès que les deux programmes sont sur des machines différentes, il faut traverser un milieu physique que l'on ne contrôle pas.

La règle nomme ce que ce milieu coûte (du temps) et ce qu'il ne garantit pas (l'arrivée, l'unicité, l'ordre). Elle transforme ainsi une intuition fausse — « envoyer un message, c'est comme appeler une fonction » — en modèle utilisable.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous servira d'abord à **configurer correctement les TP** : toute connexion échoue pour l'une de trois raisons — mauvaise adresse, mauvais port, serveur non démarré — et la règle donne la liste.

Elle nous servira ensuite à **comprendre les garanties de livraison** du chapitre 5. Les expressions *au plus une fois*, *au moins une fois* et *exactement une fois* n'ont de sens que parce que le réseau perd et duplique ; sans cette règle, elles paraîtraient absurdes.

Elle nous servira enfin à **expliquer le désordre des événements**. Si les messages arrivaient toujours dans l'ordre, la notion 11 et tout le chapitre 4 sur les watermarks n'auraient pas lieu d'être.

▶ 8. Comment on y est arrivé

Origine de la règle

Les premiers réseaux reliaient deux machines par un câble dédié : la communication était fiable mais ne passait pas à l'échelle. L'invention décisive a été la **commutation de paquets** : au lieu de réserver un chemin, on découpe le message en paquets indépendants que chaque équipement achemine au mieux.

Ce choix explique directement les trois imperfections. Puisque chaque paquet est acheminé indépendamment, deux paquets peuvent prendre des chemins de durées différentes : d'où le *désordre*. Puisque les équipements ont des tampons finis, un paquet arrivant sur un tampon plein est jeté : d'où la *perte*. Puisque l'émetteur ne peut pas distinguer un paquet perdu d'un paquet lent, il réémet après un délai : d'où la *duplication*.

Les trois imperfections ne sont donc pas des défauts de réalisation : elles sont la **contrepartie logique du partage**. Un réseau qui ne perdrait jamais rien devrait réserver des ressources pour chaque communication, et coûterait un prix que personne n'accepte de payer à l'échelle d'Internet.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Configuration des clients et serveurs.** Chaque fichier de configuration de TP contient une adresse et un port ; comprendre ce couple évite des heures de blocage.
- **Calcul de budget de latence.** Dans la décomposition de L, les maillons « réseau aller » et « réseau retour » sont souvent les plus gros ; on les estime à partir de la distance.
- **Choix de la localisation des serveurs.** Placer un service près des utilisateurs est la seule façon de réduire la part incompressible de la latence ; c'est la troisième raison d'utiliser plusieurs machines, vue à la notion 12.
- **Conception des traitements idempotents.** Puisque les doublons sont inévitables, tout traitement qui modifie un état doit pouvoir être rejoué sans dommage.
- **Diagnostic d'incident.** Un pic de latence sans pic de charge désigne presque toujours le réseau ou une file intermédiaire.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Quand deux composants sont sur des machines différentes.** Dès que le schéma montre une flèche franchissant une frontière de machine, la règle s'applique et il faut budgéter du temps et prévoir les imperfections.
- **Quand un résultat compte deux fois le même événement.** C'est la signature d'un doublon réseau ; on ne cherche pas un bug de calcul, on cherche un mécanisme de réémission.
- **Quand des événements arrivent dans le désordre.** Ce n'est pas une anomalie : c'est le comportement attendu. Le réflexe est de passer à la notion 11 et de raisonner en heure d'événement.
- **Quand une connexion est refusée.** Trois vérifications dans l'ordre : le serveur tourne-t-il, écoute-t-il bien sur ce port, l'adresse est-elle la bonne depuis là où l'on appelle (attention aux conteneurs).

▶ 11. Comment on va l'utiliser

Méthode · raisonner sur un échange réseau

1. **Écrire la destination complète.** Toujours sous la forme `adresse:port`, et vérifier que l'adresse est valide *depuis le programme appelant*.
2. **Estimer le temps de trajet.** Même salle : 1 à 5 ms. Même pays : 10 à 30 ms. Autre continent : 100 ms et plus. On compte l'aller *et* le retour.
3. **Inscrire ce temps dans le budget.** On l'ajoute aux maillons de la notion 1 avant de conclure sur $L \leq D$.
4. **Énumérer les trois imperfections pour ce lien.** Que se passe-t-il si ce message est retardé ? perdu ? reçu deux fois ? On répond pour chacune, même si la réponse est « sans conséquence ».
5. **Choisir la compensation.** Accusé de réception et réémission contre la perte ; identifiant unique et traitement idempotent contre le doublon ; horodatage d'événement contre le désordre.

À retenir

Un message peut être retardé, perdu ou arriver deux fois : ce n'est pas une panne, c'est le fonctionnement normal d'un réseau partagé. Toute la difficulté du temps réel vient en partie de là.

▶ 12. Exercices d'application corrigés

Application 7.1

Un terminal de paiement à Casablanca interroge un service de scoring hébergé à Francfort. Le trajet aller dure 48 ms, le retour 48 ms, le calcul 35 ms, la capture et la mise en forme 12 ms. Le délai utile est de 300 ms. La conception tient-elle ? Le service est ensuite déplacé à Casablanca : trajet 3 ms dans chaque sens. Que gagne-t-on ?

Corrigé 7.1

Étape 1 — destination. Le lien franchit une frontière de machine et même de continent : la règle s'applique pleinement.

Étape 2 — temps de trajet. Aller-retour : `48 + 48 = 96 ms`. L'ordre de grandeur est cohérent avec une liaison intercontinentale courte.

Étape 3 — budget. `L = 12 + 48 + 35 + 48 = 143 ms`. Or `D = 300 ms`, donc `L ≤ D` : la conception tient, avec une marge de 157 ms.

Analyse du déplacement. Avec un service local : `L' = 12 + 3 + 35 + 3 = 53 ms`. Le gain est de `143 − 53 = 90 ms`, soit 63 % de la latence initiale.

Interprétation. Avant le déplacement, le réseau consommait `96/143 = 67 %` du budget et le calcul seulement 24 %. Optimiser le modèle de scoring pour gagner 10 ms aurait été un mauvais investissement : le maillon dominant était le réseau. C'est exactement ce que la méthode de la notion 1 demande de vérifier avant d'optimiser.

Étape 4 — imperfections. Retard : la marge de 157 ms absorbe un retard modéré. Perte : il faudra une politique de délai d'attente, avec décision par défaut (accepter ou refuser) si le service ne répond pas. Doublon : une double demande de score est sans conséquence car le scoring ne modifie aucun état ; il est naturellement idempotent.

Application 7.2

Un consommateur reçoit les messages numérotés dans cet ordre : `1, 2, 4, 3, 4, 6`. Le message 5 n'arrive jamais. Identifiez chaque anomalie, dites ce qui l'a causée, et indiquez la compensation à mettre en place.

Corrigé 7.2

Anomalie 1 — désordre (4 avant 3). Cause : les paquets 3 et 4 ont emprunté des chemins de durées différentes, ou l'un a attendu dans un tampon plus chargé. Compensation : ne jamais se fier à l'ordre d'arrivée ; raisonner sur l'horodatage d'événement porté par le message lui-même (notion 11), et attendre un court délai avant de clore un calcul.

Anomalie 2 — duplication (4 reçu deux fois). Cause : l'émetteur n'a pas reçu l'accusé de réception du message 4 à temps et l'a réémis ; le premier exemplaire est finalement arrivé aussi. Compensation : donner un identifiant unique à chaque message et rendre le traitement idempotent, c'est-à-dire conçu pour que le rejouer ne change rien au résultat.

Anomalie 3 — perte (5 absent). Cause : un tampon saturé sur un équipement intermédiaire a jeté le paquet, et la réémission a elle aussi échoué ou n'a pas eu lieu. Compensation : accusés de réception et réémission, c'est-à-dire la garantie *au moins une fois* ; on notera qu'elle produit mécaniquement davantage de doublons, ce qui rend l'idempotence encore plus nécessaire.

Conclusion. Les trois anomalies se tiennent : lutter contre la perte augmente les doublons, et tolérer le désordre demande d'attendre, donc d'allonger la latence. C'est un arbitrage, non un problème à résoudre une fois pour toutes ; le chapitre 5 le formalisera sous le nom de garanties de livraison.

▶ 13. Exercices d'entraînement

Entraînement

1. Expliquez ce que désignent respectivement `localhost` et `9092` dans `localhost:9092`, avec l'analogie postale.
2. Pourquoi un émetteur ne peut-il pas distinguer un message perdu d'un message très en retard ? Quelle conséquence cela a-t-il ?
3. Un conteneur Docker n'arrive pas à joindre Kafka sur `localhost:9092`. Expliquez l'erreur et proposez la correction.

▶ 14. Exercices de difficulté croissante

Progression

1. facile Citez les trois imperfections du réseau énoncées par la règle.
2. moyen Une liaison a un aller-retour de 180 ms. Le protocole exige trois allers-retours avant d'envoyer la donnée utile. Calculez le temps perdu en établissement et dites si `D = 500 ms` reste tenable avec un calcul de 40 ms.
3. difficile Montrez qu'il est impossible de garantir à la fois « aucune perte » et « aucun doublon » avec un simple mécanisme d'accusé de réception, puis expliquez comment l'idempotence permet de contourner cette impossibilité en pratique.

N8 — Le message JSON, ou l'événement

▶ 4. La règle

Règle · Message et format JSON

Pour se comprendre, des programmes doivent s'envoyer des messages dans un format commun. JSON est une façon simple d'écrire des données en texte : entre accolades, une liste de paires « nom » : valeur, où les textes sont entre guillemets et les nombres non. Un message qui décrit quelque chose qui s'est passé s'appelle un événement.

```
{
  "carte":     "4970-XXXX-1234",
  "montant":   18.0,
  "devise":    "MAD",
  "commerce":  "Cafe Atlas",
  "ville":     "Casablanca",
  "heure":     "2026-10-15T08:30:12Z"
}
```

Conditions d'emploi. JSON suppose que les deux programmes se sont mis d'accord sur les *noms des champs* et sur leur signification : le format ne transporte pas le sens. Il suppose aussi que l'on accepte un coût : le texte occupe plus de place qu'une représentation binaire.

▶ 5. Explication de la règle

La règle répond à une question très concrète : un programme Python et un programme Java n'ont pas la même façon de représenter un nombre ou un texte en mémoire. Pour qu'ils échangent, il faut un **terrain neutre** : une suite de caractères que chacun sait écrire et relire. JSON est ce terrain neutre, et son succès vient de ce qu'il est lisible à la fois par une machine et par un humain — ce qui n'a l'air de rien jusqu'au jour où l'on doit diagnostiquer un incident à trois heures du matin.

La deuxième idée, plus profonde, est l'identité entre **message et événement**. Le message ci-dessus ne décrit pas un état (« le solde est de 1200 MAD »), il décrit un *fait déjà arrivé* (« un paiement de 18 MAD a eu lieu à 8 h 30 min 12 s »). Cette nuance commande tout le reste : un fait passé ne se modifie pas. On ne corrige pas un événement, on en émet un nouveau qui annule ou complète le précédent. C'est le fondement de l'*event sourcing* du chapitre 2.

On retrouve ici exactement les trois éléments de contexte de la notion 3 : le **sujet** (`carte`), les **grandeurs** avec leurs unités implicites ou explicites (`montant` et `devise`), et l'**instant** (`heure`). Un message JSON bien conçu n'est donc pas autre chose qu'un enregistrement contextualisé mis sous une forme transportable.

Cas particuliers et pièges

- **JSON ne porte pas de schéma.** Rien n'empêche un producteur de renommer un champ du jour au lendemain. En production on ajoute donc un registre de schémas, ou l'on passe à un format qui embarque sa structure (Avro, Protobuf) — chapitre 3.
- **Les nombres à virgule flottante sont trompeurs.** Pour de l'argent, on préfère souvent un entier de centimes (`1800`) à un flottant (`18.0`), afin d'éviter les erreurs d'arrondi.
- **Le coût en volume est réel.** Les noms de champs sont répétés dans chaque message. À un million de messages par seconde, cela se paie en réseau et en stockage.
- **Imbriquer n'est pas interdit mais coûte cher.** Un JSON profondément imbriqué est pénible à requêter en SQL continu ; on préfère des structures plates quand c'est possible.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui de l'**interopérabilité**. Deux programmes écrits dans des langages différents, compilés différemment, tournant sur des systèmes différents, doivent néanmoins s'échanger des données sans perte de sens. Avant les formats texte universels, chaque intégration demandait un travail spécifique de conversion.

Le second problème, plus discret, est celui de la **représentation d'un fait**. Un système temps réel ne manipule pas des états mais des changements d'état ; il lui faut une forme capable de dire « ceci s'est produit, à ce moment, à propos de ce sujet ». L'événement JSON est cette forme.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous servira dans **chaque TP** : les producteurs Python sérialiseront leurs mesures en JSON avant de les envoyer à Kafka, et les consommateurs les désérialiseront. Savoir lire un JSON à l'œil permet de vérifier instantanément ce qui circule dans un sujet Kafka.

Elle nous servira également à **concevoir les schémas d'événements**. Le choix des champs est une décision d'architecture : tout champ absent du message est définitivement perdu pour les consommateurs, car ils n'ont pas accès à la source.

Enfin, elle prépare la notion d'**immutabilité**, centrale au chapitre 2 : puisqu'un événement décrit un fait passé, un journal d'événements ne se réécrit pas, il s'allonge.

▶ 8. Comment on y est arrivé

Origine de la règle

Les premiers échanges entre machines utilisaient des formats binaires propres à chaque constructeur : compacts, rapides, mais illisibles et incompatibles entre eux. Sont venus ensuite les formats texte structurés, dont XML, très expressif mais verbeux. JSON est apparu comme une simplification radicale : quatre types de base, deux structures (objet et tableau), et c'est tout.

Le raisonnement qui le justifie tient en trois contraintes qu'un format d'échange doit satisfaire simultanément. Il doit être *indépendant du langage*, sinon l'interopérabilité disparaît. Il doit être *auto-descriptif au niveau des noms*, sinon un consommateur ne peut pas savoir ce que représente la troisième valeur. Et il doit être *bon marché à analyser*, sinon le coût de lecture mange le budget de latence.

JSON satisfait les trois avec un minimum de règles, ce qui explique qu'il se soit imposé alors qu'il n'offre ni typage fort ni schéma. Ses faiblesses sont le prix de sa simplicité : c'est un compromis assumé, pas un oubli.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Messages dans les files.** Le contenu d'un message Kafka est très souvent du JSON ; c'est le cas par défaut dans tous nos TP.
- **API web.** Les échanges entre une application mobile et son service sont en JSON ; le même format sert donc à la requête-réponse et à l'événement.
- **Fichiers de configuration et journaux.** Les journaux structurés en JSON sont directement interrogeables ; c'est ce qui permet aux outils d'analyse de logs de fonctionner.
- **Échange entre équipes.** Un schéma JSON sert de contrat lisible entre l'équipe qui produit les événements et celles qui les consomment.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Quand on démarre un projet ou un TP.** JSON est le bon choix par défaut : lisible, outillé partout, suffisant tant que le débit reste modéré.
- **Quand plusieurs équipes consomment le même flux.** La lisibilité devient un atout décisif : chacun peut inspecter un message sans outil spécial.
- **Quand le débit devient très élevé.** Au-delà de quelques centaines de milliers de messages par seconde, le coût du texte justifie de passer à un format binaire avec schéma.
- **Quand les champs changent souvent.** Signal qu'il faut un registre de schémas ; sans lui, chaque évolution casse silencieusement un consommateur.

▶ 11. Comment on va l'utiliser

Méthode · concevoir un événement JSON

1. **Nommer l'événement au passé.** `paiement_effectue`, `mesure_relevee`, `commande_annulee` : le nom rappelle qu'il s'agit d'un fait accompli et non d'un ordre.
2. **Placer le sujet en premier champ.** C'est lui qui servira de clé à la notion 9 ; le mettre en tête rend les messages lisibles d'un coup d'œil.
3. **Ajouter les grandeurs avec leurs unités.** On applique la méthode de la notion 3 : unité dans le nom du champ, un seul fait par champ.
4. **Ajouter l'horodatage de l'événement.** Au format `2026-10-15T08:30:12Z`, en temps universel ; on y reviendra aux notions 10 et 11.
5. **Ajouter un identifiant unique de message.** Il permettra de repérer les doublons dont parle la notion 7 et de rendre le traitement idempotent.
6. **Relire et valider.** On vérifie que les guillemets encadrent les textes, que les nombres n'en ont pas, et qu'un consommateur extérieur pourrait interpréter le message sans documentation supplémentaire.

À retenir

Un message JSON qui décrit un fait déjà arrivé est un événement ; comme le fait est passé, l'événement ne se modifie jamais : on en émet un nouveau.

▶ 12. Exercices d'application corrigés

Application 8.1

Écrivez l'événement JSON correspondant à : « Le capteur de vibration VIB-12 de la pompe P3 a mesuré 7,4 mm/s le 15 octobre 2026 à 8 h 30 min 12 s UTC. » Appliquez les six étapes de la méthode.

Corrigé 8.1

Étape 1 — nommer au passé. L'événement s'appelle `mesure_relevee` : un fait accompli, pas un ordre de mesurer.

Étape 2 — sujet en premier. Le sujet est le capteur, `VIB-12`. La pompe est une information de contexte utile mais secondaire.

Étape 3 — grandeurs et unités. La vibration se mesure ici en millimètres par seconde : le champ s'appellera `vibration_mm_s`, jamais `valeur`.

Étape 4 — horodatage. `"2026-10-15T08:30:12Z"`, avec le `Z` du temps universel.

Étape 5 — identifiant unique. On ajoute `message_id`, qui servira à détecter les doublons.

```
{
  "type":            "mesure_relevee",
  "capteur":         "VIB-12",
  "equipement":      "P3",
  "vibration_mm_s":  7.4,
  "evenement_le":    "2026-10-15T08:30:12Z",
  "message_id":      "9f1c7a30-5e2b-4d8a-9c11-2f0ab7e45d93"
}
```

Étape 6 — relecture. Tous les textes sont entre guillemets, le nombre `7.4` ne l'est pas, chaque champ porte un seul fait. Un consommateur extérieur comprend le message seul : la condition de la notion 3 est satisfaite.

Application 8.2

Un consommateur reçoit ce message : `{ "carte": 4970XXXX1234, "montant": "18,0", heure: 2026-10-15 }`. Relevez toutes les erreurs de syntaxe et de conception, puis réécrivez-le correctement.

Corrigé 8.2

Erreur 1 — `"carte": 4970XXXX1234`. La valeur n'est pas entre guillemets alors qu'elle contient des lettres : ce n'est ni un nombre valide ni un texte valide. Le document JSON est rejeté dès cette ligne. Même un numéro purement chiffré devrait rester un texte : on ne fait jamais de calcul sur un numéro de carte, et le passer en nombre perdrait les zéros de tête.

Erreur 2 — `"montant": "18,0"`. Deux fautes cumulées. Le montant est un nombre, il ne doit pas être entre guillemets. Et la virgule décimale française n'existe pas en JSON : le séparateur est le point.

Erreur 3 — `heure:` sans guillemets. En JSON, les noms de champs sont obligatoirement entre guillemets, contrairement à la notation d'objet de JavaScript. C'est une confusion très fréquente.

Erreur 4 — `2026-10-15` comme valeur nue. Non guillemetée, elle serait lue comme une soustraction. De plus la date seule ne suffit pas pour du temps réel : sans heure ni fuseau, impossible de calculer une latence.

Erreur 5 — conception. Il manque la devise (18 dirhams ou 18 euros ?), le commerce, et un identifiant de message.

Version corrigée :

```
{
  "type":         "paiement_effectue",
  "carte":        "4970-XXXX-1234",
  "montant":      18.0,
  "devise":       "MAD",
  "commerce":     "Cafe Atlas",
  "evenement_le": "2026-10-15T08:30:12Z",
  "message_id":   "7c2e18a4-0b55-4f6d-8e3a-1d9b4c6f2017"
}
```

▶ 13. Exercices d'entraînement

Entraînement

1. Écrivez en JSON : « La trottinette TR-204 a été déverrouillée par l'utilisateur U-981 à Rabat le 3 mars 2027 à 14 h 07 min 33 s UTC, batterie à 62 %. »
2. Pourquoi dit-on qu'un événement ne se modifie pas ? Comment corrige-t-on alors une erreur déjà publiée ?
3. Citez deux inconvénients de JSON à très haut débit et nommez une alternative pour chacun.

▶ 14. Exercices de difficulté croissante

Progression

1. **facile** Dans le message de Salma, dites quels champs sont des textes et lesquels sont des nombres, et pourquoi.
2. **moyen** Un producteur renomme le champ `heure` en `evenement_le` sans prévenir. Décrivez précisément ce qui arrive à un consommateur existant, et proposez une procédure de migration sans interruption.
3. **difficile** Un flux produit 400 000 messages par seconde, chacun de 280 octets en JSON dont 160 octets de noms de champs répétés. Calculez le débit réseau en mégaoctets par seconde, puis l'économie annuelle de stockage qu'apporterait un format binaire supprimant ces 160 octets.

N9 — La paire clé–valeur

▶ 4. La règle

Règle · Clé et valeur

Beaucoup de systèmes rangent les messages sous la forme (clé, valeur). La clé dit de qui ou de quoi on parle ; la valeur contient le détail.

clé = « carte 4970-1234 » valeur = { montant : 18, ville : « Casablanca », … }

Conséquence essentielle : **tous les messages portant la même clé seront rangés ensemble**, dans le même casier, et donc traités dans leur ordre d'arrivée par le même traitement.

Conditions d'emploi. La clé doit être stable dans le temps (un même sujet garde la même clé) et suffisamment variée pour que les casiers se remplissent à peu près également.

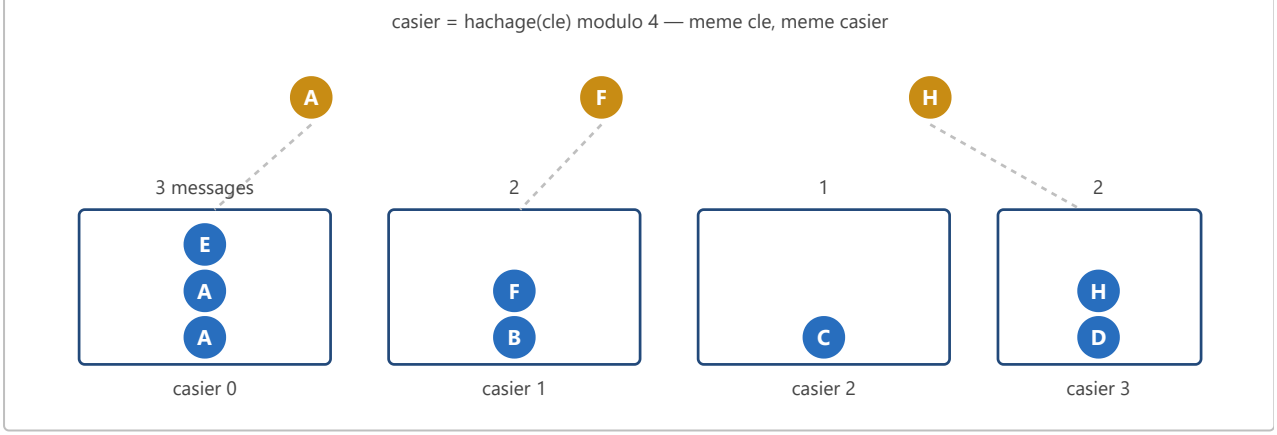


FIGURE 5 · LA CLÉ DÉCIDE DU CASIER
Les clés *A*, *B*, *C*… tombent toujours dans le même casier, quel que soit le moment de leur arrivée ; ici *A* et *E* partagent le casier 0. C'est exactement le mécanisme de partitionnement de Kafka, que nous verrons au chapitre 3.

▶ 5. Explication de la règle

L'image donnée en cours est celle du **casier de vestiaire** : le numéro du casier est la clé, ce qu'il contient est la valeur. On ne fouille pas tout le vestiaire pour retrouver son manteau ; on va directement au bon casier. La clé sert donc à deux choses : *identifier* et *ranger*.

La propriété qui compte vraiment pour le temps réel est la seconde. Puisque tous les messages d'une même clé atterrissent dans le même casier, et qu'un casier est lu par un seul traitement à la fois, on obtient gratuitement deux garanties. D'abord l'**ordre relatif par sujet** : les paiements d'une même carte seront vus dans l'ordre où ils sont arrivés, même si l'ordre global entre cartes différentes n'est

pas respecté. Ensuite la **localité de l'état** : le traitement qui suit l'historique de cette carte n'a besoin que de ce qu'il a déjà vu dans son casier.

C'est cette localité qui rend le traitement distribué possible. Sans clé, calculer « le montant cumulé de cette carte sur les 5 dernières minutes » exigerait que toutes les machines se concertent à chaque message. Avec la clé, chaque machine travaille seule sur ses propres casiers, sans jamais consulter les autres. Le choix de la clé est donc une décision de performance autant que de modélisation.

Cas particuliers et pièges

- **La clé déséquilibrée.** Si 90 % des messages portent la même clé, un casier reçoit 90 % de la charge et les autres dorment. On appelle cela une clé « chaude » ; c'est la première cause de lenteur inexpliquée.
- **Pas de clé du tout.** Un message sans clé est réparti au hasard : on gagne un équilibrage parfait, on perd toute garantie d'ordre. C'est acceptable pour des mesures indépendantes, jamais pour un historique.
- **L'ordre global n'existe pas.** Entre deux casiers, aucun ordre n'est garanti. Si un exercice demande l'ordre global, c'est qu'il faut soit un seul casier, soit raisonner en heure d'événement (notion 11).
- **Changer le nombre de casiers rebat les cartes.** Les clés se redéploient, et une clé peut changer de casier : l'ordre historique est alors rompu.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui du **partage du travail sans perte de cohérence**. Dès qu'on veut traiter plus de messages qu'une machine n'en supporte, il faut en confier une partie à d'autres machines. Mais si l'on répartit au hasard, deux paiements de la même carte peuvent être traités simultanément par deux machines différentes, qui ignoreront chacune ce que l'autre a vu.

La règle résout ce problème en faisant de la clé le critère de répartition : on partage le travail par *sujet* et non au hasard. Chaque sujet reste entier chez un seul traitement, et la cohérence est préservée sans coordination.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous permettra de **comprendre les partitions Kafka** au chapitre 3 : un casier est exactement une partition, et le nombre de partitions fixe le degré de parallélisme maximal d'un groupe de consommateurs.

Elle nous permettra aussi de **choisir la clé d'un flux**, qui est une des décisions les plus structurantes d'un projet. Une clé mal choisie se paie en déséquilibre de charge, et se corrige difficilement une fois le système en production.

Enfin elle éclaire les **agrégations par groupe** du chapitre 4 : le `GROUP BY` d'une requête continue s'appuie naturellement sur la clé de partitionnement, et le choix des deux doit être cohérent.

▶ 8. Comment on y est arrivé

Origine de la règle

La paire clé–valeur vient des structures de données les plus anciennes de l'informatique : la table de hachage, où une clé donne accès directement à une valeur sans parcourir l'ensemble. Le dictionnaire Python en est un exemple quotidien.

Son application au partitionnement repose sur un raisonnement simple. On veut répartir les messages sur `n` casiers en respectant deux exigences contradictoires : l'équilibre de la charge, et le regroupement des messages d'un même sujet. Une répartition aléatoire satisfait la première et viole la seconde ; une répartition par sujet satisfait la seconde mais risque le déséquilibre.

La solution retenue est le **hachage de la clé** : on calcule `casier = h(clé) mod n`, où `h` est une fonction de hachage. Comme `h` est déterministe, une même clé donne toujours le même casier : la seconde exigence est satisfaite. Comme `h` disperse uniformément des clés variées, la charge s'équilibre : la première l'est aussi, *à condition que les clés soient effectivement variées*. On voit ainsi d'où vient précisément le problème de la clé chaude : ce n'est pas un défaut du hachage, c'est une violation de son hypothèse.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Partitionnement des files de messages.** Kafka utilise exactement ce principe ; la clé du message détermine sa partition, et donc le consommateur qui le traitera.
- **Suivi d'un sujet dans le temps.** Historique d'une carte, série de mesures d'un capteur, parcours d'un utilisateur : tout calcul qui a besoin du passé d'un sujet exige que ce sujet soit la clé.
- **Bases clé–valeur et caches.** Redis, DynamoDB ou un simple cache mémoire reposent sur la même structure : accès direct par clé.
- **Agrégations distribuées.** Le regroupement par clé est ce qui permet à dix machines de calculer dix groupes différents sans jamais se parler.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Quand l'énoncé dit « par ».** « Par carte », « par capteur », « par utilisateur » : le mot qui suit désigne directement la clé à choisir.
- **Quand le calcul a besoin du passé d'un sujet.** Cumul, moyenne glissante, détection de répétition : tous exigent que les messages du sujet arrivent au même endroit, donc une clé.
- **Quand l'ordre compte à l'intérieur d'un sujet.** « Le second paiement en moins d'une minute » n'a de sens que si les deux paiements sont vus dans l'ordre : la clé le garantit.
- **Quand la charge est déséquilibrée.** Si un casier est saturé et les autres vides, la clé est en cause : il faut l'enrichir, par exemple en lui ajoutant un suffixe.

▶ 11. Comment on va l'utiliser

Méthode · choisir une clé

1. **Identifier le sujet du calcul.** On se demande : de quoi veut-on suivre l'historique ? La réponse est la clé candidate.
2. **Vérifier la stabilité.** La clé doit désigner le même sujet aujourd'hui et dans six mois. Une clé qui change (le nom d'un commerce, par exemple) rompt l'historique.
3. **Vérifier la cardinalité.** Il faut nettement plus de clés distinctes que de casiers ; avec 4 clés pour 12 casiers, huit casiers resteront vides.
4. **Vérifier l'équilibre.** On estime la distribution : si une clé concentre plus d'un casier de charge, elle est chaude. On la corrige en ajoutant un suffixe (`clé#0`, `clé#1`…), au prix de l'ordre à l'intérieur de cette clé.
5. **Vérifier la cohérence avec le regroupement.** La clé de partitionnement doit correspondre au `GROUP BY` prévu ; sinon chaque agrégation exigera un redéploiement coûteux des données entre machines.

À retenir

La clé n'identifie pas seulement : elle range. Tous les messages d'une même clé vont au même casier, ce qui garantit leur ordre relatif et permet de distribuer le travail sans coordination.

▶ 12. Exercices d'application corrigés

Application 9.1

On veut détecter « plus de 3 paiements de la même carte en moins de 2 minutes ». Le flux est réparti sur 6 casiers. Quelle clé choisissez-vous ? Que se passerait-il si l'on choisissait la ville ? Et si l'on n'en choisissait aucune ?

Corrigé 9.1

Étape 1 — sujet du calcul. La règle porte sur « la même carte » : le sujet est donc la carte, et la clé est le numéro de carte.

Étape 2 — stabilité. Un numéro de carte ne change pas pendant sa durée de validité : la clé est stable.

Étape 3 — cardinalité. Des millions de cartes pour 6 casiers : largement suffisant.

Étape 4 — équilibre. Aucune carte ne concentre une part notable du trafic : la distribution est naturellement équilibrée.

Si l'on choisissait la ville. Deux problèmes se cumulent. D'abord la cardinalité est faible : quelques dizaines de villes, dont une poignée concentrant l'essentiel des paiements — Casablanca serait une clé chaude, un casier recevrait la majorité du trafic. Ensuite et surtout, *le calcul deviendrait faux* : les paiements d'une même carte faits dans deux villes différentes partiraient dans deux casiers, donc vers deux traitements qui ne se parlent pas. Aucun des deux ne verrait les trois paiements, et la fraude passerait — or c'est précisément le scénario d'une carte volée utilisée dans plusieurs villes.

Si l'on ne choisissait aucune clé. Les messages seraient répartis au hasard : équilibrage parfait, mais les paiements d'une même carte seraient dispersés sur les 6 casiers. La détection serait impossible sans une coordination entre les six traitements, c'est-à-dire sans renoncer à l'avantage de la distribution.

Conclusion. La clé n'est pas un détail de configuration : elle détermine ce que le système est capable de calculer.

Application 9.2

Un flux de commandes utilise comme clé le code du magasin. Sur 8 casiers, on observe que le casier 3 traite 70 % des messages et devient le goulot d'étranglement. Diagnostiquez et proposez deux corrections en indiquant ce que chacune coûte.

Corrigé 9.2

Diagnostic. Le hachage n'est pas en cause : il répartit correctement des clés variées. Ce qui est en cause, c'est l'hypothèse de variété. Un magasin — probablement le magasin phare ou la boutique en ligne — concentre 70 % des commandes ; sa clé est « chaude », et tout son trafic tombe mécaniquement dans un seul casier.

Correction 1 — enrichir la clé par un suffixe. On remplace `MAG-03` par `MAG-03#k` avec `k` tiré entre 0 et 9 : le trafic du gros magasin se répartit alors sur jusqu'à dix casiers. *Coût :* l'ordre relatif à l'intérieur de ce magasin est perdu, et tout calcul qui a besoin de l'historique complet du magasin devra réunir les dix sous-clés dans une seconde étape. On ne l'accepte que si le calcul le permet (un simple comptage, oui ; une détection de séquence, non).

Correction 2 — changer de clé. Si le calcul porte en réalité sur la commande ou sur le client, on prend l'identifiant de commande ou de client comme clé : la cardinalité devient très élevée et la distribution s'équilibre d'elle-même. *Coût :* on perd la garantie d'ordre et la localité d'état au niveau du magasin ; ce n'est acceptable que si aucun calcul n'en dépend.

Ce qu'il ne faut pas faire. Augmenter le nombre de casiers de 8 à 32 ne résout rien : la clé chaude restera entièrement dans un seul casier, puisque `h(clé) mod n` lui attribue toujours une destination unique. C'est l'erreur de réflexe la plus courante.

▶ 13. Exercices d'entraînement

Entraînement

1. Pour chaque besoin, donnez la clé : (a) moyenne glissante de température par capteur ; (b) nombre de pages vues par session ; (c) détection de connexions répétées depuis une même adresse IP.
2. Expliquez avec l'image du vestiaire pourquoi deux messages de clés différentes n'ont aucun ordre garanti entre eux.
3. Qu'est-ce qu'une clé chaude ? Donnez un exemple tiré du commerce en ligne et dites comment vous le détecteriez.

▶ 14. Exercices de difficulté croissante

Progression

1. facile Dans l'exemple du cours, quelle est la clé et que contient la valeur ?
2. moyen Avec 4 casiers et des clés dont le hachage donne respectivement 17, 22, 35, 40 et 22, indiquez le casier de chaque message et commentez l'équilibre obtenu.
3. difficile On passe de 4 à 6 casiers sur un flux en production. Montrez, sur les clés de l'exercice précédent, que certaines changent de casier, et expliquez les conséquences sur l'ordre et sur l'état déjà accumulé. Proposez une procédure de migration qui limite les dégâts.

N10 — L'horodatage et les unités de temps

▶ 4. La règle

Règle · Horodatage

Un horodatage est la date et l'heure précises attachées à une donnée, comme le cachet de la poste sur une lettre.

2026-10-15T08:30:12.250Z

On lit : année–mois–jour, puis T, puis heure:minute:seconde.milliseconde, puis Z qui indique le temps universel coordonné (UTC). Les unités à connaître sont :

Unité	Symbole	Valeur	Pour se représenter
seconde	s	1 s	Le temps de dire « un crocodile »
milliseconde	ms	1 s = 1000 ms	Un clignement d'œil dure 100 à 150 ms
microseconde	µs	1 ms = 1000 µs	Le temps de réaction des systèmes boursiers
minute, heure	min, h	1 h = 3600 s	Le rythme des rapports classiques

Conditions d'emploi. Chaque machine possède sa propre horloge, et les horloges ne sont jamais parfaitement à l'heure. Un horodatage n'est donc fiable que relativement à l'horloge qui l'a produit.

▶ 5. Explication de la règle

Un horodatage répond à la troisième question de la notion 3 : « à quel moment ? ». Mais c'est aussi, et surtout, **l'instrument de mesure du temps réel**. Toute la notion 1 reposait sur $L = t_r - t_e$; sans horodatage, ces deux instants n'existent pas et la latence n'est pas mesurable. On ne peut pas améliorer ce qu'on ne mesure pas.

Le format retenu n'est pas arbitraire. Écrire l'année en premier, puis le mois, puis le jour, fait coïncider l'**ordre alphabétique et l'ordre chronologique** : trier des horodatages comme du texte donne le bon résultat. Le `Z` final est tout aussi important : il ancre l'instant dans une référence unique, commune à toutes les machines du monde. Un horodatage « 8 h 30 » sans fuseau est ininterprétable dès que deux machines ne sont pas dans le même pays.

L'avertissement de la règle sur les horloges est le point le plus lourd de conséquences. Deux machines voisines peuvent différer de plusieurs dizaines de millisecondes, parfois davantage après une dérive. Comparer un horodatage produit par la machine A à un horodatage produit par la machine B revient donc à comparer deux mesures faites avec deux règles légèrement différentes. Dans un système distribué, cela interdit de conclure « A s'est produit avant B » sur la seule foi de deux horodatages, dès que l'écart mesuré est du même ordre que l'erreur des horloges.

Cas particuliers et pièges

- **Secondes ou millisecondes ?** Un horodatage numérique (« temps Unix ») peut compter les secondes ou les millisecondes depuis le 1er janvier 1970. Un facteur 1000 d'erreur place l'événement en 1970 ou en l'an 56 000.
- **Heure locale et changement d'heure.** Enregistrer en heure locale crée, deux fois par an, une heure qui se répète ou une heure qui n'existe pas. On stocke toujours en UTC, on n'affiche en local qu'à la dernière seconde.
- **La précision n'est pas l'exactitude.** Afficher trois décimales ne garantit pas que la milliseconde soit juste ; elle dépend de la synchronisation de l'horloge.
- **Une horloge peut reculer.** Lors d'une correction, l'heure peut sauter en arrière, et deux événements successifs recevoir des horodatages décroissants.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui de la **datation commune de faits produits en des lieux différents**. Un terminal à Casablanca, un serveur à Francfort et un capteur à Tanger produisent chacun des faits qu'il faudra ensuite ordonner, comparer, agréger par tranches de temps. Sans référence commune, aucune de ces opérations n'a de sens.

Le second problème est celui de la **mesure des durées courtes**. Le temps réel se joue à l'échelle de la milliseconde ; un vocabulaire qui s'arrête à la seconde ne permet même pas d'énoncer le problème.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous servira à **calculer toutes les latences du module**. Chaque fois que nous dirons « ce traitement prend 40 ms », ce sera une soustraction entre deux horodatages.

Elle nous servira ensuite à **découper le temps en fenêtres** au chapitre 4 : une fenêtre de 5 minutes est un intervalle d'horodatages, et un événement y tombe ou non selon le sien.

Elle nous servira enfin à **poser la question de la notion 11** : puisqu'un message peut porter plusieurs horodatages (celui du fait, celui de l'envoi, celui de la réception), il faudra choisir lequel fait foi.

▶ 8. Comment on y est arrivé

Origine de la règle

La nécessité d'un temps commun n'est pas née de l'informatique mais du chemin de fer : tant que chaque ville vivait à son heure solaire, il était impossible d'établir un horaire cohérent, et des collisions ont eu lieu. Les fuseaux horaires sont la première réponse à ce problème, et UTC en est l'héritier moderne.

Le format `AAAA-MM-JJTHH:MM:SSZ` a été normalisé pour résoudre une ambiguïté bien concrète : « 03/04/2026 » désigne le 3 avril en France et le 4 mars aux États-Unis. En imposant l'ordre du plus significatif au moins significatif, on supprime l'ambiguïté et on obtient en prime la coïncidence entre tri alphabétique et tri chronologique.

Quant à l'imperfection des horloges, elle découle de la physique : un oscillateur à quartz dérive avec la température et l'âge, de quelques parties par million — soit environ une seconde par jour pour un quartz ordinaire. On corrige cette dérive en interrogeant périodiquement des serveurs de temps, mais la correction elle-même emprunte le réseau, dont la notion 7 nous a appris qu'il introduit un délai variable. L'incertitude résiduelle est donc irréductible : c'est une conséquence logique, pas un défaut de mise en œuvre.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Mesure de performance.** Latence, durée de traitement, temps d'attente en file : toutes ces métriques sont des différences d'horodatages.
- **Fenêtrage et agrégation temporelle.** « Par tranche de 5 minutes » suppose de savoir dans quelle tranche tombe chaque événement.
- **Audit et conformité.** Un journal d'opérations bancaires doit pouvoir établir l'ordre exact des faits ; l'horodatage en UTC est ce qui rend l'audit possible.
- **Corrélation d'incidents.** Rapprocher les journaux de cinq services pour reconstituer une panne suppose que leurs horloges soient synchronisées ; sinon la cause paraît postérieure à l'effet.
- **Déduplication.** Un même événement reçu deux fois porte le même horodatage d'événement, ce qui aide à le reconnaître.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Dès qu'un énoncé donne deux instants.** La question appelle presque toujours une soustraction ; vérifiez d'abord que les deux sont dans la même unité et la même référence.
- **Dès qu'un énoncé mélange les unités.** « 2 s, 350 ms, 1200 µs » : il faut tout ramener à une unité commune avant toute comparaison. L'erreur d'unité est la faute la plus fréquente en examen.
- **Dès que deux machines sont impliquées.** Posez-vous la question de la synchronisation avant de conclure sur un ordre.
- **Dès qu'une durée négative apparaît.** Une latence négative signale soit une horloge décalée, soit deux horodatages pris sur des machines différentes, soit un recul d'horloge.

▶ 11. Comment on va l'utiliser

Méthode · calculer avec des horodatages

1. **Vérifier la référence.** Les deux horodatages sont-ils en UTC ? Produits par la même machine ? Si non, on signale l'incertitude avant de calculer.
2. **Ramener à une unité commune.** On choisit la milliseconde comme unité de travail par défaut : `1 s = 1000 ms`, `1 ms = 1000 µs`, `1 h = 3 600 000 ms`.
3. **Soustraire du plus récent le plus ancien.** $L = t_r - t_e$. Un résultat négatif est un signal d'alerte, pas une valeur.
4. **Exprimer le résultat dans l'unité qui parle.** On dit « 143 ms » et non « 0,143 s » quand on travaille à l'échelle du paiement ; l'unité choisie doit rendre le nombre lisible.
5. **Comparer au budget.** On confronte à D, conformément à la notion 1, et on conclut.

À retenir

On stocke toujours en UTC, au format année–mois–jour, et l'on garde à l'esprit qu'une horloge n'est jamais exacte : comparer deux horodatages venus de deux machines demande de la prudence.

▶ 12. Exercices d'application corrigés

Application 10.1

Un événement porte `evenement_le = 2026-10-15T08:30:12.250Z`. Il est traité à `2026-10-15T08:30:12.418Z`. Calculez la latence en millisecondes, puis en microsecondes. Le délai utile est de 0,2 seconde : la règle est-elle respectée ?

Corrigé 10.1

Étape 1 — vérifier la référence. Les deux horodatages portent le `Z` : ils sont en UTC, donc directement comparables. On suppose ici qu'ils proviennent de la même machine ou de machines synchronisées.

Étape 2 — unité commune. Les deux instants partagent la même date, la même heure, la même minute et la même seconde ; seule la partie décimale diffère. On travaille donc directement en millisecondes : 250 ms et 418 ms.

Étape 3 — soustraire. `L = 418 − 250 = 168 ms`.

Conversion en microsecondes. `168 ms × 1000 = 168 000 µs`.

Étape 5 — comparer. Le délai utile vaut `D = 0,2 s = 200 ms`. Or `168 ms ≤ 200 ms` : la règle est respectée, avec une marge de 32 ms, soit 16 % du budget.

Commentaire. La marge est faible. Conformément à l'avertissement de la notion 1, il faudrait vérifier le p99 et non ce seul cas : 16 % de marge sur une mesure isolée ne dit rien du comportement sous charge.

Application 10.2

Un ingénieur calcule la latence d'un message en soustrayant l'horodatage produit par le terminal (machine A) de celui produit par le serveur (machine B). Il obtient `−12 ms`. Expliquez ce résultat et dites comment mesurer correctement.

Corrigé 10.2

Lecture du résultat. Une latence négative signifierait que le message a été reçu avant d'avoir été émis, ce qui est impossible. Le nombre n'est donc pas une latence : c'est un symptôme.

Diagnostic. Les deux horodatages viennent de deux horloges différentes. Si l'horloge du serveur retarde de 20 ms sur celle du terminal, une latence réelle de 8 ms sera mesurée comme `8 - 20 = -12 ms`. La mesure n'a capturé que l'écart entre les horloges, pas la durée du trajet.

Pourquoi c'est inévitable. Comme l'explique la section 8, chaque quartz dérive, et la correction par serveur de temps passe elle-même par le réseau, dont le délai est variable. Une incertitude de quelques millisecondes subsiste toujours entre deux machines.

Mesure correcte — première méthode. Mesurer un aller-retour sur une seule horloge : le terminal note t_1 au départ, reçoit la réponse et note t_2. La durée $t_2 - t_1$ est fiable, car les deux lectures viennent de la même horloge. On estime ensuite l'aller simple à environ la moitié, si le chemin est symétrique.

Seconde méthode. Synchroniser les horloges par un service de temps et accepter une incertitude résiduelle : on ne publie alors la mesure qu'accompagnée de sa marge d'erreur, et on s'interdit de conclure sur des écarts inférieurs à cette marge.

Conclusion. Règle pratique : *une durée ne se mesure qu'avec une seule horloge*. Dès que deux horloges interviennent, on mesure un écart, pas une durée.

▶ 13. Exercices d'entraînement

Entraînement

1. Convertissez : 2,5 s en ms ; 1750 ms en s ; 3 ms en µs ; 1 h en ms.
2. Expliquez pourquoi le format `2026-10-15` permet de trier des dates comme du texte, alors que `15/10/2026` ne le permet pas.
3. Un journal contient `"t": 1792182612`. Indiquez les deux interprétations possibles et dites comment trancher.

▶ 14. Exercices de difficulté croissante

Progression

1. facile Combien de millisecondes dans une seconde ? Combien de microsecondes dans une milliseconde ?
2. moyen Un capteur mesure à `23:59:59.800Z` le 15 octobre et le résultat est produit à `00:00:00.150Z` le 16 octobre. Calculez la latence et expliquez le piège.
3. difficile Trois services ont des horloges décalées de `+15 ms`, `−8 ms` et `+2 ms` par rapport à la référence. On cherche à reconstituer l'ordre de trois événements dont les horodatages locaux sont respectivement 100 ms, 104 ms et 110 ms. L'ordre apparent est-il fiable ? Quelle conclusion peut-on tirer et quelle marge d'incertitude faut-il annoncer ?

N11 — Heure d'événement et heure de traitement

▶ 4. La règle

Règle · Les deux heures d'un message

L'heure de l'événement est le moment où le fait s'est réellement produit. L'heure de traitement est le moment où l'ordinateur reçoit et traite le message. Les deux peuvent être très différentes si le message a été retardé.

On note t_e l'heure d'événement et t_p l'heure de traitement. On a toujours $t_p \geq t_e$, et l'écart $t_p - t_e$ est le retard du message. Ce retard est variable d'un message à l'autre : c'est ce qui produit le désordre.

Conditions d'emploi. Pour pouvoir raisonner en heure d'événement, il faut que celle-ci soit *transportée dans le message*. Si le producteur ne l'a pas écrite, elle est définitivement perdue et seul t_p reste disponible.

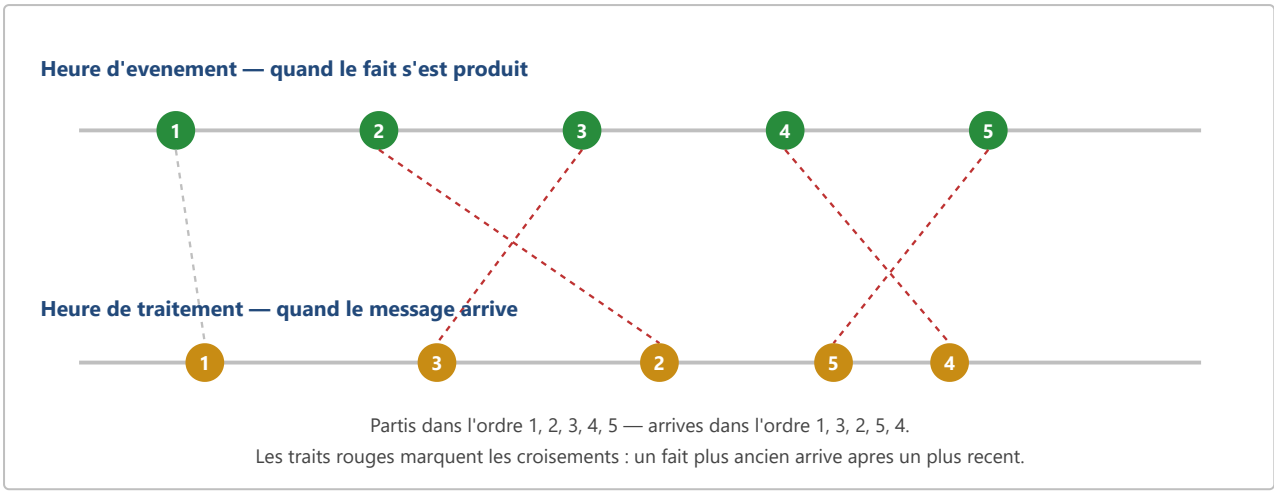


FIGURE 6 · LES DEUX HEURES D'UN MESSAGE

En haut, les faits se produisent dans l'ordre ; en bas, ils arrivent avec des retards inégaux. Les traits rouges marquent les croisements : un événement plus ancien peut arriver après un plus récent. Un calcul fait en heure de traitement les rangerait dans le mauvais ordre.

▶ 5. Explication de la règle

L'image de la carte postale, donnée en cours, est la plus claire : vous écrivez une carte le 1er août, en vacances ; votre famille la reçoit le 10 août. Si elle range ses cartes par date de réception, elle croira que vous avez écrit le 10. L'heure d'événement est le 1er août, l'heure de traitement le 10 août, et le retard vaut neuf jours.

L'idée profonde est qu'il existe **deux histoires parallèles** : celle du monde réel, dans laquelle les faits se produisent dans un certain ordre, et celle du système informatique, dans laquelle les messages arrivent dans un autre ordre. Ces deux histoires ne coïncident que si le réseau est parfait — ce que la notion 7 nous a appris à ne jamais supposer.

La conséquence pratique est qu'il faut **choisir explicitement laquelle des deux histoires on raconte**. Si l'on compte « les ventes de 8 h à 9 h » en heure de traitement, on compte en réalité les messages reçus pendant cette heure, dont certains décrivent des ventes de 7 h 50. Le résultat est reproductible mais faux vis-à-vis du métier. Si l'on compte en heure d'événement, le résultat est juste, mais il faut *attendre* les retardataires, donc accepter de la latence — un arbitrage direct avec la notion 1.

Cas particuliers et pièges

- **Le rejeu d'historique.** Si l'on rejoue trois ans d'archives en une heure, l'heure de traitement est aujourd'hui pour tous les messages, alors que les heures d'événement s'étalent sur trois ans. Un calcul en heure de traitement donnerait un résultat absurde.
- **Le mode hors ligne.** Une application mobile sans réseau accumule les événements et les envoie tous à la reconnexion : retards de plusieurs heures, parfaitement normaux.
- **L'attente ne peut pas être infinie.** On ne peut pas attendre indéfiniment les retardataires ; il faut fixer une limite, et les messages arrivant après sont dits « tardifs ». Le mécanisme s'appelle *watermark* et fait l'objet du chapitre 4.
- **Trois horodatages, pas deux.** En pratique on distingue souvent l'heure du fait, l'heure d'ingestion dans la file, et l'heure de traitement. Ce chapitre se limite aux deux extrêmes.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui de la **justesse des calculs temporels en présence de retards**. Dès qu'un résultat est découpé par tranches de temps — ventes par heure, alertes par minute, moyenne glissante — il faut décider à quelle tranche appartient chaque événement. Sans cette règle, la décision se prend par défaut, c'est-à-dire en heure de traitement, et les résultats deviennent silencieusement faux.

Le problème est aggravé par le fait que l'erreur est **invisible** : un total par heure calculé en heure de traitement a l'air parfaitement normal. Rien ne signale qu'il attribue à 9 h des ventes faites à 8 h 55.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous servira à **concevoir correctement les événements** : puisque l'heure d'événement doit être transportée, elle devient un champ obligatoire de tout schéma, comme nous l'avons fait à la notion 8.

Elle nous servira ensuite à **lire les résultats avec prudence**. Devant un tableau de bord, la première question à poser est : ces tranches horaires sont-elles en heure d'événement ou en heure de traitement ?

Elle prépare enfin tout le **chapitre 4** : fenêtres, watermarks, événements tardifs n'ont de sens que si cette distinction est acquise.

▶ 8. Comment on y est arrivé

Origine de la règle

Les premiers systèmes de traitement de flux raisonnaient exclusivement en heure de traitement, parce que c'est l'information la plus simple à obtenir : il suffit de lire l'horloge locale au moment où le message arrive. Tant que les sources étaient proches et le réseau peu chargé, l'écart restait négligeable.

Deux évolutions ont rendu ce choix intenable. La mobilité a généralisé les sources déconnectées par intermittence, dont les événements arrivent par paquets après plusieurs heures. Et le besoin de rejouer les historiques — pour corriger un bug ou initialiser un nouveau calcul — a rendu absurde tout raisonnement fondé sur l'instant de réception.

Le raisonnement qui fonde la règle est le suivant. Un résultat métier doit être *reproductible* : rejouer les mêmes événements doit donner le même résultat. Or t_p dépend du moment où l'on exécute le calcul, donc change à chaque rejeu, tandis que t_e est une propriété du fait lui-même et ne change jamais. Seul un calcul fondé sur t_e peut être reproductible ; c'est ce qui impose de transporter l'heure d'événement dans le message.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Facturation et comptabilité.** Une consommation du 31 octobre reçue le 1er novembre appartient à octobre ; la facture se calcule en heure d'événement, sans discussion possible.
- **Applications mobiles.** Mode hors ligne, tunnels, zones blanches : les retards de plusieurs heures y sont la norme.
- **Internet des objets.** Un capteur sur batterie peut grouper ses mesures et les envoyer une fois par heure pour économiser l'énergie ; chaque mesure porte alors son propre t_e.
- **Reprise après incident.** Après une panne de deux heures, tous les messages accumulés arrivent d'un coup ; seul le raisonnement en heure d'événement évite un pic artificiel dans les statistiques.
- **Supervision technique.** À l'inverse, pour mesurer la santé du système lui-même (débit reçu, saturation), l'heure de traitement est la bonne référence : c'est elle qui décrit la charge réelle de la machine.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Choisissez l'heure d'événement dès que le résultat décrit le monde réel.** Ventes, consommations, mesures physiques, parcours utilisateur : le métier parle du fait, pas de sa réception.
- **Choisissez l'heure de traitement quand le résultat décrit le système.** « Combien de messages avons-nous reçus cette minute ? » porte bien sur la réception.
- **Méfiez-vous quand un total par heure change après coup.** C'est le signe que des retardataires sont arrivés : comportement normal en heure d'événement, à expliquer aux utilisateurs.
- **Méfiez-vous d'un pic après une panne.** Si les statistiques montrent un pic au moment de la reprise, le calcul est en heure de traitement : le pic est un artefact.

▶ 11. Comment on va l'utiliser

Méthode · choisir et appliquer la bonne heure

1. **Formuler la question du métier.** « Combien de ventes ont eu lieu entre 8 h et 9 h ? » parle du fait : heure d'événement. « Combien de messages avons-nous traités ? » parle du système : heure de traitement.
2. **Vérifier que t_e est disponible.** Si le message ne la transporte pas, il faut modifier le producteur ; aucun traitement en aval ne peut l'inventer.
3. **Mesurer le retard observé.** On calcule $t_p - t_e$ sur un échantillon et on regarde sa distribution, en particulier les valeurs extrêmes.
4. **Fixer une limite d'attente.** On choisit un délai couvrant la grande majorité des retards, par exemple le p99 ; au-delà, les messages seront traités comme tardifs.
5. **Décider du sort des tardifs.** Trois options : les ignorer, les compter dans une tranche spéciale, ou corriger rétroactivement le résultat déjà publié. Le choix appartient au métier, pas à la technique.

À retenir

L'heure d'événement appartient au fait et ne change jamais ; l'heure de traitement appartient au système et change à chaque rejeu. Seule la première permet un résultat reproductible.

▶ 12. Exercices d'application corrigés

Application 11.1

Quatre ventes ont lieu à `08:58`, `08:59`, `09:01` et `09:02`. À cause d'une coupure réseau, les deux premières n'arrivent qu'à `09:05` ; les deux autres arrivent immédiatement. On veut le total des ventes de la tranche 8 h–9 h. Donnez le résultat en heure de traitement puis en heure d'événement, et commentez.

Corrigé 11.1

Étape 1 — formuler la question. « Les ventes de la tranche 8 h–9 h » parle de faits survenus dans le monde réel : c'est une question en heure d'événement.

Calcul en heure de traitement. La tranche 8 h–9 h retient les messages *reçus* entre 8 h et 9 h. Les deux premières ventes sont reçues à 09:05, donc hors tranche ; les deux autres sont reçues après 9 h, donc hors tranche aussi. Résultat : **0 vente** entre 8 h et 9 h, et 4 ventes entre 9 h et 10 h.

Calcul en heure d'événement. On regarde t_e : 08:58 et 08:59 tombent dans la tranche ; 09:01 et 09:02 n'y tombent pas. Résultat : **2 ventes** entre 8 h et 9 h, et 2 ventes entre 9 h et 10 h.

Commentaire. Le calcul en heure de traitement se trompe deux fois : il vide une tranche qui contenait deux ventes et en surcharge une autre. Surtout, l'erreur est invisible ; le tableau de bord affiche simplement « 0 », ce qu'un responsable pourrait interpréter comme une panne de caisse.

Étape 4 — conséquence opérationnelle. Pour que le bon résultat soit publiable, il faut attendre au moins jusqu'à 09:05 avant de clore la tranche 8 h–9 h, soit 5 minutes de latence ajoutée. C'est l'arbitrage annoncé par la notion 1 : la justesse se paie en latence.

Application 11.2

Après correction d'un bug, une équipe rejoue 6 mois d'événements archivés en 3 heures. Le tableau de bord « ventes par jour » affiche alors trois barres énormes et rien ailleurs. Expliquez et corrigez.

Corrigé 11.2

Diagnostic. Le calcul est fait en heure de traitement. Pendant le rejeu, t_p vaut « aujourd'hui » pour les six mois d'événements : tous sont donc attribués aux trois heures du rejeu. D'où trois barres contenant 180 jours de ventes, et aucune ailleurs.

Pourquoi c'était prévisible. C'est exactement le raisonnement de la section 8 : t_p dépend du moment de l'exécution, donc un calcul fondé sur elle n'est pas reproductible. Le rejeu est précisément l'opération qui révèle cette non-reproductibilité.

Correction. Recalculer en heure d'événement : chaque vente porte son `evenement_le`, le regroupement par jour utilise ce champ, et le graphique retrouve sa forme d'origine quel que soit le moment du rejeu. On vérifie au passage que le champ est bien présent dans les archives ; s'il ne l'est pas, les données sont irrécupérables en l'état et il faudra retourner à la source.

Précaution supplémentaire. Pendant le rejeu, la limite d'attente fixée à l'étape 4 de la méthode doit être désactivée ou élargie : sinon la quasi-totalité des événements rejoués serait classée « tardive », puisqu'ils ont six mois de retard apparent.

Leçon générale. Un système qu'on ne peut pas rejouer est un système qu'on ne peut pas corriger. Raisonner en heure d'événement n'est donc pas un raffinement : c'est ce qui rend les corrections possibles.

▶ 13. Exercices d'entraînement

Entraînement

1. Une carte postale est écrite le 1er et reçue le 10. Donnez l'heure d'événement, l'heure de traitement et le retard.
2. Pour chaque indicateur, dites quelle heure utiliser : (a) consommation électrique mensuelle facturée ; (b) nombre de messages reçus par seconde par le cluster ; (c) nombre de trajets effectués hier.
3. Pourquoi est-il impossible de retrouver l'heure d'événement si le producteur ne l'a pas écrite ?

▶ 14. Exercices de difficulté croissante

Progression

1. facile Un événement a t_e = `10:00:00` et t_p = `10:00:03`. Quel est le retard ?
2. moyen Sur 10 000 messages, le retard a pour médiane 120 ms, pour p99 4 s et pour maximum 3 h. Proposez une limite d'attente et justifiez-la ; dites ce qu'il advient du maximum.
3. difficile Un tableau de bord publie le total horaire à la minute près. Des retardataires arrivent jusqu'à 20 minutes après la fin d'une tranche. Proposez trois stratégies de publication, en indiquant pour chacune la latence, la justesse et ce que l'utilisateur perçoit.

N12 — Le système distribué et le cluster

▶ 4. La règle

Règle · Système distribué

On utilise plusieurs machines pour trois raisons : parce qu'il y a trop de travail pour une seule, pour ne pas tout perdre en cas de panne, et pour être plus près des utilisateurs. Un ensemble de machines qui travaillent ensemble s'appelle un cluster ; on parle de système distribué.

En contrepartie, quatre difficultés apparaissent, qui n'existaient pas sur une seule machine : les machines doivent se coordonner (qui fait quoi ?), l'une d'elles peut tomber en panne au milieu d'un travail, les messages entre elles peuvent être perdus ou dupliqués, et leurs horloges ne sont pas exactement synchronisées.

Conditions d'emploi. La distribution n'apporte de gain que si le travail est *décomposable* : s'il peut être découpé en parts largement indépendantes. Un travail purement séquentiel ne gagne rien à être distribué, et perd en fiabilité.

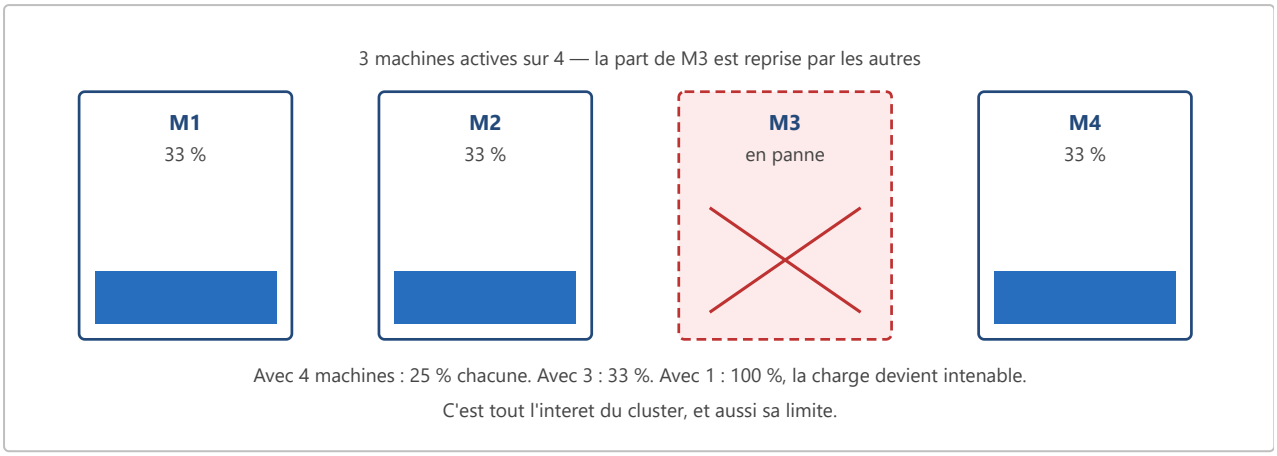


FIGURE 7 · PARTAGER LE TRAVAIL, SURVIVRE AUX PANNES
Chaque machine prend une part de la charge : 25 % chacune à quatre, 33 % à trois lorsque M3 tombe. C'est le gain du cluster, et aussi sa limite : à une seule machine restante, la charge devient intenable.

▶ 5. Explication de la règle

Les trois raisons de la règle répondent à trois contraintes de nature différente, et il est utile de les distinguer. La première est une contrainte de **capacité** : des millions de messages par seconde dépassent ce qu'une machine traite, et l'on partage le travail comme on ouvre plusieurs caisses dans un supermarché. La deuxième est une contrainte de **disponibilité** : une machine tombe tôt ou tard, et l'on veut qu'une autre prenne le relais à partir de copies des données. La troisième est une contrainte **physique** : la vitesse de la lumière impose une latence incompressible, et la seule parade est de rapprocher les machines des utilisateurs — Casablanca, Paris, Dakar.

L'image des déménageurs, donnée en cours, dit bien ce que coûte ce gain : à dix, on va plus vite qu'à un seul, mais il faut se répartir les cartons, éviter de porter deux fois le même, et savoir quoi faire si quelqu'un se blesse. Plus l'équipe est grande, plus l'organisation compte. C'est exactement la situation d'un cluster : la coordination devient elle-même un travail, et ce travail grandit plus vite que l'équipe.

Les quatre difficultés ne sont pas quatre problèmes indépendants : ce sont **quatre conséquences d'un même fait**, à savoir que les machines ne partagent ni mémoire ni horloge et ne communiquent que par un réseau imparfait. La coordination est difficile parce qu'aucune machine ne connaît l'état global ; la panne est difficile à détecter parce qu'une machine silencieuse est indiscernable d'une machine lente ; les pertes et doublons viennent de la notion 7 ; la désynchronisation des horloges vient de la notion 10.

Cas particuliers et pièges

- **Deux machines ne sont pas deux fois plus rapides.** Une part du travail reste séquentielle, et la coordination consomme des ressources : le gain réel est toujours inférieur au gain théorique, et peut même devenir négatif.
- **Panne et lenteur sont indiscernables.** Une machine qui ne répond pas est peut-être tombée, peut-être juste surchargée. Déclarer une panne à tort fait travailler deux machines sur la même tâche.
- **Distribuer un petit problème l'aggrave.** Pour un volume que gagnerait à traiter une seule machine, le cluster ajoute de la complexité et des modes de panne sans rien apporter.
- **La panne devient la norme.** Sur mille machines, il y a toujours quelque chose en panne. Un système distribué se conçoit en supposant la panne permanente, pas exceptionnelle.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui des **limites d'une machine unique**, sous trois formes. Limite de débit : au-delà d'un certain nombre de messages par seconde, aucune machine ne suit, et acheter plus gros finit par devenir impossible ou hors de prix. Limite de disponibilité : une machine seule est un point de défaillance unique, et sa panne arrête tout. Limite géographique : une machine est en un seul endroit, donc loin de la plupart des utilisateurs.

La règle énonce la solution — plusieurs machines — et, ce qui est plus important, le prix à payer. Un cours qui ne présenterait que les trois raisons sans les quatre difficultés donnerait une image fausse : c'est l'ensemble qui constitue la règle.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous permet de **comprendre pourquoi Kafka, Flink et Spark existent**. Ces logiciels ne font pas autre chose que résoudre, une fois pour toutes et de façon éprouvée, les quatre difficultés de la règle. Sans eux, chaque projet devrait les résoudre lui-même, et les résoudrait mal.

Elle nous donne ensuite le **vocabulaire des chapitres 3 et 5** : partitionnement, réplication, points de reprise, garanties de livraison sont tous des réponses à l'une des quatre difficultés.

Elle nous apprend enfin à **ne pas distribuer sans raison**. Devant un besoin modeste, la bonne réponse reste souvent une seule machine bien dimensionnée.

▶ 8. Comment on y est arrivé

Origine de la règle

Pendant longtemps, la réponse à un besoin croissant fut d'acheter une machine plus puissante. Cette stratégie, dite verticale, a deux limites. Le prix croît plus vite que la puissance, et la puissance d'un processeur a cessé de doubler régulièrement : on n'augmente plus la fréquence, on ajoute des cœurs. Au-delà d'un certain point, la seule option restante est d'ajouter des machines.

Pour la disponibilité, le raisonnement est probabiliste et très parlant. Si une machine a une probabilité p d'être en panne à un instant donné, deux machines indépendantes capables de se remplacer sont toutes deux en panne avec une probabilité p^2. Avec $p = 1\ \%$, on passe de 1 % à 0,01 % d'indisponibilité. L'indépendance est ici l'hypothèse clé : deux machines dans la même salle, sur la même alimentation, ne sont pas indépendantes, et le calcul ne tient plus.

Quant aux quatre difficultés, elles ont été formalisées par la recherche en systèmes distribués à partir d'un constat : en l'absence de mémoire partagée et d'horloge commune, certaines choses que l'on croit évidentes deviennent impossibles — par exemple savoir avec certitude si une machine est tombée, ou obtenir un accord entre toutes les machines dans un réseau qui peut retarder les messages. Ces résultats négatifs expliquent pourquoi il n'existe pas de solution simple, et pourquoi l'on s'appuie sur des logiciels spécialisés.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Files de messages.** Un cluster Kafka répartit les partitions sur plusieurs serveurs et réplique chacune : capacité et disponibilité en même temps.
- **Moteurs de calcul en flux.** Flink et Spark distribuent les opérateurs sur plusieurs machines ; c'est la clé de la notion 9 qui détermine quelle machine traite quel sujet.
- **Bases de données répliquées.** Une copie secondaire prend le relais en cas de panne de la principale.
- **Diffusion de contenu.** Des serveurs répartis géographiquement servent les utilisateurs depuis le point le plus proche : c'est la troisième raison, appliquée à grande échelle.
- **Calcul scientifique et apprentissage.** L'entraînement de grands modèles est réparti sur des centaines de machines ; la coordination y est le principal coût.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Quand le débit dépasse la capacité d'une machine.** Le signal est une file d'attente qui grandit sans jamais se vider : la machine reçoit plus qu'elle ne traite.
- **Quand l'interruption est inacceptable.** Si l'énoncé impose une disponibilité élevée, une seule machine ne suffit pas, quel que soit son dimensionnement.
- **Quand les utilisateurs sont dispersés.** Si une partie d'entre eux subit 150 ms de latence réseau incompressible, la seule parade est de rapprocher le service.
- **Méfiance quand le travail n'est pas décomposable.** Si chaque étape dépend du résultat de la précédente, distribuer n'apporte rien : on ajoute de la coordination sans gagner de parallélisme.

▶ 11. Comment on va l'utiliser

Méthode · justifier et dimensionner un cluster

1. **Nommer la raison principale.** Capacité, disponibilité ou proximité ? Les trois ne conduisent pas aux mêmes choix : la capacité demande du partitionnement, la disponibilité demande de la réplication, la proximité demande une répartition géographique.
2. **Vérifier que le travail est décomposable.** On cherche la clé de la notion 9 : s'il en existe une, le travail se partage proprement.
3. **Calculer le nombre de machines.** On divise le débit visé par le débit d'une machine, puis on ajoute une marge et les copies nécessaires à la disponibilité.
4. **Prévoir le comportement en cas de panne.** Si une machine tombe, les autres absorbent-elles sa part ? On vérifie que le système reste sous sa capacité maximale avec une machine de moins.
5. **Traiter les quatre difficultés une par une.** Coordination, détection de panne, pertes et doublons, horloges : pour chacune, on nomme le mécanisme retenu, en s'appuyant sur ce que fournissent Kafka ou Flink plutôt qu'en l'écrivant soi-même.

À retenir

Plusieurs machines apportent capacité, résistance aux pannes et proximité ; elles coûtent en coordination, en incertitude sur les pannes, en pertes et doublons, et en décalage d'horloges. Kafka, Flink, Spark et Storm sont précisément des logiciels qui résolvent ces problèmes pour nous.

▶ 12. Exercices d'application corrigés

Application 12.1

Un système doit traiter 30 000 messages par seconde. Une machine en traite 9 000. L'exploitant exige que le service continue même si une machine tombe. Combien de machines prévoyez-vous ? Détaillez.

Corrigé 12.1

Étape 1 — raison principale. Deux raisons se cumulent : la capacité (30 000 > 9 000) et la disponibilité (tolérance à une panne). Il faudra donc du partitionnement *et* une marge.

Étape 2 — décomposabilité. On suppose qu'une clé existe et que les messages se répartissent équitablement ; sinon tout le calcul qui suit est invalide, comme l'a montré la notion 9.

Étape 3 — nombre de machines pour la capacité. `30 000 / 9 000 = 3,33`, donc **4 machines** au minimum, qui offrent `4 × 9 000 = 36 000` messages par seconde.

Étape 4 — comportement en panne. Avec 4 machines dont une tombe, il reste `3 × 9 000 = 27 000 < 30 000` : insuffisant, la file se remplirait. Il faut donc **5 machines** : en fonctionnement normal `45 000`, et après une panne `4 × 9 000 = 36 000 ≥ 30 000`. Le besoin est couvert.

Vérification de la charge par machine. En régime normal, chaque machine traite `30 000/5 = 6 000` messages par seconde, soit 67 % de sa capacité ; après une panne, `7 500`, soit 83 %. Cela reste acceptable, même si une marge plus confortable justifierait une sixième machine.

Étape 5 — les quatre difficultés. Coordination : assurée par le groupe de consommateurs Kafka, qui attribue les partitions. Détection de panne : par battements de cœur, avec un délai au-delà duquel la machine est déclarée absente. Pertes et doublons : par réplication des partitions et traitement idempotent. Horloges : en raisonnant en heure d'événement, conformément à la notion 11.

Remarque importante. Il faut au moins 5 partitions pour que 5 machines travaillent : le nombre de casiers borne le parallélisme. Avec 4 partitions, la cinquième machine resterait inactive.

Application 12.2

Une équipe constate que son cluster de 12 machines traite à peine plus de messages qu'un cluster de 6. Donnez trois explications possibles, classées de la plus probable à la moins probable, avec pour chacune la vérification à mener.

Corrigé 12.2

Explication 1 — le parallélisme est borné par le nombre de casiers. Si le flux ne compte que 6 partitions, six machines suffisent à les consommer et les six autres restent inactives, quel que soit leur nombre. *Vérification :* comparer le nombre de partitions au nombre de consommateurs, et observer si six machines affichent une charge nulle. C'est de loin la cause la plus fréquente.

Explication 2 — une clé chaude déséquilibre la charge. Si une clé concentre l'essentiel du trafic, la machine qui la traite sature pendant que les autres attendent ; ajouter des machines ne déplace pas ce goulot. *Vérification :* relever le débit par partition. Une partition très au-dessus des autres confirme le diagnostic, et la correction relève de la notion 9.

Explication 3 — le goulot est ailleurs que dans le calcul. Si toutes les machines écrivent dans la même base ou appellent le même service externe, c'est cette ressource partagée qui plafonne ; la distribution du calcul ne sert alors à rien. *Vérification :* mesurer le temps passé en attente de la ressource commune ; s'il domine, le problème n'est pas le cluster.

Leçon générale. Ajouter des machines n'augmente le débit que si le travail est effectivement décomposable *et* effectivement décomposé. C'est l'étape 2 de la méthode, celle qu'on saute le plus souvent.

▶ 13. Exercices d'entraînement

Entraînement

1. Citez les trois raisons d'utiliser plusieurs machines et donnez pour chacune un exemple concret différent de ceux du cours.
2. Expliquez pourquoi une machine silencieuse est indiscernable d'une machine lente, et quelle conséquence cela a sur la détection de panne.
3. Avec `p = 2 %` de probabilité de panne par machine, calculez la probabilité que deux machines indépendantes soient en panne en même temps. Quelle hypothèse rend ce calcul fragile ?

▶ 14. Exercices de difficulté croissante

Progression

1. facile Comment appelle-t-on un ensemble de machines qui travaillent ensemble ?
2. moyen Un service doit tenir 50 000 messages/s ; une machine en traite 12 000 et l'on veut survivre à deux pannes simultanées. Combien de machines, et quelle charge par machine en régime normal ?
3. difficile Expliquez pourquoi passer de 1 à 2 machines améliore beaucoup la disponibilité alors que passer de 20 à 21 ne l'améliore presque pas, puis discutez ce que devient ce raisonnement si les machines partagent la même alimentation électrique.

N13 — Le conteneur Docker

▶ 4. La règle

Règle · Le conteneur

Un conteneur Docker est une petite boîte qui contient un logiciel et tout ce dont il a besoin pour fonctionner. On le lance avec une commande, et il fonctionne de la même façon sur tous les ordinateurs.

La boîte est décrite par une *image*, qui est un modèle figé et versionné ; un conteneur est une exécution de cette image. Deux personnes qui lancent la même image obtiennent le même environnement, quelles que soient leurs machines.

Conditions d'emploi. Le conteneur partage le noyau du système hôte : il isole les fichiers, les processus et le réseau, mais ce n'est pas une machine virtuelle complète. Et tout ce qu'il écrit à l'intérieur disparaît à sa suppression, sauf si l'on monte un volume.

▶ 5. Explication de la règle

L'image donnée en cours est celle de la **boîte repas** : au lieu d'acheter tous les ingrédients et de cuisiner, on reçoit un plat déjà préparé ; il suffit de le réchauffer, et le résultat est le même chez tout le monde. Transposé : au lieu d'installer Java, puis Kafka, puis de configurer dix fichiers en espérant que la version de votre système soit compatible, vous lancez une image qui contient déjà tout cela, assemblé et testé.

L'idée profonde est celle de la **reproductibilité de l'environnement**. Un programme ne dépend pas seulement de son code : il dépend d'une version de langage, de bibliothèques, de variables d'environnement, de fichiers de configuration. Cet ensemble, invisible dans le dépôt de code, est la cause du célèbre « ça marche sur ma machine ». Le conteneur rend cet ensemble explicite et transportable.

Pour ce module, l'intérêt est très concret : nous devons faire tourner ensemble Kafka, une interface d'administration, Flink et Spark. Installer tout cela à la main sur trente ordinateurs différents prendrait plusieurs séances et produirait trente environnements différents. Avec Docker, une seule commande donne à chacun le même point de départ — et un TP qui échoue échoue alors pour tout le monde de la même façon, ce qui est précisément ce qui permet de le déboguer.

Cas particuliers et pièges

- **Conteneur n'est pas machine virtuelle.** Une machine virtuelle embarque un système d'exploitation complet ; un conteneur partage celui de l'hôte. Il démarre en quelques secondes au lieu de quelques minutes, mais isole moins fortement.
- **Les données disparaissent.** Supprimer un conteneur supprime ce qu'il contenait. Pour garder les données d'un TP, il faut monter un volume ; c'est l'oubli le plus fréquent.
- **`localhost` dans un conteneur désigne le conteneur.** Déjà signalé à la notion 7, et c'est la première cause d'échec de connexion en TP : entre conteneurs, on s'adresse par le nom du service.
- **Les ressources doivent être allouées.** Kafka, Flink et Spark ensemble demandent au moins 4 processeurs et 8 Go de mémoire ; en dessous, les conteneurs s'arrêtent sans message clair.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui de la **divergence des environnements**. Le même code produit des résultats différents selon la machine, parce que les versions des dépendances diffèrent. En pédagogie, cela transforme une séance de TP en séance de dépannage ; en entreprise, cela produit des pannes qui n'apparaissent qu'en production.

Le second problème est celui du **coût d'installation d'une pile logicielle complète**. Un système temps réel met en jeu cinq ou six logiciels qui doivent être compatibles entre eux ; les assembler manuellement est long et fragile.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous servira à **monter l'environnement de TP** en une commande, avec Kafka en mode KRaft, une interface d'administration, Flink et Spark, dans des versions identiques pour tous.

Elle nous servira aussi à **simuler un cluster sur un seul ordinateur** : plusieurs conteneurs jouent plusieurs machines, ce qui permet d'expérimenter concrètement les notions 9 et 12 sans salle serveur.

Elle nous servira enfin à **provoquer des pannes volontairement** : arrêter un conteneur, c'est faire tomber une machine ; on observe alors en direct le comportement décrit par la figure 7.

▶ 8. Comment on y est arrivé

Origine de la règle

L'isolation de processus existe depuis longtemps dans les systèmes Unix : on savait déjà restreindre un programme à une partie du système de fichiers. Les mécanismes modernes du noyau Linux — espaces de noms et groupes de contrôle — ont généralisé cette isolation aux processus, au réseau et aux ressources.

L'apport de Docker n'est donc pas l'isolation elle-même : c'est le **format d'image partageable** et la simplicité d'usage. En rendant un environnement descriptible dans un fichier, versionnable et téléchargeable, Docker a transformé une capacité technique existante en pratique courante.

Le raisonnement sous-jacent est le même que pour les conteneurs maritimes, dont le nom est emprunté. Avant leur normalisation, chaque marchandise se chargeait à sa façon et le transbordement coûtait cher. En normalisant la *boîte* plutôt que son contenu, on a rendu interchangeables navires, camions et grues. Docker fait exactement cela pour le logiciel : en normalisant l'enveloppe, il rend l'exécution indépendante de la machine.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Environnements de travaux pratiques.** C'est notre usage principal : une pile complète, identique pour tous, démontable sans laisser de traces.
- **Développement local.** Un développeur lance une base, une file et un cache sur son poste sans rien installer durablement.
- **Intégration continue.** Les tests s'exécutent dans un conteneur, donc dans le même environnement que la production.
- **Déploiement en production.** Les orchestrateurs déploient des conteneurs sur des clusters ; la notion 12 et celle-ci se rejoignent ici.
- **Reproductibilité scientifique.** Publier une image avec un article permet à d'autres de rejouer exactement les expériences.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Quand plusieurs logiciels doivent cohabiter.** Dès qu'un TP demande trois services ou plus, l'installation manuelle n'est plus raisonnable.
- **Quand l'environnement doit être identique pour tous.** Signal typique d'un enseignement ou d'une équipe : on veut que l'erreur d'un étudiant soit reproductible chez l'enseignant.
- **Quand on veut pouvoir tout effacer.** Un conteneur supprimé ne laisse rien derrière lui ; c'est précieux sur un poste partagé.
- **Quand ce n'est pas la bonne réponse.** Pour un notebook Python sans service externe, Docker ajoute de la complexité sans bénéfice : Google Colab, mentionné dans les prérequis du module, suffit.

▶ 11. Comment on va l'utiliser

Méthode · monter un environnement conteneurisé

1. **Lister les services nécessaires et leurs versions.** Par exemple Kafka en mode KRaft, une interface d'administration, Flink 1.20, Spark 3.5. On fige les versions : c'est ce qui rend l'environnement reproductible.
2. **Vérifier les ressources de la machine hôte.** Au moins 4 processeurs et 8 Go de mémoire alloués à Docker ; sous Windows, via WSL 2.
3. **Décrire les services dans un fichier de composition.** Un service par logiciel, avec son image, ses ports publiés et ses variables d'environnement.
4. **Définir les adresses entre conteneurs.** Entre services, on utilise le nom du service et non `localhost` ; depuis la machine hôte, on utilise `localhost` et le port publié.
5. **Démarrer dans l'ordre et vérifier.** On applique la notion 6 : les serveurs d'abord. On vérifie ensuite que chaque service répond avant de lancer le code du TP.

À retenir

Une image fige un environnement complet : le même logiciel, les mêmes versions, le même comportement sur toutes les machines. À l'intérieur d'un conteneur, `localhost` désigne le conteneur lui-même.

▶ 12. Exercices d'application corrigés

Application 13.1

Un étudiant lance Kafka dans un conteneur avec le port 9092 publié. Son script `producteur.py`, lancé *depuis sa machine*, se connecte à `localhost:9092` et fonctionne. Il place ensuite ce script dans un conteneur ; la connexion échoue. Expliquez et corrigez.

Corrigé 13.1

Pourquoi cela fonctionnait avant. Lancé depuis la machine hôte, le script s'adressait à `localhost`, c'est-à-dire à la machine de l'étudiant, sur le port 9092 publié par Docker ; la redirection vers le conteneur Kafka était assurée par Docker lui-même.

Pourquoi cela échoue maintenant. À l'intérieur d'un conteneur, `localhost` ne désigne plus la machine de l'étudiant mais *le conteneur du producteur*. Or aucun Kafka n'écoute dans ce conteneur : la connexion est refusée. C'est exactement le piège signalé à la notion 7, où l'on a vu que l'adresse doit être valide *depuis le programme appelant*.

Correction. Utiliser le nom du service Kafka tel qu'il est déclaré dans le fichier de composition, par exemple `kafka:9092`. Docker fournit une résolution de noms entre services du même réseau : le nom du service devient l'adresse.

Vérification. On applique l'étape 4 de la méthode : depuis l'hôte on garde `localhost:9092`, depuis un conteneur on utilise `kafka:9092`. Les deux adresses coexistent et désignent le même serveur vu de deux endroits différents.

Application 13.2

Après avoir arrêté puis supprimé ses conteneurs, un étudiant constate que tous les messages produits pendant le TP précédent ont disparu. Expliquez et proposez une solution.

Corrigé 13.2

Explication. Un conteneur possède son propre système de fichiers, créé à partir de l'image au démarrage et détruit avec lui. Les messages Kafka étaient écrits dans ce système de fichiers interne ; sa suppression les a emportés. Ce n'est pas une anomalie : c'est la condition d'emploi énoncée à la section 4.

Pourquoi ce comportement est voulu. C'est ce qui permet de repartir d'un environnement propre à chaque TP, et c'est précisément l'avantage recherché sur un poste partagé. La persistance doit donc être demandée explicitement.

Solution. Monter un *volume* : un espace de stockage géré par Docker, indépendant du cycle de vie du conteneur, associé au répertoire de données de Kafka. Les messages y sont alors écrits et survivent à la suppression du conteneur ; au redémarrage, Kafka retrouve son journal.

Précision utile. Arrêter un conteneur ne détruit rien : c'est la suppression qui efface. Pour une interruption de séance, arrêter suffit ; le volume n'est nécessaire que si l'on veut survivre à une suppression ou à une reconstruction.

▶ 13. Exercices d'entraînement

Entraînement

1. Expliquez l'image de la boîte repas et dites ce que représentent respectivement l'image et le conteneur.
2. Donnez deux différences entre un conteneur et une machine virtuelle.
3. Un conteneur Flink n'arrive pas à joindre Kafka. Listez dans l'ordre les trois vérifications à faire.

▶ 14. Exercices de difficulté croissante

Progression

1. facile Que contient un conteneur Docker, d'après la règle ?
2. moyen Vous devez fournir à trente étudiants un environnement avec Kafka, Flink et une interface d'administration. Listez les éléments à figer pour que les trente environnements soient identiques, et dites ce qui pourrait encore différer malgré cela.
3. difficile On simule un cluster de 3 courtiers Kafka avec 3 conteneurs sur un seul ordinateur. Expliquez ce que cette simulation reproduit fidèlement des quatre difficultés de la notion 12, et ce qu'elle ne reproduit pas.

N14 — Le générateur Python

▶ 4. La règle

Règle · Générateur

Une fonction Python qui contient `yield` est un générateur : elle donne une valeur, puis reprend là où elle s'était arrêtée quand on lui en redemande une. Associée à une boucle infinie, elle produit des valeurs sans fin : c'est la première image d'un flux.

```
import random, time

def capteur():
    """Produit des temperatures sans fin."""
    while True:                      # boucle infinie
        temperature = random.gauss(25, 2)
        yield round(temperature, 1)    # on « donne » une valeur
        time.sleep(1)                  # on attend 1 seconde

for valeur in capteur():             # on recoit les valeurs une par une
    print("Nouvelle mesure :", valeur)
```

Conditions d'emploi. Un générateur ne se parcourt qu'une fois et dans un seul sens : on ne peut ni revenir en arrière, ni connaître à l'avance le nombre de valeurs, ni demander la dernière.

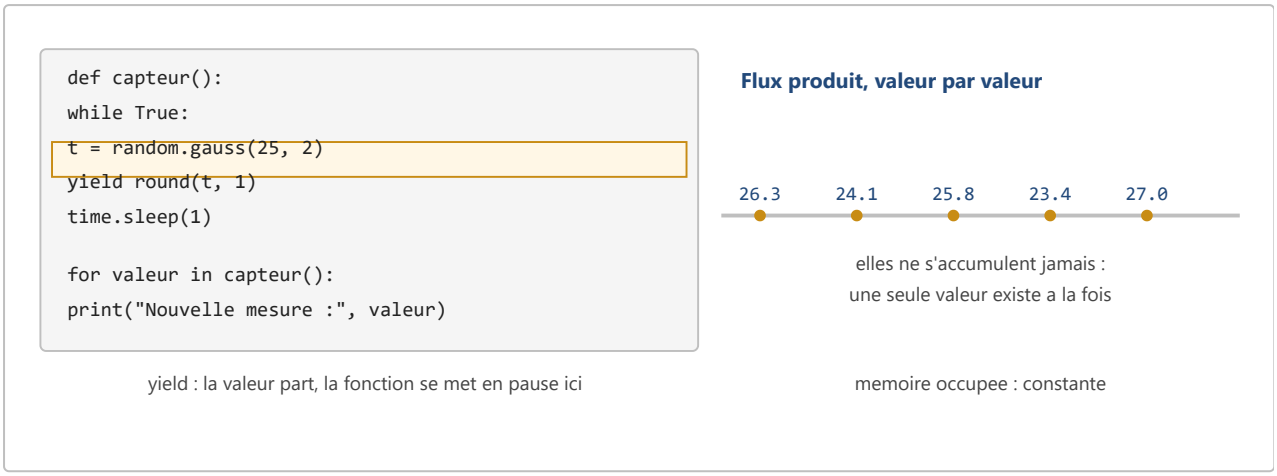


FIGURE 8 · LE GÉNÉRATEUR, PREMIÈRE IMAGE D'UN FLUX

La ligne surlignée montre où en est la fonction. À chaque yield elle rend une valeur et se met en pause ; elle reprend exactement là quand la boucle for en redemande une. Les valeurs ne s'accumulent jamais : la mémoire occupée reste constante.

▶ 5. Explication de la règle

Une fonction ordinaire calcule et rend un résultat avec `return` : elle se termine, et tout ce qu'elle avait en mémoire disparaît. Un générateur fait autre chose : à `yield`, il **rend une valeur sans se terminer**. Son état — la position dans la boucle, les variables locales — est conservé, et la prochaine demande le fait repartir exactement de ce point.

C'est ce comportement qui en fait l'image juste d'un flux. Trois propriétés se correspondent terme à terme. La **boucle infinie** correspond au fait qu'un flux n'a pas de fin : `while True` ne s'arrête jamais, comme un capteur qui mesure sans interruption. Le **`yield`** correspond à l'arrivée d'un événement : une valeur, une seule, à cet instant. Et la **boucle `for`** correspond au consommateur : elle reçoit chaque mesure dès qu'elle est produite, sans attendre les suivantes.

La conséquence la plus importante est que le générateur ne stocke rien. Une fonction qui construirait la liste de toutes les mesures épuiserait la mémoire au bout de quelques heures ; le générateur, lui, ne détient jamais qu'une valeur à la fois. C'est exactement la contrainte d'un système de traitement de flux : on ne peut pas garder l'ensemble, donc on traite au passage. Les limites énoncées en conditions d'emploi — pas de retour en arrière, pas de longueur connue — ne sont pas des faiblesses de Python : ce sont les limites réelles d'un flux, que le générateur reproduit fidèlement.

Cas particuliers et pièges

- **Appeler `capteur()` n'exécute rien.** Cela crée l'objet générateur ; le corps ne démarre qu'à la première demande de valeur. C'est déroutant la première fois.
- **`len()` et l'indexation sont impossibles.** On ne peut pas écrire `valeurs[3]` ni compter les éléments ; il faudrait pour cela tout matérialiser, ce que l'on cherche précisément à éviter.
- **Un générateur épuisé ne se rejoue pas.** Pour reparcourir, il faut le recréer. Un flux réel, lui, peut parfois être rejoué depuis le journal : c'est une différence importante avec Kafka.
- **`time.sleep(1)` est une simulation.** Il imite la cadence d'un capteur ; dans un vrai producteur, l'attente vient de la source elle-même, pas d'une pause artificielle.

▶ 6. Le problème que la règle traite

Problème traité

Le problème est celui de la **représentation d'une suite infinie dans un programme**. Les structures habituelles — liste, tableau, fichier — supposent un contenu fini que l'on peut construire puis parcourir. Aucune ne permet d'écrire « la suite des mesures à venir ».

Le second problème est celui de la **mémoire**. Même pour une suite finie mais très longue, tout charger est impossible. Le générateur résout les deux en produisant à la demande, une valeur à la fois.

▶ 7. À quoi elle va nous servir

Utilité pratique

Elle nous servira dès le premier TP à **simuler une source de données** sans matériel : un générateur produit des températures, des paiements ou des clics, que l'on envoie ensuite à Kafka.

Elle nous servira ensuite de **modèle mental**. Quand nous écrirons une boucle de consommation Kafka, nous retrouverons exactement la structure de la boucle `for` : recevoir une valeur, la traiter, revenir attendre la suivante.

Elle nous servira enfin à **comprendre la contrainte de mémoire** du traitement de flux : puisqu'on ne garde pas l'ensemble, tout calcul devra se faire avec un état borné. C'est ce qui justifiera les sketches et les agrégats incrémentaux du chapitre 4.

▶ 8. Comment on y est arrivé

Origine de la règle

L'idée de produire les valeurs à la demande plutôt que toutes d'un coup est ancienne : on la trouve sous le nom d'évaluation paresseuse dans les langages fonctionnels, et sous celui de coroutine dans les travaux sur la programmation concurrente des années 1960. Python l'a introduite avec `yield` en la rendant accessible sans vocabulaire spécialisé.

Le besoin vient d'un raisonnement simple sur la mémoire. Pour produire `n` valeurs, une fonction ordinaire construit une liste de taille `n` : la mémoire occupée croît proportionnellement à `n`, et devient infinie quand la suite l'est. Un générateur ne détient qu'une valeur et l'état de sa boucle : la mémoire occupée est constante, indépendante de `n`. C'est ce passage d'une occupation proportionnelle à une occupation constante qui rend possible le traitement d'une suite infinie.

On notera que ce raisonnement est exactement celui qui fonde tout le traitement de flux : un système de streaming est, au fond, un générateur distribué et tolérant aux pannes.

▶ 9. Où on va l'utiliser

Domaines d'application

- **Simulation de capteurs en TP.** Produire des mesures réalistes sans matériel ; c'est l'usage direct de l'exemple du cours.
- **Lecture de gros fichiers.** Parcourir un fichier de plusieurs gigaoctets ligne par ligne sans le charger en mémoire ; le même mécanisme est à l'œuvre.
- **Chaînes de transformation.** On enchaîne des générateurs — lire, filtrer, enrichir — chacun traitant une valeur à la fois : c'est la structure d'un pipeline de streaming en miniature.
- **Pagination d'API.** Un générateur masque la récupération page par page et présente un flux continu d'éléments.

▶ 10. Quand on va l'utiliser

Situations de choix · les signaux à repérer

- **Quand la suite n'a pas de fin.** Capteur, journal, flux de clics : si l'on ne peut pas dire combien il y aura de valeurs, le générateur s'impose.
- **Quand le volume dépasse la mémoire.** Même pour une source finie, dès que le total ne tient pas en mémoire, on passe au traitement à la demande.
- **Quand le consommateur veut commencer tout de suite.** Si l'on doit traiter la première valeur sans attendre la dernière, c'est le signal d'un générateur.
- **Quand ce n'est pas le bon outil.** Si l'on a besoin de trier, de compter ou d'accéder par indice, il faut matérialiser les données : la liste reprend alors ses droits.

▶ 11. Comment on va l'utiliser

Méthode · écrire un producteur simulé

1. **Écrire la fonction avec une boucle infinie.** `while True:` traduit l'absence de fin du flux.
2. **Produire une valeur réaliste.** On choisit une loi plausible plutôt qu'un nombre arbitraire : `random.gauss(25, 2)` imite une température autour de 25 °C.
3. **Rendre la valeur avec `yield`.** Une seule valeur par tour ; on ne construit jamais de liste.
4. **Donner une cadence.** `time.sleep` fixe le rythme de production et permet d'observer le flux en direct.
5. **Consommer avec une boucle `for`.** On traite chaque valeur immédiatement ; c'est ici que viendra plus tard l'envoi vers Kafka.
6. **Ajouter le contexte de la notion 3.** Au lieu d'un nombre nu, on produit un dictionnaire avec l'identifiant du capteur, la grandeur et son unité, et l'horodatage : le générateur produit alors de vrais événements.

À retenir

Une boucle infinie avec `yield` produit des données qui arrivent une par une, sans fin, sans jamais s'accumuler en mémoire : c'est exactement l'idée d'un flux.

▶ 12. Exercices d'application corrigés

Application 14.1

Modifiez le générateur du cours pour qu'il produise non plus un nombre nu, mais un événement complet au sens des notions 3 et 8, pour le capteur `TEMP-04`.

Corrigé 14.1

Étapes 1 à 4 — la structure ne change pas. Boucle infinie, valeur réaliste, `yield`, cadence : tout cela est conservé.

Étape 6 — ajouter le contexte. On applique la notion 3 : sujet (`capteur`), grandeur avec unité dans le nom du champ (`temperature_c`), instant (`evenement_le` en UTC). On ajoute l'identifiant de message de la notion 8.

```
import random, time, uuid
from datetime import datetime, timezone

def capteur(capteur_id="TEMP-04"):
    """Produit des evenements de temperature, sans fin."""
    while True:
        yield {
            "type":          "mesure_relevee",
            "capteur":       capteur_id,
            "temperature_c": round(random.gauss(25, 2), 1),
            "evenement_le":  datetime.now(timezone.utc).isoformat(),
            "message_id":    str(uuid.uuid4()),
        }
        time.sleep(1)

for evenement in capteur():
    print(evenement)
```

Vérification. Chaque événement est interprétable seul : on sait de quel capteur il vient, ce qu'il mesure, dans quelle unité, à quel instant. L'horodatage est pris en UTC, conformément à la notion 10, et il représente bien l'heure d'*événement* au sens de la notion 11 : il est posé au moment de la mesure, pas à la réception.

Remarque. La mémoire occupée reste constante : à tout instant, un seul dictionnaire existe. On pourrait laisser tourner ce programme un an sans qu'il grossisse.

Application 14.2

Un étudiant écrit le code suivant et s'étonne que rien ne s'affiche et que la mémoire sature :

```
def capteur():
    mesures = []
    while True:
        mesures.append(round(random.gauss(25, 2), 1))
    return mesures

for v in capteur():
    print(v)
```

Expliquez les deux problèmes et corrigez.

Corrigé 14.2

Problème 1 — rien ne s'affiche. La fonction ne contient pas `yield` : ce n'est pas un générateur mais une fonction ordinaire, qui doit se terminer avant de rendre quoi que ce soit. Or `while True` ne se termine jamais : le `return` est inatteignable, la boucle `for` n'est jamais atteinte, et aucun affichage n'a lieu.

Problème 2 — la mémoire sature. Chaque tour ajoute un élément à `mesures`, sans jamais rien libérer. C'est exactement l'occupation proportionnelle décrite à la section 8 : la mémoire croît avec le nombre de valeurs, donc ici sans limite. Le programme finit par épuiser la mémoire disponible.

Les deux problèmes ont la même racine. L'étudiant a voulu *constituer* l'ensemble des mesures avant de le rendre. C'est précisément ce qui est impossible sur une suite infinie, et c'est le point que le générateur résout.

Correction.

```
def capteur():
    while True:
        yield round(random.gauss(25, 2), 1)
        time.sleep(1)

for v in capteur():
    print(v)
```

La liste disparaît, `yield` remplace `append`, et `time.sleep` donne une cadence observable. La mémoire reste constante et l'affichage commence immédiatement.

▶ 13. Exercices d'entraînement

Entraînement

1. Expliquez la différence entre `return` et `yield` en trois phrases.
2. Écrivez un générateur qui produit sans fin des paiements simulés avec une carte tirée parmi trois et un montant entre 10 et 500 MAD.
3. Pourquoi `len(capteur())` est-il impossible ? Quel rapport avec un flux réel ?

▶ 14. Exercices de difficulté croissante

Progression

1. facile Dans le code du cours, à quoi servent `while True`, `yield` et la boucle `for` ?
2. moyen Écrivez un second générateur qui prend le premier en entrée et ne laisse passer que les températures supérieures à 28 °C. Expliquez pourquoi la mémoire reste constante malgré l'enchaînement.
3. difficile On veut la moyenne des 10 dernières mesures, mise à jour à chaque nouvelle valeur. Écrivez-le avec un état borné, montrez que l'occupation mémoire ne dépend pas du nombre total de mesures, et dites à quelle notion du chapitre 4 cela correspond.

15. Problèmes du réel et leur solution détaillée

Les trois situations qui suivent mobilisent plusieurs notions à la fois. Lisez l'énoncé, cherchez seul, puis comparez avec la solution.

Domaine · Monétique

Problème 15.1 — Reprendre la seconde de Salma. La banque veut refuser un paiement si la même carte a déjà effectué trois paiements dans les deux dernières minutes. Le terminal est à Casablanca, le service de scoring à Francfort (48 ms par trajet), le calcul dure 35 ms, la capture 12 ms. Le réseau des cartes impose une réponse en 300 ms. Le flux est réparti sur 6 casiers et reçoit 12 000 paiements par seconde ; une machine en traite 5 000. Concevez la solution complète.

Solution 15.1

1. Qualifier la situation (notion 1). Le fait est « une carte est présentée », l'action est « accepter ou refuser », et le délai utile est contractuel : `D = 300 ms`. Les trois conditions de la règle sont réunies : source continue, action attachée, délai fini et connu. C'est du temps réel.

2. Lot ou flux (notion 2) ? Il n'existe aucun instant de complétude. Un batch respectant `P/2 ≤ 300 ms` exigerait `P ≤ 600 ms`, zone où le batch coûte plus cher qu'un flux. **Streaming.**

3. Vérifier le budget (notions 1, 7 et 10). `L = 12 + 48 + 35 + 48 = 143 ms ≤ 300 ms`. La conception tient, avec 157 ms de marge. Le réseau consomme 96 ms, soit 67 % du total : c'est le maillon dominant, et rapprocher le scoring de Casablanca le ramènerait à 53 ms (troisième raison de la notion 12).

4. Concevoir l'événement (notions 3 et 8). Sujet, grandeurs avec unités, instant, identifiant de message :

```
{ "type": "paiement_presente", "carte": "4970-XXXX-1234",
  "montant_mad": 18.0, "commerce": "Cafe Atlas", "ville": "Casablanca",
  "evenement_le": "2026-10-15T08:30:12.250Z", "message_id": "7c2e…" }
```

5. Choisir la clé (notion 9). La règle métier porte sur « la même carte » : la clé est le numéro de carte. Prendre la ville rendrait le calcul *faux*, puisqu'une carte volée utilisée dans deux villes partirait dans deux casiers. Cardinalité élevée, distribution équilibrée, stabilité assurée : les quatre vérifications passent.

6. Écrire la requête (notion 5). Version bornée : `SELECT carte, COUNT(*) FROM paiements GROUP BY carte`. L'agrégat exige l'ensemble complet : on introduit une fenêtre de 2 minutes glissante, et l'on déclenche le refus dès que le compte atteint 3.

7. Choisir l'heure (notion 11). « Trois paiements en deux minutes » décrit le monde réel : **heure d'événement**. Mais la décision doit tomber en 300 ms : on ne peut pas attendre les retardataires. On fixe donc une limite d'attente très courte, et l'on accepte qu'un paiement tardif ne soit pas compté — arbitrage explicite entre justesse et latence, à faire valider par le métier.

8. Dimensionner (notion 12). `12 000 / 5 000 = 2,4` donc 3 machines pour la capacité. Avec une panne : `2 × 5 000 = 10 000 < 12 000`, insuffisant ; il faut **4 machines** (`3 × 5 000 = 15 000 ≥ 12 000` après panne). Les 6 casiers permettent bien d'occuper 4 machines.

9. Traiter les imperfections (notion 7). Retard : absorbé par la marge de 157 ms, avec une décision par défaut si le scoring ne répond pas. Doublon : le `message_id` permet de ne pas compter

deux fois le même paiement, ce qui provoquerait un refus injustifié. Perte : réplication des partitions.

10. Monter l'environnement (notion 13). En TP, Kafka, Flink et l'interface d'administration dans des conteneurs ; entre conteneurs on s'adresse par `kafka:9092`, depuis l'hôte par `localhost:9092`.

Domaine · Industrie · maintenance prédictive

Problème 15.2 — La pompe qui dérive. Une usine surveille 400 pompes. Chaque pompe porte un capteur de vibration qui mesure toutes les secondes. On veut alerter quand la moyenne des vibrations d'une pompe sur 5 minutes dépasse 8 mm/s. Les capteurs sont sur batterie et, pour économiser l'énergie, envoient leurs mesures groupées toutes les 2 minutes. Le service maintenance dispose de 30 minutes pour intervenir avant la casse. Concevez la solution.

Solution 15.2

1. Qualifier (notion 1). Fait : « une vibration est mesurée ». Action : « envoyer une équipe de maintenance ». Délai utile : `D = 30 min = 1800 s`. C'est du temps réel, mais avec un budget très large comparé au cas bancaire.

2. Lot ou flux (notion 2) ? Un batch de période `P` donnerait `L ≈ P/2`. Avec `P = 10 min`, `L ≈ 300 s`, très en dessous de 1800 s. **Un micro-batch de quelques minutes suffirait donc**, et c'est une conclusion importante : tout ne demande pas du streaming pur. On choisira néanmoins un flux, pour une raison technique exposée au point 5.

3. Le point décisif — les deux heures (notion 11). Les capteurs groupent leurs envois toutes les 2 minutes : une mesure peut donc arriver avec jusqu'à 120 s de retard, et un lot de 120 mesures arrive d'un coup. Calculer la moyenne « sur 5 minutes » en heure de traitement donnerait n'importe quoi : toutes les mesures d'un envoi seraient datées du même instant. **Heure d'événement obligatoire**, et chaque mesure doit porter son propre horodatage, posé par le capteur au moment de la mesure (notion 10).

4. Concevoir l'événement (notions 3 et 8). Un message par mesure, même si l'envoi est groupé : `{"capteur":"VIB-12","equipement":"P3","vibration_mm_s":7.4,"evenement_le":"…Z","message_id":"…"}`. Grouper plusieurs mesures dans un seul message serait une faute : le champ ne serait plus atomique.

5. Choisir la clé (notion 9). La moyenne porte sur « une pompe » : la clé est l'identifiant de pompe (ou de capteur, équivalent ici). 400 clés distinctes, charge identique pour chacune : distribution parfaitement équilibrée. C'est aussi la raison de préférer un flux : le regroupement par clé donne gratuitement la localité d'état nécessaire à la moyenne glissante.

6. Écrire la requête (notion 5). `SELECT pompe, AVG(vibration_mm_s) FROM mesures GROUP BY pompe, fenêtre glissante de 5 minutes sur evenement_le HAVING AVG(…) > 8`. La fenêtre rend l'ensemble fini à chaque instant.

7. Fixer la limite d'attente (notion 11). Le retard maximal attendu est de 120 s. On fixe la limite à 180 s pour couvrir une variation. Latence totale : 120 s de groupage + 180 s d'attente + quelques secondes de calcul, soit environ 5 minutes. Très en dessous de `D = 30 min` : **le budget tient largement**.

8. Dimensionner (notion 12). Débit : `400 mesures/s`. C'est très faible : une seule machine suffit pour la capacité. On en met deux, pour la disponibilité seulement. Ici la raison « capacité » ne s'applique pas ; distribuer davantage ajouterait de la complexité sans bénéfice.

9. Imperfections (notion 7). Un doublon fausserait la moyenne : la déduplication par `message_id` est indispensable. Une perte isolée sur 300 mesures de fenêtre est sans gravité ; on surveille néanmoins le nombre de mesures par fenêtre et l'on ignore une fenêtre trop incomplète, pour ne pas alerter à tort.

10. Leçon. Ce cas montre que le temps réel n'est pas une affaire de millisecondes : avec `D = 30 min`, la difficulté n'est pas la vitesse mais la *justesse temporelle*, c'est-à-dire le traitement correct des retards.

Domaine · Transport urbain

Problème 15.3 — Les trottinettes de Rabat. Un service de trottinettes en libre-service veut : (a) afficher sur une carte, en continu, le nombre de trottinettes disponibles par quartier ; (b) facturer chaque trajet à la minute ; (c) alerter l'équipe terrain quand une trottinette reste immobile plus de 48 heures. Les trottinettes perdent le réseau dans les parkings souterrains et renvoient leurs événements à la reconnexion. Traitez les trois besoins.

Solution 15.3

(a) Carte des disponibilités par quartier. Qualification : l'action est « l'usager choisit où aller », le délai utile est de l'ordre de $D = 30$ s — au-delà, la carte ment. Un batch exigerait $P \leq 60$ s : on est dans la zone du flux. Clé : le quartier, puisque le regroupement porte sur lui ; attention, la cardinalité est faible (quelques dizaines) et un quartier central sera une clé chaude — acceptable ici car le débit total est modeste. Heure : **heure de traitement**, exceptionnellement. « Disponible maintenant » décrit l'état présent du système, pas un fait passé ; une trottinette qui n'a pas donné signe de vie depuis 10 minutes doit d'ailleurs être retirée de la carte.

(b) Facturation à la minute. Qualification : aucune action immédiate n'est attachée — la facture peut partir quelques minutes après la fin du trajet. D vaut plusieurs minutes, voire l'heure. Un traitement par lots fréquent suffirait. Heure : **heure d'événement, impérativement**. Un trajet terminé dans un parking souterrain à 18 h 02 et remonté à 18 h 40 doit être facturé sur sa durée réelle, pas sur les 38 minutes de déconnexion. C'est une exigence contractuelle, pas un choix technique. Clé : l'identifiant de trajet, pour que début et fin se retrouvent dans le même casier. Doublons : critiques ici, puisqu'ils facturent deux fois ; déduplication obligatoire par `message_id`.

(c) Alerte d'immobilité de 48 heures. Qualification : l'action est « envoyer une équipe », D vaut quelques heures. Particularité : **l'événement à détecter est une absence d'événement**. Aucun message n'arrive ; il faut donc un mécanisme qui se déclenche sur le silence, c'est-à-dire un minuteur par trottinette, armé à chaque message reçu. Clé : l'identifiant de trottinette. Piège à signaler : une trottinette en parking souterrain depuis 50 heures a pu être utilisée sans qu'on le sache ; l'alerte doit donc être levée, puis annulée si des événements tardifs la contredisent.

Synthèse. Les trois besoins vivent dans le même système mais n'ont ni le même D, ni la même clé, ni la même heure de référence. C'est le cas général en entreprise : on ne conçoit pas « un » pipeline temps réel, on conçoit un pipeline par question posée, alimentés par le même flux d'événements.

16. Banque d'exercices de fin de cours

Trente exercices couvrant les quatorze notions. Les corrigés détaillés se trouvent à la section 17, après la banque : cherchez d'abord.

▶ Dix exercices faciles

Faciles · E1 à E10

1. facile Un événement se produit à `14:20:03.100` ; le résultat est disponible à `14:20:03.480`. Calculez la latence en millisecondes.
2. facile Convertissez : 4 s en ms, 2500 ms en s, 7 ms en µs, 0,25 h en s.
3. facile Parmi ces situations, lesquelles exigent un traitement en temps réel ? (a) le bulletin de notes du semestre ; (b) l'alerte d'un détecteur de fumée ; (c) le calcul du salaire mensuel ; (d) le GPS qui recalcule l'itinéraire après une erreur de route.
4. facile Citez les quatre rangements de données vus en cours et donnez pour chacun l'image simple associée.
5. facile Dans `localhost:9092`, que désigne chaque partie ? Utilisez l'analogie postale du cours.
6. facile Écrivez en JSON : « Le capteur HUM-02 a mesuré 54 % d'humidité le 12 mars 2027 à 9 h 15 UTC. »
7. facile Dans la paire `clé = "carte 4970-1234", valeur = { montant: 18, ville: "Casablanca" }`, que dit la clé et que contient la valeur ?
8. facile Une carte postale est écrite le 1er août et reçue le 10 août. Donnez l'heure d'événement, l'heure de traitement et le retard.
9. facile Citez les trois raisons d'utiliser plusieurs machines, et dites comment on appelle un ensemble de machines qui travaillent ensemble.
10. facile Un générateur Python avec `while True` et `yield` ressemble-t-il à un fichier ou à un flux sans fin ? Justifiez en une phrase.

▶ Dix exercices de difficulté moyenne

Moyens · E11 à E20

11. **moyen** Une chaîne comprend : capture 8 ms, réseau aller 55 ms, file 40 ms, calcul 25 ms, réseau retour 55 ms. Le délai utile est de 150 ms. La conception tient-elle ? Quel maillon optimisez-vous en premier et pourquoi ?
12. **moyen** Un traitement par lots tourne toutes les 45 minutes et son calcul dure 6 minutes. Donnez la latence moyenne et la latence maximale. À partir de quelle période ce lot devient-il impossible à tenir ?
13. **moyen** Un capteur en panne écrit `0` au lieu de ne rien écrire. Sur 500 mesures réelles de moyenne 40 °C, 75 sont remplacées par des zéros. Calculez la moyenne observée et l'erreur relative.
14. **moyen** Transformez en requête continue : « nombre de connexions par utilisateur », avec une fenêtre d'une minute. Que devient le résultat pour un utilisateur inactif pendant dix minutes ?
15. **moyen** Un serveur traite une demande en 25 ms. À partir de quel débit d'arrivée sature-t-il s'il est mono-tâche ? Que se passe-t-il juste au-dessus de ce seuil ?
16. **moyen** Un consommateur reçoit `1, 3, 2, 3, 5, 6` et le message 4 n'arrive jamais. Identifiez chaque anomalie, sa cause et sa compensation.
17. **moyen** Avec 4 casiers et des clés dont le hachage vaut 17, 22, 35, 40, 22 et 31, donnez le casier de chaque message et commentez l'équilibre.
18. **moyen** Un capteur mesure à `23:59:59.700Z` le 31 décembre 2026 et le résultat est produit à `00:00:00.250Z` le 1er janvier 2027. Calculez la latence et expliquez le piège.
19. **moyen** Un service doit tenir 45 000 messages/s ; une machine en traite 8 000 et l'on veut survivre à une panne. Combien de machines ? Quelle charge par machine en régime normal et après panne ?
20. **moyen** Écrivez un générateur Python qui filtre un autre générateur et ne laisse passer que les valeurs supérieures à un seuil. Expliquez pourquoi la mémoire reste constante.

▶ Dix exercices difficiles

Difficiles · E21 à E30

21. **difficile** Un système affiche une latence moyenne de 35 ms, un p95 de 110 ms et un p99 de 2800 ms, pour `D = 250 ms`. Respecte-t-il la règle ? Reformulez le critère d'acceptation et chiffrez l'impact sur un million de transactions.
22. **difficile** Un métier demande le nombre exact de visiteurs *uniques* du jour, rafraîchi toutes les 10 secondes. Montrez que ce besoin mêle une contrainte batch et une contrainte flux, et proposez deux conceptions en disant ce que chacune sacrifie.
23. **difficile** Concevez le schéma d'événement d'un service de livraison à domicile. Justifiez chaque champ par une notion du cours, puis expliquez comment ajouter plus tard un champ « température du colis » sans casser les consommateurs en place.
24. **difficile** Une banque doit conserver chaque transaction pour trois usages : solde instantané dans l'application, reporting réglementaire sur 5 ans, détection de fraude en temps réel. Décrivez le parcours d'une même transaction à travers les contenants, en disant lequel reçoit l'écriture en premier et pourquoi.
25. **difficile** On veut la médiane des montants de la dernière heure, mise à jour chaque seconde. Montrez pourquoi le calcul exact est coûteux et décrivez deux stratégies, en chiffrant ce que chacune coûte en mémoire ou en précision.
26. **difficile** Comparez interrogation périodique, interrogation longue et connexion poussée sur trois critères : latence ajoutée, nombre de messages échangés, complexité. Concluez pour un système d'alerte de fraude à 100 000 utilisateurs.
27. **difficile** Montrez qu'un simple mécanisme d'accusé de réception ne peut garantir à la fois « aucune perte » et « aucun doublon », puis expliquez comment l'idempotence contourne cette impossibilité.
28. **difficile** Trois services ont des horloges décalées de `+15 ms`, `−8 ms` et `+2 ms`. Leurs horodatages locaux pour trois événements valent 100, 104 et 110 ms. L'ordre apparent est-il fiable ? Quelle marge d'incertitude annoncer ?
29. **difficile** Un tableau de bord publie un total horaire. Des retardataires arrivent jusqu'à 20 minutes après la fin de la tranche. Proposez trois stratégies de publication en comparant latence, justesse et perception par l'utilisateur.

30. **difficile** Un cluster passe de 8 à 16 machines sans gain de débit. Donnez trois explications possibles classées par probabilité, la vérification à mener pour chacune, et la correction associée.

17. Corrigé de tous les exercices de la section 16

▶ Corrigés des exercices faciles

E1

Les deux instants partagent la même heure, la même minute et la même seconde ; seule la partie décimale diffère. On travaille donc directement en millisecondes : `L = 480 − 100 = 380 ms`.

E2

- `4 s = 4 × 1000 = 4000 ms`
- `2500 ms = 2500 / 1000 = 2,5 s`
- `7 ms = 7 × 1000 = 7000 µs`
- `0,25 h = 0,25 × 3600 = 900 s`, soit 15 minutes.

E3

(b) et (d) exigent du temps réel. Dans les deux cas une réponse tardive n'a plus de valeur : l'incendie se propage, la mauvaise route est déjà prise. Une action immédiate est attachée au résultat, et le délai utile se compte en secondes.

(a) et (c) peuvent attendre : un traitement par lots suffit. Le test du lendemain le confirme — un bulletin ou un salaire calculé demain matin sert encore.

À retenir : le temps réel se justifie quand la valeur de la réponse diminue vite avec le temps.

E4

- **Fichier** — une feuille dans un classeur ; sert à échanger et archiver.
- **Base de données** — une bibliothèque bien rangée avec un bibliothécaire ; sert à chercher et modifier rapidement.
- **Entrepôt de données** — un grand entrepôt pour les analyses ; sert aux tableaux de bord et aux rapports.
- **Lac de données** — un grand lac où l'on dépose tout, brut ; sert au Big Data et à l'apprentissage automatique.

Point commun : dans les quatre cas, les données sont au repos et attendent qu'on vienne les interroger.

E5

`localhost` est l'**adresse de la machine**, comme une adresse postale ; ici elle désigne la machine sur laquelle on se trouve. `9092` est le **port**, comme le numéro d'appartement dans l'immeuble : il désigne lequel des programmes de cette machine doit recevoir le message. Sans le numéro d'appartement, le facteur sait dans quel immeuble aller mais pas à quelle porte frapper.

E6

```
{
  "type":         "mesure_relevee",
  "capteur":      "HUM-02",
  "humidite_pct": 54,
  "evenement_le": "2027-03-12T09:15:00Z"
}
```

Le sujet (`capteur`), la grandeur avec son unité dans le nom du champ (`humidite_pct`) et l'instant en UTC sont présents : le message est interprétable seul.

E7

La **clé** dit de qui ou de quoi on parle : ici, de la carte 4970-1234. La **valeur** contient le détail du fait : le montant de 18 et la ville de Casablanca. Comme le casier de vestiaire : la clé est le numéro du casier, la valeur est ce qu'il contient. Conséquence : tous les paiements de cette carte iront dans le même casier.

E8

Heure d'événement : le 1er août, moment où le fait « écrire la carte » s'est produit. **Heure de traitement : le 10 août**, moment où le message est reçu et traité. **Retard : 9 jours.** Si la famille range ses cartes par date de réception, elle croira que la carte a été écrite le 10 : c'est exactement l'erreur que commet un calcul fait en heure de traitement.

E9

Les trois raisons : **(1)** il y a trop de travail pour une seule machine, on partage comme on ouvre plusieurs caisses dans un supermarché ; **(2)** ne pas tout perdre en cas de panne, une autre machine prend le relais à partir de copies ; **(3)** être proche des utilisateurs, pour répondre plus vite malgré la latence réseau.

Un ensemble de machines qui travaillent ensemble s'appelle un **cluster** ; on parle de système distribué.

E10

À un **flux sans fin**. Un fichier a une taille connue et une dernière ligne ; on peut le relire, compter ses éléments, revenir en arrière. Un générateur avec `while True` ne se termine jamais, produit ses valeurs une par une à la demande, et ne permet ni `len()` ni retour en arrière : ce sont exactement les propriétés d'un flux.

▶ Corrigés des exercices moyens

E11

Somme des maillons : `L = 8 + 55 + 40 + 25 + 55 = 183 ms`. Or `D = 150 ms`, donc `L > D` : **la conception ne tient pas**, le dépassement est de 33 ms.

Maillon dominant. Le réseau totalise `55 + 55 = 110 ms`, soit 60 % du budget ; la file en consomme 22 %, le calcul 14 %, la capture 4 %.

Action. On optimise le réseau en premier, car c'est là que se trouve la marge. Rapprocher le service des utilisateurs est la seule parade réelle : en ramenant chaque trajet à 25 ms, on obtient `8 + 25 + 40 + 25 + 25 = 123 ms ≤ 150 ms`. Optimiser le calcul de 25 à 15 ms, en revanche, ne ferait gagner que 10 ms et ne suffirait pas : travailler sur un maillon mineur est une erreur classique.

E12

Latence moyenne : `P/2 + c = 2700/2 + 360 = 1350 + 360 =` **`1710 s`**, soit environ 28 min 30 s.

Latence maximale : `P + c = 2700 + 360 =` **`3060 s`**, soit 51 minutes, pour un événement arrivant juste après le départ d'un lot.

Seuil d'impossibilité. Le lot devient intenable dès que `P ≤ c = 6 minutes` : le calcul d'un lot ne serait pas terminé que le suivant devrait démarrer. Les exécutions se chevauchent, la file s'accumule et la latence croît sans limite. En pratique on garde une marge et l'on ne descend pas sous `P = 2c`, soit 12 minutes ; en dessous, il faut passer en flux.

E13

Somme réelle : `500 × 40 = 20 000`.

Effet des zéros. 75 mesures valant en moyenne 40 sont remplacées par 0 : on retire `75 × 40 = 3000` de la somme. Somme observée : `20 000 − 3000 = 17 000`.

Moyenne observée : `17 000 / 500 =` **`34 °C`** au lieu de 40 °C.

Erreur relative : `(40 − 34)/40 = 6/40 =` **`15 %`** — exactement la proportion de mesures corrompues, ce qui n'est pas un hasard : remplacer une fraction `f` des valeurs par zéro fait perdre la fraction `f` de la moyenne.

Leçon. Une valeur absente n'est pas zéro. Écrire 0 transforme une panne en mesure et biaise tous les calculs en aval, sans qu'aucune alerte ne se déclenche.

E14

```
SELECT utilisateur, COUNT(*) AS connexions
FROM   connexions
GROUP BY utilisateur,
         TUMBLE(evenement_le, INTERVAL '1' MINUTE);
```

Cas de l'utilisateur inactif. Pendant ses dix minutes d'inactivité, aucun événement ne porte son identifiant : **aucune ligne n'est émise pour lui**. Le résultat n'est pas « 0 », c'est une absence de groupe. La distinction compte : un tableau de bord qui affiche la dernière valeur connue montrera le compte de la dernière minute active, ce qui est trompeur. Si l'on veut un zéro explicite, il faut une liste de référence des utilisateurs et une jointure, ce qui change la nature du calcul.

E15

Capacité maximale. Un serveur mono-tâche qui met 25 ms par demande en traite au plus `1000 / 25 = 40 demandes par seconde`.

Juste au-dessus du seuil. Si le débit d'arrivée dépasse 40 demandes/s, ne serait-ce que de peu, chaque seconde apporte plus de travail que le serveur n'en évacue. La file s'allonge *indéfiniment*, et le temps d'attente croît linéairement avec la durée de la surcharge. Il n'y a pas de régime stable dégradé : on passe d'un système qui répond à un système qui s'effondre.

Nuance. Même en dessous de 40, la file n'est pas vide : les arrivées sont irrégulières. À 35 demandes/s, soit 87 % de la capacité, l'attente moyenne est déjà plusieurs fois supérieure à 25 ms. C'est pourquoi on dimensionne rarement au-delà de 70 % de la capacité théorique.

E16

Séquence reçue : `1, 3, 2, 3, 5, 6` ; le message 4 n'arrive jamais.

Anomalie 1 — désordre (3 avant 2). *Cause :* les paquets ont emprunté des chemins de durées différentes, ou l'un a attendu dans un tampon plus chargé. *Compensation :* ne pas se fier à l'ordre d'arrivée, raisonner sur l'horodatage d'événement porté par le message, et attendre un court délai avant de clore un calcul.

Anomalie 2 — duplication (3 reçu deux fois). *Cause :* l'émetteur n'a pas reçu l'accusé à temps et a réémis ; le premier exemplaire est finalement arrivé aussi. *Compensation :* identifiant unique par message et traitement idempotent.

Anomalie 3 — perte (4 absent). *Cause :* un tampon saturé a jeté le paquet et la réémission a échoué ou n'a pas eu lieu. *Compensation :* accusés de réception et réémission, c'est-à-dire la garantie « au moins une fois ».

Remarque. Les compensations s'opposent : lutter contre la perte produit davantage de doublons, et tolérer le désordre coûte de la latence. C'est un arbitrage, pas un problème à supprimer.

E17

On applique `casier = h(clé) mod 4` :

Hachage	Calcul	Casier
17	17 = 4×4 + 1	1
22	22 = 5×4 + 2	2
35	35 = 8×4 + 3	3
40	40 = 10×4 + 0	0
22	même clé que plus haut	2
31	31 = 7×4 + 3	3

Répartition : casier 0 → 1 message ; casier 1 → 1 ; casier 2 → 2 ; casier 3 → 2.

Commentaire. L'équilibre est correct pour un si petit échantillon. Le point à relever est que les deux messages de hachage 22 tombent dans le même casier : c'est le comportement attendu et souhaité, puisqu'il s'agit de la même clé. C'est précisément ce qui garantit leur ordre relatif.

E18

Calcul. De `23:59:59.700` à `00:00:00.000`, il s'écoule 300 ms. De `00:00:00.000` à `00:00:00.250`, 250 ms. Donc `L = 300 + 250 = **550 ms**`.

Le piège. Une soustraction naïve des seules heures donnerait `00:00:00.250 − 23:59:59.700`, soit un résultat négatif d'environ 23 h 59 min. L'erreur vient d'avoir ignoré le changement de jour, et même ici d'année.

Comment l'éviter. On ne soustrait jamais des champs d'affichage : on convertit d'abord les deux instants en une grandeur unique et continue — par exemple le nombre de millisecondes depuis une origine fixe — puis on soustrait. C'est la raison d'être du format complet `AAAA-MM-JJTHH:MM:SS.sssZ` : il porte la date, donc le passage au jour suivant est explicite.

E19

Capacité brute : `45 000 / 8 000 = 5,625`, donc 6 machines au minimum (`6 × 8000 = 48 000`).

Test de la panne. Avec 6 machines dont une tombe : `5 × 8000 = 40 000 < 45 000`. Insuffisant. Il faut donc **7 machines** : après une panne, `6 × 8000 = 48 000 ≥ 45 000`.

Charge par machine. En régime normal : `45 000 / 7 ≈ 6429` messages/s par machine, soit **80 %** de sa capacité. Après une panne : `45 000 / 6 = 7500`, soit **94 %**.

Commentaire. 94 % en mode dégradé est très tendu : au moindre pic de trafic, la file s'accumule. Une huitième machine ramènerait la charge à 70 % en normal et 80 % après panne ; c'est le dimensionnement que l'on proposerait en production. Il faut par ailleurs au moins 7 casiers pour que 7 machines travaillent.

E20

```python
def au_dessus_de(source, seuil):
    """Ne laisse passer que les valeurs strictement superieures au seuil."""
    for valeur in source:
        if valeur > seuil:
            yield valeur

for v in au_dessus_de(capteur(), 28):
    print("Alerte :", v)
```

Pourquoi la mémoire reste constante. Aucun des deux générateurs ne construit de liste. À tout instant, une seule valeur circule : `capteur()` en produit une, `au_dessus_de` la reçoit, la compare, la transmet ou l'abandonne, puis en redemande une. L'occupation mémoire ne dépend donc pas du nombre total de mesures mais seulement du nombre d'étages de la chaîne — ici deux.

Remarque. C'est exactement la structure d'un pipeline de streaming : une suite d'opérateurs qui se passent les événements un par un, chacun avec un état borné.

▶ Corrigés des exercices difficiles

E21

Lecture des chiffres. La moyenne (35 ms) et le p95 (110 ms) respectent `D = 250 ms`. Le p99 (2800 ms) le dépasse d'un facteur 11.

Réponse. Le système **ne respecte pas la règle**. Un p99 de 2800 ms signifie que 1 % des transactions dépassent 2800 ms, donc largement `D`. On ne juge jamais un système temps réel sur sa moyenne : elle est dominée par les cas faciles et masque complètement la queue de distribution.

Reformulation du critère. Au lieu de « `L ≤ D` », on écrit : « `p99(L) ≤ D` », ou de façon plus complète « au moins 99 % des transactions doivent être traitées en moins de 250 ms, et aucune ne doit dépasser 2 s ». Le critère devient vérifiable et contractualisable.

Impact chiffré. Sur un million de transactions, 1 % dépasse le p99 : **10 000 transactions** sortent du budget. Si chacune correspond à un paiement refusé par dépassement de délai, ce sont 10 000 clients mécontents — un volume parfaitement visible du service client, alors que le tableau de bord affiche fièrement « 35 ms de moyenne ».

Piste de correction. Un p99 très au-dessus du p95 désigne presque toujours un phénomène de file : saturation périodique, pause de ramasse-miettes, ou clé chaude. On cherche la cause dans cette direction, pas dans le code du calcul.

E22

La contrainte batch. « Exact » et « uniques » : compter des visiteurs distincts exige de se souvenir de *tous* les identifiants déjà vus dans la journée, sinon on ne peut pas savoir si celui qui arrive est nouveau. C'est un `COUNT(DISTINCT)`, opérateur qui demande l'ensemble complet.

La contrainte flux. « Rafraîchi toutes les 10 secondes » : `D ≈ 10 s`, ce qui exclut tout traitement par lots de période raisonnable. Les deux contraintes tirent en sens opposé.

Conception A — exacte, avec état complet. On maintient en mémoire l'ensemble des identifiants vus depuis minuit ; chaque nouvel événement teste l'appartenance et incrémente le compteur si l'identifiant est nouveau. *Sacrifice :* la mémoire croît avec le nombre de visiteurs uniques — pour 50 millions de visiteurs et 16 octets par identifiant, environ 800 Mo par jour et par partition, auxquels s'ajoute le coût de la structure. Il faut aussi gérer la remise à zéro à minuit et la reprise après panne de cet état volumineux.

Conception B — approchée, par résumé probabiliste. On utilise un sketch de cardinalité (HyperLogLog), qui estime le nombre de valeurs distinctes avec une mémoire fixe de quelques kilooctets. *Sacrifice :* le résultat n'est plus exact, l'erreur typique étant de l'ordre de 1 à 2 %. En contrepartie, la mémoire est constante, la reprise après panne triviale, et les résultats de plusieurs partitions se combinent sans recompter.

Recommandation. On retourne voir le métier avec une question précise : « une erreur de 1 % sur un compteur affiché toutes les 10 secondes change-t-elle une décision ? » Dans la quasi-totalité des cas, la réponse est non, et la conception B l'emporte. Le mot « exact » d'un cahier des charges est rarement une exigence réelle ; c'est souvent une formulation par défaut.

E23

```
{
  "type":           "livraison_livree",
  "livraison_id":   "LIV-2027-004821",
  "commande_id":    "CMD-2027-119034",
  "livreur_id":     "LVR-77",
  "origine":        "Rabat-Agdal",
  "destination":    "Rabat-Hassan",
  "poids_kg":       3.2,
  "montant_mad":    145.0,
  "evenement_le":   "2027-03-12T18:42:05Z",
  "message_id":     "b41e…",
  "schema_version": 1
}
```

Justification champ par champ. `type` nomme le fait au passé (notion 8) : un événement décrit ce qui s'est produit, pas un ordre. `livraison_id` est le sujet, donc la clé de partitionnement (notion 9) ; c'est lui qui garantit que début et fin d'une même livraison se retrouvent dans le même casier. `commande_id` et `livreur_id` permettent les rapprochements avec d'autres flux (notion 3 : sans sujet commun, pas de jointure). `poids_kg` et `montant_mad` portent leur unité dans leur nom (notion 3). `evenement_le` est l'heure d'événement en UTC (notions 10 et 11), indispensable pour un livreur momentanément hors réseau. `message_id` permet la déduplication (notion 7).

Ajouter « température du colis » sans rien casser. Trois règles suffisent. *(1)* Le nouveau champ `temperature_c` est **optionnel** : un consommateur qui ne le connaît pas l'ignore, et JSON l'y autorise sans erreur. *(2)* On n'enlève ni ne renomme aucun champ existant : renommer est la seule opération qui casse à coup sûr. *(3)* On incrémente `schema_version` à 2 et l'on déploie **les consommateurs avant le producteur**, pour qu'ils soient prêts quand le champ apparaît. En production, c'est un registre de schémas qui fait respecter ces règles automatiquement.

E24

Écriture en premier : dans le journal d'événements (la file de messages), pas dans la base. La raison tient en une phrase : c'est le seul contenant qui puisse servir les trois usages, parce qu'il est le point unique où la transaction existe avant d'être interprétée. Écrire d'abord dans la base obligerait chaque autre usage à venir l'interroger, avec le délai et la charge que cela suppose.

Parcours 1 — détection de fraude (temps réel). Un consommateur lit le journal avec quelques millisecondes de retard, calcule le score, et répond. D de quelques centaines de millisecondes ; clé = numéro de carte ; heure d'événement avec une limite d'attente très courte. Rien n'est stocké durablement ici sinon l'état glissant.

Parcours 2 — solde instantané (application mobile). Un second consommateur, indépendant du premier, applique la transaction au solde dans une **base de données** (notion 4 : accès unitaire, écriture fréquente, accès concurrents). D de l'ordre de la seconde. L'idempotence est critique : un doublon débiterait deux fois.

Parcours 3 — reporting réglementaire (5 ans). Un troisième consommateur dépose les transactions dans un **entrepôt** ou un **lac**, par lots, pour les requêtes agrégées et l'archivage. D de plusieurs heures ; traitement par lots parfaitement légitime, et même préférable (notion 2).

Ce que cette architecture apporte. Les trois consommateurs sont indépendants : l'un peut tomber sans affecter les autres, chacun avance à son rythme, et l'on peut en ajouter un quatrième plus tard en rejouant le journal depuis le début — ce qui serait impossible si la transaction n'avait été écrite que dans la base. C'est l'idée de l'*event sourcing*, développée au chapitre 2.

E25

Pourquoi l'exact est coûteux. La médiane n'est pas décomposable : contrairement à une somme ou à un maximum, on ne peut pas la mettre à jour à partir de l'ancienne valeur et du nouvel élément. Il faut connaître la position relative de toutes les valeurs. Pour une fenêtre d'une heure à 10 000 messages/s, cela représente `3600 × 10 000 = 36 millions` de montants à garder triés, soit environ 288 Mo en flottants 64 bits, plus le coût de la structure ordonnée. Et il faut répéter l'opération chaque seconde, en insérant 10 000 valeurs et en en retirant autant.

Stratégie A — exacte, par structure ordonnée glissante. On maintient deux tas (un tas-max pour la moitié basse, un tas-min pour la moitié haute), plus une file pour l'expiration. *Coût :* environ 300 à 500 Mo de mémoire par partition, insertion et retrait en temps logarithmique. *Avantage :* résultat exact. *Limite :* la mémoire croît linéairement avec le débit et la taille de la fenêtre ; doubler l'un ou l'autre double le coût.

Stratégie B — approchée, par histogramme ou sketch de quantiles. On découpe l'axe des montants en intervalles et l'on compte les occurrences, ou l'on utilise un sketch de quantiles. *Coût :* quelques kilooctets, indépendamment du débit, avec une erreur de l'ordre de 1 % sur le quantile. *Avantage :* mémoire constante, fusion triviale entre partitions, reprise après panne immédiate.

Conclusion. Pour une médiane affichée sur un tableau de bord, la stratégie B est presque toujours le bon choix : personne ne prend une décision différente selon que la médiane vaut 148 ou 149 MAD. La stratégie A ne se justifie que si l'exactitude a une valeur réglementaire.

E26

Technique	Latence ajoutée	Messages (100 000 utilisateurs)	Complexité
Interrogation périodique (1 s)	`P/2 = 500 ms` en moyenne	100 000 req/s, dont presque toutes vides	Très faible
Interrogation longue	Quelques ms	1 requête ouverte par utilisateur, renouvelée	Moyenne
Connexion poussée	Quelques ms	1 connexion par utilisateur, 1 message par alerte	Moyenne à élevée

Analyse de l'interrogation périodique. La latence suit exactement la formule du batch : avec `P = 1 s`, on ajoute 500 ms en moyenne. Surtout, 100 000 requêtes par seconde pour quelques dizaines d'alertes réelles signifie que plus de 99,9 % du trafic est inutile ; chaque requête occupe une place dans la file du serveur, qui devient le goulot.

Analyse de l'interrogation longue. Le serveur garde la requête ouverte jusqu'à ce qu'une alerte survienne. La latence tombe à quelques millisecondes et le nombre de messages s'effondre. En contrepartie, 100 000 connexions restent ouvertes simultanément, ce qui demande un serveur conçu pour cela.

Analyse de la connexion poussée. Mêmes bénéfices, avec en plus la bidirectionnalité et l'absence de renouvellement. C'est l'inversion complète de l'initiative : le serveur cesse d'être passif.

Conclusion pour la fraude. Connexion poussée. Le délai utile d'une alerte de fraude se compte en secondes ; ajouter 500 ms de latence et 100 000 requêtes par seconde pour un bénéfice nul serait un mauvais arbitrage sur les deux tableaux à la fois. L'interrogation périodique ne reste défendable que pour un prototype ou un très petit nombre d'utilisateurs.

E27

Le point de départ. L'émetteur envoie un message et attend un accusé. S'il ne le reçoit pas dans un certain délai, il doit choisir entre deux actions, et deux seulement : réémettre, ou abandonner.

Pourquoi il ne peut pas décider correctement. L'absence d'accusé a trois causes possibles, que l'émetteur ne peut pas distinguer : le message a été perdu ; le message est arrivé mais l'accusé a été perdu ; tout est simplement très lent. Aucune information disponible localement ne permet de trancher, puisque les trois produisent exactement le même symptôme : le silence.

Conséquence du choix « réémettre ». Dans le cas où seul l'accusé avait été perdu, le récepteur reçoit le message une seconde fois : **doublon**. On a garanti l'absence de perte au prix de la duplication.

Conséquence du choix « abandonner ». Dans le cas où le message avait bien été perdu, il ne sera jamais livré : **perte**. On a garanti l'absence de doublon au prix de la perte.

Conclusion. Les deux seules actions possibles mènent chacune à l'une des deux anomalies ; il n'existe donc pas de stratégie, fondée sur le seul accusé de réception, qui les évite toutes les deux. C'est ce qui donne les deux garanties classiques : *au moins une fois* et *au plus une fois*.

Comment l'idempotence contourne l'obstacle. On renonce à empêcher les doublons sur le réseau, et l'on rend leur *effet* nul. Chaque message porte un identifiant unique ; le récepteur tient la liste des identifiants déjà traités et ignore ceux qu'il reconnaît. Le résultat observable devient le même que si chaque message n'était arrivé qu'une fois. On n'a pas résolu l'impossibilité — des doublons circulent toujours — on l'a déplacée vers un endroit où elle est sans conséquence. C'est ainsi que l'on obtient en pratique l'effet « exactement une fois ».

E28

Correction des horodatages. Pour obtenir l'instant réel, on retire de chaque horodatage local le décalage de son horloge :

- Service A : horloge `+15 ms`, horodatage local 100 → instant réel `100 − 15 = 85 ms`
- Service B : horloge `−8 ms`, horodatage local 104 → instant réel `104 + 8 = 112 ms`
- Service C : horloge `+2 ms`, horodatage local 110 → instant réel `110 − 2 = 108 ms`

L'ordre apparent est faux. Les horodatages locaux suggéraient `A (100) < B (104) < C (110)`. Les instants réels donnent `A (85) < C (108) < B (112)` : **B et C sont inversés**. Un raisonnement fondé sur les horodatages bruts aurait conclu à l'envers.

Marge d'incertitude à annoncer. En situation réelle on ne connaît pas les décalages ; on connaît seulement leur amplitude maximale. Ici les décalages s'étalent de `−8` à `+15 ms`, soit une étendue de `15 − (−8) = 23 ms`. Deux événements dont les horodatages locaux diffèrent de moins de 23 ms **ne peuvent pas être ordonnés de façon fiable**.

Application. A et B diffèrent de 4 ms, B et C de 6 ms : tous ces écarts sont inférieurs à 23 ms, donc aucun des trois ordres n'est établissable sur la seule foi des horodatages. On ne peut conclure que si l'écart dépasse la marge.

Que faire. Soit obtenir l'ordre par un autre moyen — un numéro de séquence attribué par une source unique, ou une relation de causalité explicite entre les événements. Soit accepter l'incertitude et publier « ordre indéterminable à 23 ms près », ce qui est honnête et souvent suffisant.

E29

Stratégie 1 — publier à l'heure pile, sans attendre. La tranche 9 h–10 h est publiée à 10 h 00. *Latence :* nulle. *Justesse :* mauvaise — tous les retardataires sont perdus, et le total est systématiquement sous-évalué. *Perception :* le chiffre est stable, donc rassurant, mais faux ; c'est le pire cas, car l'erreur est invisible.

Stratégie 2 — attendre 20 minutes avant de publier. La tranche 9 h–10 h est publiée à 10 h 20. *Latence :* 20 minutes. *Justesse :* très bonne — la quasi-totalité des retardataires est incluse. *Perception :* le chiffre est définitif et ne bouge plus, mais l'utilisateur voit un tableau de bord qui a toujours 20 minutes de retard.

Stratégie 3 — publier tôt, puis corriger. On publie à 10 h 00 un résultat provisoire, mis à jour à mesure que les retardataires arrivent, et marqué comme définitif à 10 h 20. *Latence :* nulle pour la première valeur. *Justesse :* convergente — provisoire d'abord, exacte ensuite. *Perception :* le chiffre bouge après coup, ce qui déroute si on ne l'a pas annoncé ; d'où la nécessité d'un marqueur visuel explicite (« provisoire » / « consolidé »).

Recommandation. La stratégie 3, qui est aussi la plus répandue en production. Elle ne sacrifie ni la latence ni la justesse : elle déplace la difficulté vers l'interface, où elle se traite par un simple libellé. La stratégie 2 reste préférable quand le chiffre a une valeur contractuelle et ne doit jamais changer après publication — une facture, par exemple.

E30

Explication 1 (la plus probable) — le parallélisme est borné par le nombre de casiers. Si le flux ne compte que 8 partitions, huit machines suffisent à les consommer ; les huit autres restent inactives, quel que soit leur nombre. *Vérification :* comparer le nombre de partitions au nombre de consommateurs, et regarder si huit machines affichent une charge nulle. *Correction :* augmenter le nombre de partitions — en sachant que cela redéploie les clés et rompt l'ordre historique, donc en opération planifiée.

Explication 2 — une clé chaude déséquilibre la charge. Si une clé concentre l'essentiel du trafic, la machine qui la traite sature pendant que les autres attendent ; ajouter des machines ne déplace pas ce goulot, puisque `h(clé) mod n` envoie toujours cette clé vers une seule destination. *Vérification :* relever le débit par partition ; une partition très au-dessus des autres confirme. *Correction :* enrichir la clé par un suffixe, au prix de l'ordre à l'intérieur de cette clé, ou changer de clé si le calcul le permet.

Explication 3 — le goulot est ailleurs que dans le calcul. Si toutes les machines écrivent dans la même base ou appellent le même service externe, c'est cette ressource partagée qui plafonne ; distribuer le calcul ne sert alors à rien. *Vérification :* mesurer la part du temps passée en attente de la ressource commune ; si elle domine, le cluster n'est pas en cause. *Correction :* écrire par paquets, mettre en cache, ou partitionner la ressource elle-même.

Leçon générale. Ajouter des machines n'augmente le débit que si le travail est effectivement décomposable *et* effectivement décomposé. C'est l'étape de vérification de la décomposabilité, celle que l'on saute le plus souvent.